



中国研究生创新实践系列大赛

中国光谷·“华为杯”第十九届中国研究生

数学建模竞赛

学 校 上海海事大学

参赛队号 22102540295

1. 栗语璇

队员姓名 2. 马睿

3. 李姚娜

中国研究生创新实践系列大赛

中国光谷·“华为杯”第十九届中国研究生 数学建模竞赛

题目 COVID-19 疫情期间生活物资的科学管理问题建模与优化

摘要：

2020 年爆发的新冠肺炎疫情，时至今日国内和国际仍呈现多发频发态势。受国际环境更趋复杂严峻和国内疫情冲击明显的超预期影响，我国经济下行的压力进一步增大。面对复杂的防疫局势，建立规范化的疫情快速清零机制十分必要，特别是亟需解决居民生活物资发放的科学管理问题。生活物资发放是否科学，不仅影响隔离居民的基本生活，而且影响着疫情的二次传播。与此同时，应急物资管理本身就是一项复杂的系统工程，具有非线性、不确定性、动态性，给抗疫过程中的生活物资发放带来了巨大挑战，以长春市抗疫过程中的数据为依托，利用数据挖掘、数据分析、统计建模、运筹优化等方法探索一疫情下兼顾防疫安全与效率的科学物资管理方案既具现实意义又具研究价值。

问题一的主要目的探究生活物资的大规模流动方式对疫情的影响。对长春市 2022 年 3 月 26 日实行发放蔬菜包前后防控工作的效果进行判别与分析。本部分的研究分为三部分，包括可视化分析、统计分析（统计特征指标和时间序列统计特征指标）、SEIR 预测对比分析（4.2 节）。分析结果显示：（1）疫情的发展或控制与生活物资发放方式有关。长春市实行的蔬菜包方式是疫情期间为保障居民生活和减少疫情传播的有效发放方法；（2）在发放初期由于经验不足、备货力度不够、保供人员缺少、供货源不稳定等问题，使感染人数不减反升；（3）蔬菜包的发放时间间隔越大，能够减少人员接触频率，减轻保供工作人员的工作强度，减少疫情传播的风险。

问题二的主要目的解决生活物资投放点数量与位置问题。子问题一是要求讨论投放点数量的合理性并优化，子问题二和子问题三是在子问题一的基础上对应急物资供应链上游的大规模物资分拣场所与政府储备物资的规划。针对子问题一，本文首先对隔离人口数和生活物资投放点数量做 Person 相关系数，发现隔离人口数与生活物资投放点数量之间是极低的正相关，表明投放点的数量与分配方案不合理（5.2 节）。其次，采用层次分析法计算长春市生活物资投放点数量与分配的影响因素指标权重，综合评价投放点数量的合理性（5.2 节），发现投放点的数量严重冗余、分配不合理。在此基础上，修正附件 2 投放点数量得到各区优化调整的投放数量（表 5-8）。针对子问题二，本文首先使用平均法和数据挖掘估算出各区生活必需品的需求情况，依次制定政府储备物资安全库存（表 5-9）。针对子问题三，本文首先建立采用 K-means 聚类算法和遗传算法求解政府储备物资和大规模物资分拣场所的位置、数量规模与所服务的区域并提出最优的选址数量、规模（表 5-13）。

问题三的主要目的是优化生活物资的发放效果。子问题一是分析蔬菜包需求和发放规律，子问题二是评价和调整 4 月 10 日至 4 月 15 日蔬菜包供应方案。针对子问题一，本文采用文本挖掘技术，统计了 2022 年 3 月 26 日到 5 月 1 日九区总体和分区的每日蔬菜包供给量和需求量。分别从蔬菜包的供应规律、需求规律和供需规律三方面切入，运用描述性统计、正态性检验、相关性分析的方法结合新冠肺炎感染人数变化（6.2 节），多层次分

析蔬菜包的供需规律。针对问题二，本文采用 **TOPSIS 综合评价方法**，建立包含：各区本土感染人数和无症状感染者人数，各区小区栋数、小区户数和小区人口数、街道数等指标构成的评价体系，计算各指标权重，获得各区的得分与排名（表 6-5）。为评价蔬菜包供应方案的调整效果，本文利用**数据挖掘技术**统计出 4 月 10 日至 4 月 15 日，各区每天的蔬菜包接收数目和发放数目（表 6-9）。本文将蔬菜包浪费或缺货数目作为蔬菜包发放效果的评价指标，计算出各区蔬菜包供应的优化方案与调整幅度（表 6-10）。从各区调整效果对比（表 6-11）结果显示，采用此优化方案九个区的蔬菜包浪费量或缺货问题得到明显改善。

问题四的目的是为长春市做好大规模封控情况下居民生活物资有序发放预案。子问题一是对上游物资来源地进行合理选址，子问题二是对中游物资集散地进行合理分配与选址，子问题三是在考虑车辆限重的情况下，将“工作量”（工作量 = 运输里程 × 小区居民人数）的最小化合理规划末端配送路径以及配送量。针对子问题一，采用**射线法**“由点连面”判定交通路口节点的区域划分，同时能够过滤掉冗余的交通路口节点信息。在此基础上，采用**枚举法**在保留下来的交通路口节点中寻找与所有小区的“工作量”指标之和最小的位置。每个区重复上述方法便可得到各区的中心点坐标，即为九个上游物资来源地位置（表 7-2）。针对子问题二，本文采用轴辐射网络对中游的集散地进行选址和分配（表 7-4）。针对子问题三，本文考虑病毒传播和车量容量限制的路径优化模型（7.4 节）。鉴于此问题属于 NP-hard 问题，本文采用**大规模邻域算法（ALNS）**和**模拟退火算法（SA）**构成的**混合启发式算法**进行求解，最终得到由上游物资来源地-中游货物集散地-下游小区的多级有序物流网络（图 7-4）、最短配送路径以及合理的配送车辆数量与种类（7.5 节）和有序网络可视化（7.6 节）。最后对模型的改进与推广提出展望。

关键词：多层物流网络优化；选址与路径优化；数据挖掘；K-means 聚类；传染病模型；综合评价方法；相关性分析；时间序列分析；混合启发式算法；NP-hard；应急物流

目录

目录	3
一、问题重述	5
1.1. 问题背景	5
1.2. 问题提出	6
二、模型假设	6
三、符号说明	7
四、问题一分析与求解	7
4.1. 问题一分析：生活物资发放方式对疫情的影响分析	7
4.2. 评判发放蔬菜包对疫情的影响	8
4.2.1. 可视化分析	8
4.2.2. 统计分析	11
4.2.3. 预测对比分析	12
4.3. 结果分析	13
五、问题二分析与求解	14
5.1. 问题二分析：投放点合理性评价与优化	14
5.2. 子问题 1：投放点数量的合理性评价与优化——Person、层次分析法	15
5.2.1. 投放点数量的评价	15
5.2.2. 投放点数量的优化	19
5.3. 子问题 2：政府储备物资规划——平均法和数据挖掘	20
5.4. 子问题 3：大规模物资分拣场所选址与优化——K-means 聚类 and 遗传算法的综合算法	21
5.4.1. 基于 K-means 的大规模物资分拣场所选址	21
5.4.2. 基于遗传算法的选址优化模型	25
5.4.3. 计算结果	27
六、问题三分析与求解	28
6.1. 问题三分析	28
6.2. 子问题 1：蔬菜包的供应规律和需求规律——数据挖掘和数据分析	29
6.2.1. 蔬菜包的供应规律	29
6.2.2. 蔬菜包的需求规律	35
6.2.3. 蔬菜包的供需规律	39

6.3. 子问题 2：调整蔬菜包供应方案——TOPSIS 综合评价与优化.....	40
6.3.1. TOPSIS 介绍.....	40
6.3.2. 评价指标体系.....	42
6.3.3. 评价结果.....	42
6.3.4. 结果分析.....	45
七、问题四分析与求解.....	49
7.1. 问题分析.....	49
7.2. 上游物资来源地的选址——射线法、枚举法.....	50
7.2.1. 划分交通路口点的区域.....	50
7.2.2. 确定上游物资来源地在各区的选址.....	51
7.3. 中游集散点的辐射网络模型.....	51
7.4. 考虑病毒传播和车容量限制的路径优化模型.....	53
7.4.1 基于复杂网络的病毒传播模型.....	53
7.4.2 考虑节省人力，减少人员的直接、间接接触的配送路径优化模型.....	54
7.4.3 混合元启发式算法.....	56
7.5. 结果分析.....	59
7.6. 有序网络结果可视化.....	61
八、模型的改进与推广.....	62
8.1. 模型的优点.....	62
8.2. 模型的缺点.....	62
8.3. 模型的改进与推广.....	63
参考文献.....	64
附录 Python 程序.....	65
问题一算法程序.....	65
问题二算法程序.....	66
问题三算法程序.....	71
问题四算法程序.....	73

一、问题重述

1.1. 问题背景

2020 年爆发的新冠肺炎疫情，时至今日国内和国际仍呈现多发频发态势。2022 年以来由于新冠病毒变异株奥密克戎 BA.2 的高传播性、高传染性和高隐匿性，新一轮新冠肺炎疫情在我国多地散发，陆续发生多次大规模疫情爆发危机。凭借我国的制度优势，在疫情大规模爆发期间采取封闭式管理方式实现了疫情的快速清零。我国现行新冠肺炎疫情防控政策来看，涉及到封闭式管理方式的情况主要有两种，一是在静态管理情况下的“三个暂停”，即除参与防疫人员外，全市所有行政事业单位人员全部居家办公；除相关重点企业、保障民生的公共服务类企业外，所有经营性场所暂停营业；除具备条件的集散点、药店、医疗机构等外，其他商户暂停营业；出租车、网约车等车辆停运。以及三个“不”，即：指居民不聚集、不流动、不出门^[1]。二是对在风险区的划定与管控中划分为高风险区的居住小区（村）实行封控措施。实行“足不出户、上门服务”的政策，只有当连续 7 天无新增感染者，且第 7 天风险区域内所有人员完成一轮核酸筛查均为阴性时，才能降为低风险地区。连续 3 天无新增感染者时，才能降为低风险区^[2]。由此可见，封闭式管理的目的是减少城市人口流动，同时开展大规模核酸检测，实现以最快速度排查潜在感染者，从而切断疫情传播链。

但是，受国际环境更趋复杂严峻和国内疫情冲击明显的超预期影响，我国经济下行的压力进一步增大。面对复杂的防疫局势，从宏观调控的角度来看，需要政府高效统筹疫情防控和社会经济发展成效。从微观规制的角度来看，则急需建立规范化的疫情快速清零机制。特别是，亟需解决居民生活物资发放的科学管理问题。生活物资发放是否科学，不仅影响隔离居民的基本生活，而且很有可能在物资发放过程中造成疫情的二次传播。从学术界现有研究来看，学者们普遍认可了封闭式管理对快速抗击疫情的重要作用，认为采用封闭式应急管理方式是医院应对突发新型冠状病毒肺炎疫情的有效方式^[3]。但是，关于封闭管理下居民生活物资的发放问题的研究，目前主要聚焦在研讨居民生活物资的采购与配送方式，如刘志强等提出物业代购、电商配送等采购方式，陈镜羽等总结分析出“小区团购+集中取货”“门店共享+非接触式自提”“O2O 平台+共享配送员”等末端物流配送模式^[4]。

然而，由于疫情的超传播力往往超出预期，同时应急工作的时效性、安全性和不计成本性的特点，疫情期间生活物资的管理是一项复杂的系统工程，往往需要政府居于主导地位进行统筹管理。数据表明，在疫情期间政府完全有能力调动足够数量的生活物资，只是生活物资发放给小区居民的渠道不畅^[5]。从不同城市大规模疫情的控制工作实践来看，采用蔬菜包的做法是疫情期间为居民发放生活物资的有效方式。但是，实现蔬菜包科学供应十分困难，一是居民对新鲜蔬菜需求频率高，因而配送频率也较高，容易造成人员间密切接触^[6]。同时，米、面、油及肉等基本生活物资，由于保质期较长且主要由政府储备，因此其配送任务一般穿插到蔬菜包的配送过程。另外，水果也是居民们普遍需要的，虽属于非必须食品，但如果生产出来却无法销售也对经济发展不利。

因此，蔬菜无论在数量、频次、影响等方面都更突出，故本赛题聚焦蔬菜的科学供应问题。本文致力于研究在疫情封控期间，政府如果制定科学的物资管理方案，以高效应对疫情居民生活物资发放的管理问题，旨在全面提高疫情防控效率，同时节约人员投入和经费支出。

1.2. 问题提出

为制定科学的物资管理方案，保证蔬菜的科学供应，全面提高疫情防控效率的同时节约人员投入和经费支出。本文利用数学建模和数据挖掘技术依次解决如下问题。

问题一：生活物资发放方式对疫情的影响分析

结合附件 1 中所提供的长春市 COVID-19 疫情期间病毒感染人数数据及其它附件数据或团队能搜集到的数据对长春市实行发放蔬菜包前后效果进行判别与分析，为今后的防控工作提供理论依据。

问题二：生活物资投放点数量评价与优化、物资储备与选址规划

结合附件 2 中提供的当时长春市不同区域投放点数量分布结果。同时，结合附件 3 中和附件 4 中有关数据，讨论投放点数量的合理性，并通过数学建模进行适当的优化。另外，请充分考虑未来疫情、自然灾害等特殊事件，对于政府储备物资和大规模物资分拣场所的位置与数量规模进行合理规划，并提出最优的选址数量、规模及其潜在的备用场所位置。并将相关结果以表格的形式放置在正文中，需要包含选址位置、所属区域、选址半径、管辖范围小区个数、管辖范围内人口数等关键信息。

问题三：生活物资的供需关系分析与分配优化

结合附件 5 分析蔬菜包需求、发放规律，并根据附件 3 中的各小区位置与人口信息，评价并调整 4 月 10 日至 4 月 15 日蔬菜包供应方案。

问题四：生活物资发放的多层次物流网络规划与评价

在第二、三问的基础上，结合附件 3 给出的长春市街道和小区情况的表格，做出特殊时期保障居民生活物资供应的详细预案（有序网络图）。网络上游是各项物资来源（每个区选一个地点，参赛队可自行根据坐标选择），中游是各项物资的集散地（集散地数量自行选择，可以先按附件 2 设置，再调整优化），网络下游是长春市所有小区。物流是一个周期内各天通过网络各条边所运输的各项生活物资的数量（开始可以只考虑蔬菜，不同日期可以发送不同品种蔬菜以增加居民的蔬菜品种）。在开始时，网络的各条边可以不使用真实的街道，允许采用两点之间最短路连接。后来，可以选择少数行政区按真实街道选择路线，直至全市。考虑到特殊时期，所以节省人力是最重要的指标，同时希望减少人员的直接、间接接触。其中，工作量指标按运输里程与小区居民人数乘积计算。在完成有序网络图后需要进一步考虑用卡车运送物资，大卡车每辆可装 10 吨，小卡车每辆可装 4 吨，观察预案有无显著不同。要求在正文以图形或表格的方式精简地将相关结果表述出来，并请对照指标分析与评价所给出的发放预案的优势。

二、模型假设

假设 1：假设题目附件数据真实可靠，适用于做数学建模、数据挖掘和数据分析；

假设 2：假设题目附件数据中的缺失值对结果不会造成重大影响；

假设 3：假设每个投送点向各小区配送的速度相同，不考虑由车的性能、天气等因素造成的配送速度变化；

假设 4：假设每个投送点向各小区配送的过程中不考虑恶劣天气、自然灾害、交通拥堵、红绿灯等意外情况；

假设 5：由于题目中没有限制货车的派车成本、装卸货成本，因此假设上述成本忽略不计；

假设 6：假设货车数量充足且不会超载行驶；

假设 7：假设每辆货车工作时间不大于 10 小时；

假设 8：假设物资投放点的容量相同；

假设 9：假设蔬菜包隔夜存放第二天会变质不能再继续发放给隔离群众；

假设 10: 假设每区当日的蔬菜包接收量等价于每日每区的供应能力, 每区当日的蔬菜包发放量等价于每日各区隔离群众的需求量。附件五中所存在的每日发放量大于接收量的情况视为缺货, 反之视为浪费。

三、符号说明

No.	符号	含义
1	N	2022 年长春市总人口
2	λ_1	患病者每天有效接触的易感者的平均人数
3	λ_2	潜伏者每天有效接触的易感者的平均人数
4	δ	日发病率, 每天发病成为患病者的潜伏者占潜伏者总数的比例
5	μ	日治愈率, 每天治愈的患病者人数占患病者总数的比例
6	σ	传染期接触数 ($\sigma = \lambda_1/\mu$)
7	t_{End}	预测日期长度
8	i_0	患病者比例的初值
9	e_0	潜伏者比例的初值
10	s_0	易感者比例的初值
11	Y_0	微分方程组的初值
12	$\bar{G}$	平均增长率
13	R	极差
14	s	标准差
15	θ_i	交付的病毒传播风险
16	N_p	小区居民人数
17	H	需要选取的集散点个数
18	Q	集散点容量的集合
19	EO, EI	起始路线编号和终止路线编号
20	$F(i)$	路线编号终点的前序节点集合
21	$A(i)$	路线编号起点的后序节点集合
22	P	需要选取的集散点数量
23	K	车辆集合

四、问题一分析与求解

4.1. 问题一分析: 生活物资发放方式对疫情的影响分析

根据问题一要求, 本题需要对长春市实行发放蔬菜包前后效果进行判别与分析, 为今后的防控工作提供理论依据。首先, 从定性的角度, 依据附件 1 中所提供的长春市 COVID-19 疫情期间 2022 年 3 月 4 日到 2022 年 5 月 23 日的全市和九个区的新增本土感染者人数、新增无症状感染者人数, 分别绘制时间序列折线图, 通过折线随时间的波动变化和发展趋势来对比 3 月 26 日开始发放蔬菜包前后新增本土感染者人数和新增无症状感染者人数的变化。其次, 计算极差、标准差、均值、Hurst 指数、平均增长率 (初始值到峰值) 等指标, 对比蔬菜包发放前后新冠肺炎疫情时间序列统计特征的变化。最后, 参考现有文献中新冠病毒奥密克戎亚型变异株 BA.2 的传染率^[7]、恢复率^[8]、潜伏期^[7], 运用 SEIR 传染病预测模型, 将 2022 年 3 月 25 日的新增本土感染者人数作为感染者的初始人数, 将同日的新增

无症状感染者人数作为潜伏者的人数，通过查询长春市卫生健康委员会网站,收集同日的累计治愈人数作为治愈者的初始人数，将全市人口数减去感染者人数、潜伏者人数和治愈者人数后得到易感染者的人数。由此预测在不采用蔬菜包物资发放方式的情况下，2022年3月26日至2022年5月23日长春市每天新增本土感染人数和无症状感染者人数。从而，将此预测结果与附件1的数据进行对比，量化评价蔬菜包对疫情影响。

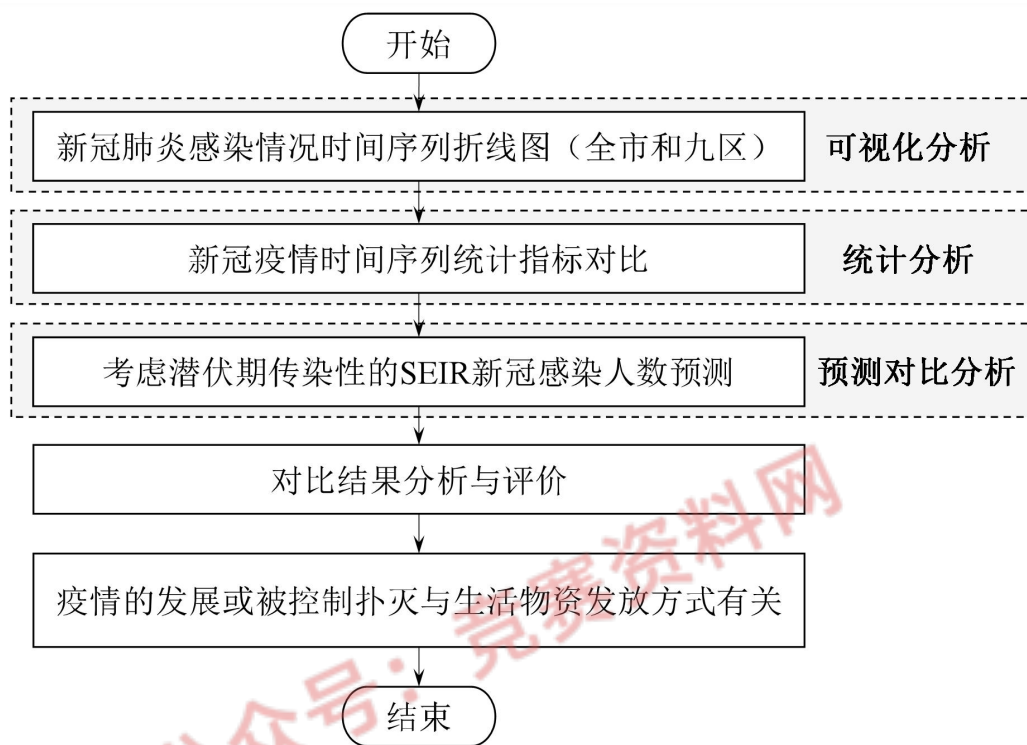


图 4-1. 问题一的思路流程图

4.2. 评判发放蔬菜包对疫情的影响

4.2.1. 可视化分析

从定性的角度，依据附件1中所提供的长春市 COVID-19 疫情期间 2022 年 3 月 4 日到 2022 年 5 月 23 日的全市和九个区的新增本土感染者人数、新增无症状感染者人数，分别绘制时间序列折线图，通过折线随时间的波动变化和发展趋势来对比 3 月 26 日开始发放蔬菜包前后新增本土感染者人数和新增无症状感染者人数的变化。具体分析如下：

首先，计算长春市自 2022 年 3 月 4 日至 2022 年 5 月 23 日新冠肺炎疫情发展的总体情况，包括感染人数、无症状感染者人数和总计感染人数，如图 4-2、图 4-3 和图 4-4 所示。从图可看出自 2 月 26 日开始发放蔬菜包后，随后两天内新增无症状感染者人数和总计新增人数（累计新增人数）都呈倒“V”下降趋势，新增本土感染人数也有所下降，但自 3 月 28 号后，新冠疫情总体呈上升态势并在 4 月 2 日达到顶峰，这可能是由于 3 月 29 日开始长春市东北亚粮油、海吉星两大蔬菜批发市场突发疫情而临时关闭^[9]，加上全市疫情形势严峻，蔬菜包的备货力度不足，保供人员上岗严重不足，导致市民大部分市民开始采取网上订购的方式，这一定程度上增加了在网购蔬菜的配送过程的感染风险，体现了采用蔬菜包发放方式的必要性。

随着长春市努力尽快释放蔬菜包的末端配送队伍，强化配送力量、多渠道稳定蔬菜包货源、多点位通物流和多层次保供应，自 4 月 28 日开始有序恢复生产生活秩序，全市疫

情形势逐步向好。并在 5 月 12 日首次实现了确诊病例和无症状感染者的“零”新增，除 5 月 14 日发现 1 例集中隔离管控阳性人员外，截至 5 月 23 日，连续 11 天无新增确诊病例和无症状感染者。这表明蔬菜包是疫情期间为居民发放生活物资的有效方式，能够有效的保证居民生活蔬菜肉蛋奶等基本生活需求，同时减少了配送频率从而降低了因频繁的配送造成的感染风险。但是，在蔬菜包发放的初期由于经验不足导致蔬菜包备货力度不够、保供人员缺少、供货源不稳定等问题，使感染人数有所增加。随着政府努力释放保供力量，使蔬菜包较为有序地发放一定程度上帮助实现短期清零的抗疫目标。

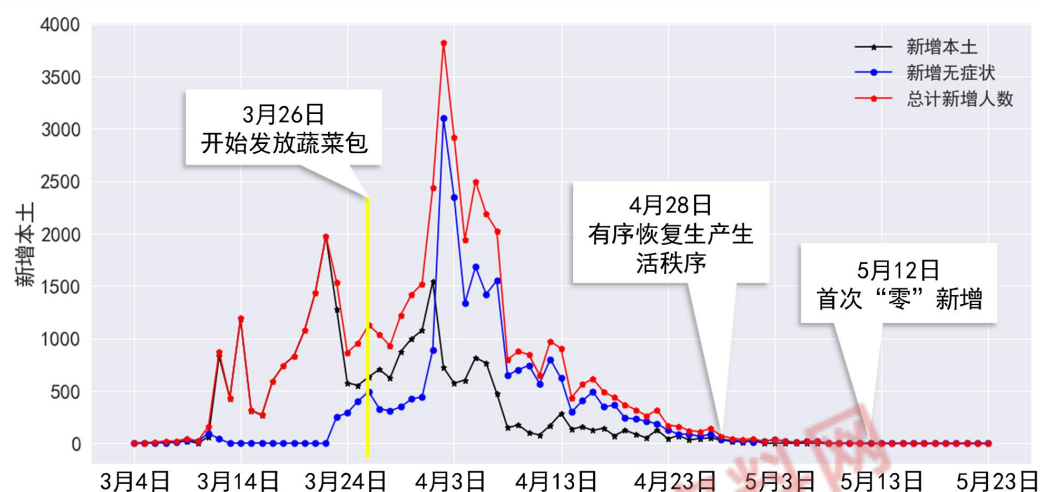
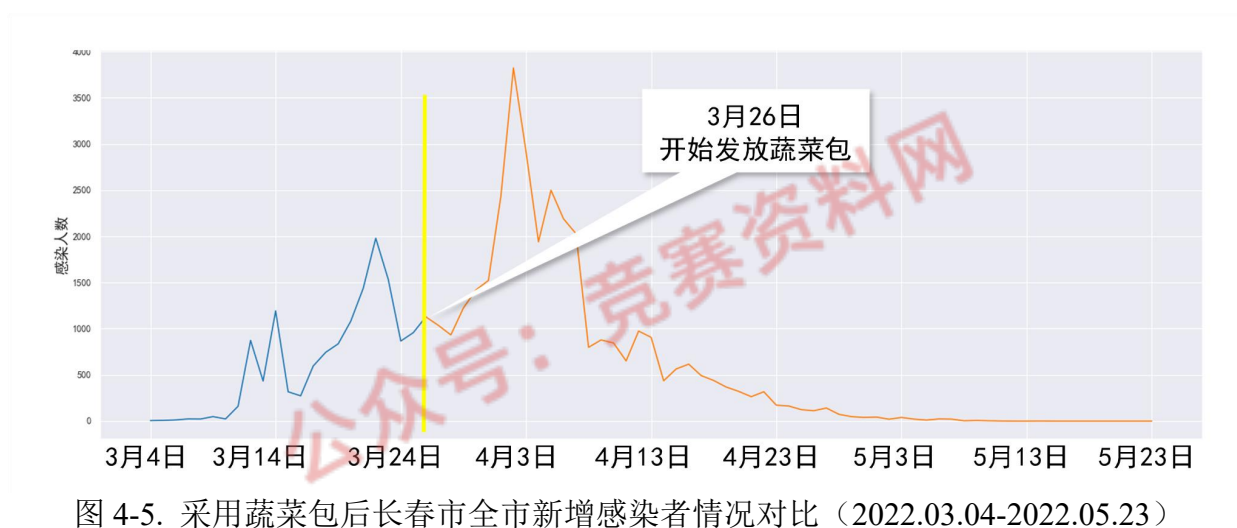
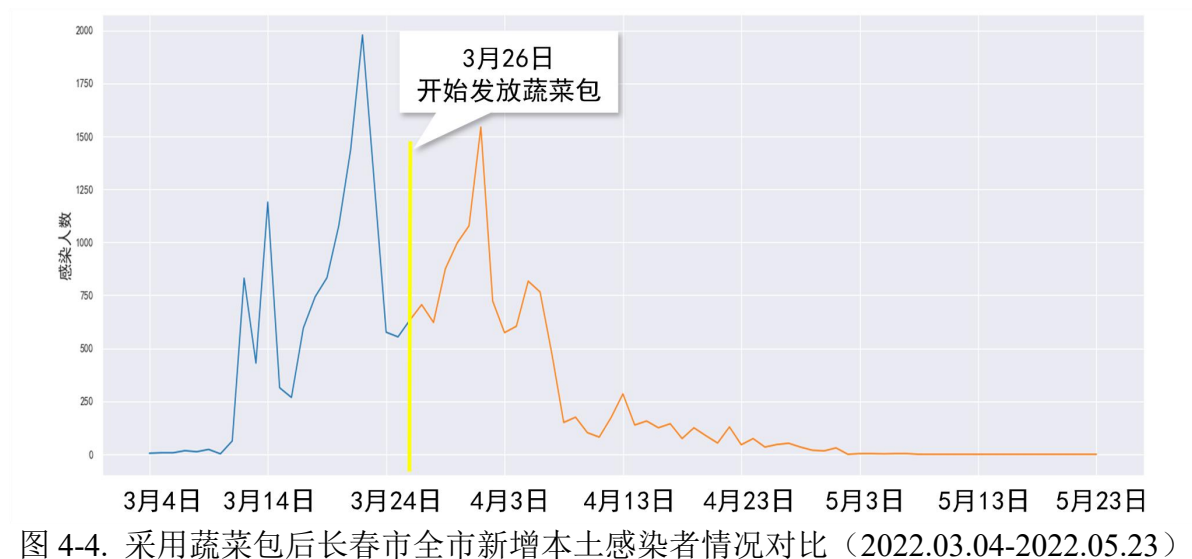


图 4-2. 长春市全市新冠肺炎疫情发展的总体情况（2022.03.04-2022.05.23）



图 4-3. 采用蔬菜包后长春市全市新增无症状感染者情况对比（2022.03.04-2022.05.23）



其次，分别计算长春市九个区 2022 年 3 月 4 日至 2022 年 5 月 23 日开始新增本土感染者人数和新增无症状感染人数的变动趋势，分别如图 4-6 和图 4-7 所示。由此可看出，自 3 月 26 日开始发放蔬菜包，长春市九区的新增本土病例有所下降，虽然 4 月份疫情有所反弹，但整体最终趋势趋于零，体现了蔬菜包对减少疫情传播的有效性。

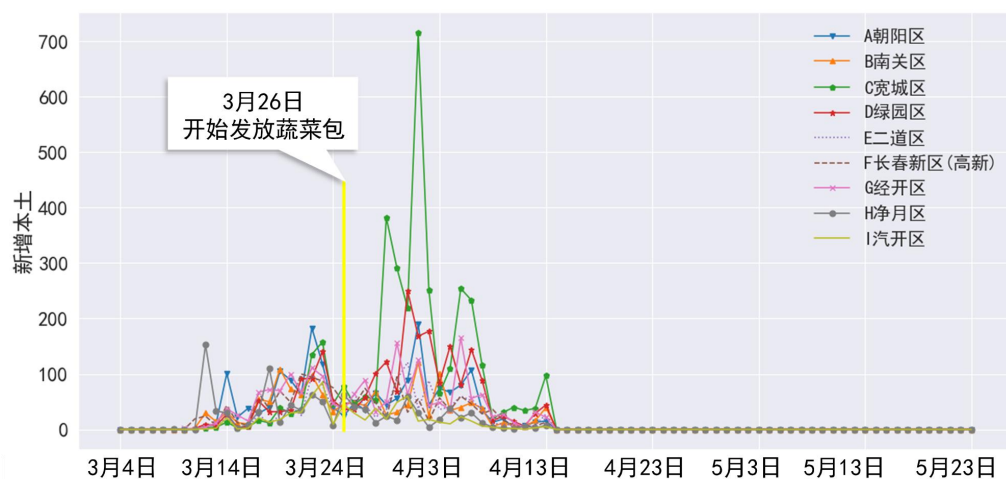


图 4-6. 长春市九区新冠肺炎新增本土病例的情况（2022.03.04-2022.05.23）

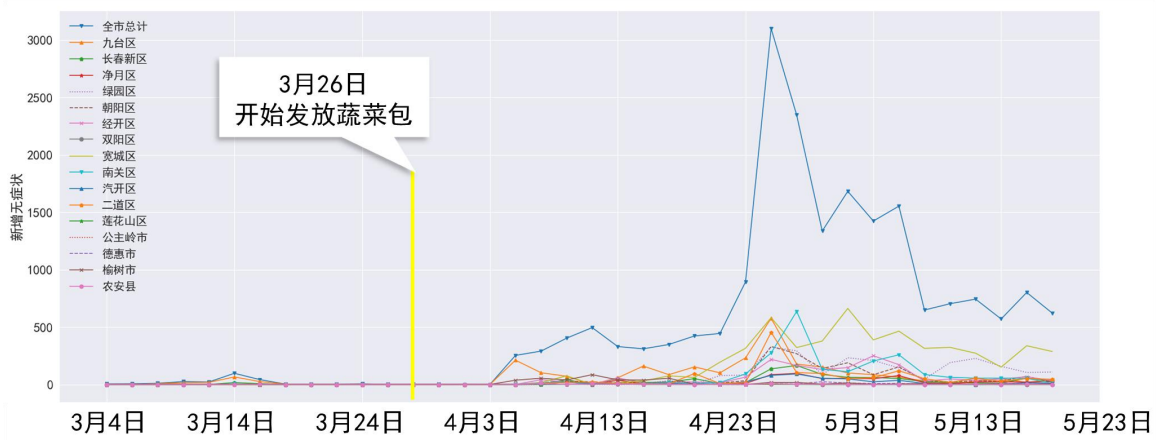


图 4-7. 长春市九区新冠肺炎新增无症状病例的情况（2022.03.04-2022.05.23）

4.2.2. 统计分析

各统计指标的介绍如下：

(1) 统计特征指标

1) 均值：是表示数据集中趋势的量数，用于描述新增确诊病例的时间序列集中趋势。其计算公式如式（4-1）所示。

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n} \quad (4-1)$$

2) 极差：是标志值变动的最大范围，也是测定标志变动的最简单的指标。新冠疫情时间序列统计指标中发挥着评价新增确诊病例的离散度的作用，其计算公式如式（4-2）下所示：

$$R = x_{\max} - x_{\min} \quad (4-2)$$

3) 标准差：是描述时间序列全局结构的统计量，用于评价新增确诊病例时间序列的变化程度^[10]，其计算公式如式（4-2）所示。其中， i 表示新增确诊病例时间序列的时长； x_i 表示每日新增确诊病例的均值； n 代表每日新增确诊病例总时长。

$$s = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}} \quad (4-3)$$

(2) 时域特征指标

*Hurst*指数常用于分析时间序列的分形特征和长期记忆过程，目前在时间序列变化趋势的持续性或反持续性强度判断方面得到广泛引用。新冠疫情的*Hurst*指数可以定量描述疫情将来的发展趋势与现有趋势的关系。*Hurst*指数越大表示时间序列趋势的延续性最高。

本文采用 R/S 分析即重新标度的极差分析（rescaled range analysis），计算 *Hurst* 指数。计算公式如式（4-4）所示。其中 H 表示新增确诊病例时间序列的 *Hurst* 指数； R 表示每日新增确诊病例每个时间序列段的最大距离； s 表示每日新增确诊病例每个事件段的标准差； K 为正常人口数； t 表示每日新增确诊时间序列段的时间窗口长度。当 *Hurst* 指数为 0.5 时，时间序列记忆表现为随机游走形式，无法判定走向；当 *Hurst* 指数为 $[0.5, 1]$ 时，时间序列数据存在长期持续性。当 *Hurst* 指数为 $[0, 0.5]$ 时，时间序列数据为反持续性。

$$\log\left(\left(\frac{R}{s}\right)_t\right) = \log(K) + H \log(t) \quad (4-4)$$

(3) 疫情特征指标

平均增长率，可以用来反映研究期内从发现病例的第一天到新增病例达到最大值的速度，可以用来表示疫情的蔓延速度^[10]，同时也能用于反映国家的防控措施是否有效。计算公式如式(4-5)所示。其中， N 代表每日新增确诊病例时间序列初始值到峰值的时长； x_{max} 代表每日新增确诊病例的最大值。 x_0 代表第一个不为零的每日新增确诊病例数。

$$\bar{G} = \sqrt[N]{\frac{x_{max}}{x_0}} - 1 \quad (4-5)$$

统计分析的计算结果如表 4-1 所示。从长春市新冠肺炎新增本土病例的均值来看，在 3 月 26 日使用蔬菜包之前每日平均新增本土病例为 556.59 例，而使用蔬菜包之后平均每日新增本土病例为 220.28 例，使用蔬菜包后每日平均新增本土感染人数下降 60.42%。从极差指标来看，在 3 月 26 日使用蔬菜包之前每日新增本土病例的最大值为 1997 例，最小值为 2，极差为 1977。而使用蔬菜包之后最大值为 1544，最小值为 0，极差为 1544。使用蔬菜包后，新增本土感染人数极大值降低了 21.98%，极小值出现了 0 表明疫情趋缓出现了“零”增长，极差降低了 21.9%。从 Hurst 指数来看，指数从[0.5,1.0]区间变为[0,0.5]区间，表明疫情的持续性有所减弱。从平均增长率来看，使用蔬菜包之前每日平均新增本土病例的平均增长率为 37%，使用蔬菜包之后平均增长率为 16%，表明使用蔬菜包之后疫情的蔓延速度明显减缓。

表 4-1. 新冠疫情时间序列统计指标

NO.	统计特征分类	统计指标	计算结果		结果对比
			3 月 4 日-3 月 25 日	3 月 26 日-5 月 23 日	
1	统计特征	均值 ($\bar{x}$)	556.59	220.28	-60.42%
2		极差 (R)	1977	1544	-21.90%
3		标准差 (s)	563.76	345.37	-38.74%
4		极大值 (x_{max})	1979	1544	-21.98%
5		极小值 (x_{min})	2	0	-2
6	时域特征	Hurst 指数	0.55	0.413	-24.90%
7	疫情特征	平均增长率 ($\bar{G}$)	0.37	0.16	-56.76%

4.2.3. 预测对比分析

运用 SEIR 传染病预测模型，将 2022 年 3 月 25 日的新增本土感染者人数作为感染者的初始人数，将同日的新增无症状感染人数作为潜伏者的人数，通过查询长春市卫生健康委员会网站，收集同日的累计治愈人数作为治愈者的初始人数，将全市人口数减去感染者人数、潜伏者人数和治愈者人数后得到易感染者的人数。模型参数如表 4-2 所示，计算结果如图 4-8 所示。预测在不采用蔬菜包物资发放方式的情况下 2022 年 3 月 26 日至 2022 年 5 月 23 日长春市每天新增本土感染人数累计 298.69 万人约占全市总人口的 35%，无症状感染者累计达 170.68 万人约占全市总人口的 20%。与附件 1 的数据进行对比，采用蔬菜包后 2022 年 3 月 26 日至 2022 年 5 月 23 日长春市累计新增本土感染人数降低了 77.5%，无症状感染者降低了近 30%。由此可见，使用蔬菜包的方式能够有效减少感染新冠病毒的风险，减少病毒的传播，是实现快速“清零”和保障隔离居民生活物资的有效方式。

表 4-2. 考虑潜伏期传染性的 SEIR 新冠感染人数预测模型参数

NO.	符号	含义	数值
1	N	2022 年长春市总人口	8534000

2	λ_1	患病者每天有效接触的易感者的平均人数	1.00
3	λ_2	潜伏者每天有效接触的易感者的平均人数	0.25
4	δ	日发病率，每天发病成为患病者的潜伏者占潜伏者总数的比例	0.05
5	μ	日治愈率，每天治愈的患病者人数占患病者总数的比例	0.05
6	σ	传染期接触数 ($\sigma = \lambda_1/\mu$)	20
7	t_{End}	预测日期长度	60

表 4-3. 采用蔬菜包和不采用蔬菜包的新冠感染人数对比

2022 年 3 月 26 日至 2022 年 5 月 23 日			
	采用蔬菜包 (万人)	不采用蔬菜包 (万人)	结果对比
累计新增本土感染人数	1.28	5.69	-77.5%
累计新增无症状感染人数	2.23	8.68	-28.92%

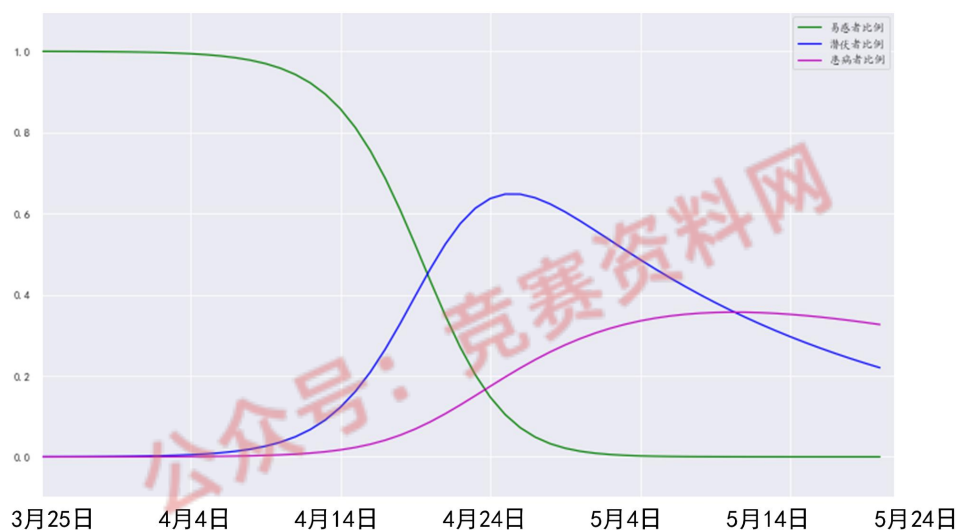


图 4-8. SEIR 新冠感染人数预测结果可视化

4.3. 结果分析

经过可视化分析、统计分析和预测对比分析，结果显示：疫情的发展或被控制扑灭与生活物资发放方式有关，发放蔬菜包的做法是疫情期间为居民发放生活物资的有效方式，能够有效的保证居民生活蔬菜肉蛋奶等基本生活需求，蔬菜包的发放时间间隔越大，能够减少人员接触频率，减轻保供工作人员的工作强度，减少疫情传播的风险。

但是，在蔬菜包发放的初期由于经验不足导致蔬菜包备货力度不够、保供人员缺少、供货源不稳定等问题，使感染人数有所增加。未来，为保证蔬菜包对疫情的正向控制作用，一是需要政府努力释放保供力量，包括释放蔬菜包的末端配送队伍，强化配送力量、多渠道稳定蔬菜包货源、多点位通物流和多层次保供应。二是蔬菜包等生活物资时一定要采取无接触配送的方式，不要和配送人员接触。三是蔬菜包的外包装一定要消毒处理，尽量在静置 2 至 3 个小时以后再取用，并且在冲洗时也应该佩戴一次性手套并一用一扔。

五、问题二分析与求解

5.1. 问题二分析：投放点合理性评价与优化

根据问题二要求，此题可以分为两个子问题。子问题一是要求讨论投放点数量的合理性，并通过数学建模进行适当的优化，此问题可看作是数学建模中的综合评价问题，可以通过建立评价指标体系及相应的权重系数，构造综合评价模型，计算各系统的综合评价价值并得出综合评价结果，此外还需结合附件 3 和附件 4 的数据对评价对象进行优化。子问题二是要求充分考虑未来疫情、自然灾害等特殊事件，对于政府储备物资和大规模物资分拣场所的位置与数量规模进行合理规划，并提出最优的选址数量、规模及其潜在的备用场所位置。两题均要求将相关结果需要包含选址位置、所属区域、选址半径、管辖范围小区个数、管辖范围内人口数等关键信息。子问题 2 分解两小问，第一小问是政府储备物资的规划即对物资的库存优化，第二小问是对大规模物资分拣场所投放点进行选址规划。因此，子问题 2 可看作是在子问题 1 的基础上对投放点上游的仓库和库存进行合理规划的问题。

针对子问题一，首先对附件 2 的隔离人口数和生活物资投放点数量做 Person 相关系数分析结果为 0.136，说明各区隔离人口数与生活物资投放点数量存在极低的正相关关系，隔离人口数越多投放点数量不一定多。同时也发现附件二物资投放点数量的不合理问题包括：没有考虑隔离人口数、交通路口和交通线路密度、投放点选址不明确、数据为截面数据不能体现投放点的数量随日期的动态变化情况。其次，采用层次分析法分析长春市生活物资投放点数量分布的影响因素，综合评价附件 2 中投放点数量的合理性。结合层次分析法的得分和修正附件 2 投放点数量得到各区合理的投放数量。

针对子问题二的第一小问政府储备物资的规划问题，首先采用平均法对附件 2 的投放点服务能力进行估算，得到平均每个投放点可以服务 0.207 万人。其次，依照附件 4 中《长春市重点民生商品供应情况表》以及商务部印发的《生活必需品市场供应保障工作手册》

（2021 版）中规定，每人每日消费粮食 325 克，食用油 30 克，蔬菜 400 克，肉类 115 克以及附件 3 中的小区人口数量，运用平均法估算出了每个区的生活必需品需求情况作为储备物资的安全库存。针对子问题二的第二小问大规模物资分拣场所进行选址规划的问题，建立采用 K-means 聚类算法和遗传算法求解政府储备物资和大规模物资分拣场所的位置、数量规模与所服务的区域，并提出最优的选址数量、规模及其潜在的备用场所位置等关键信息。

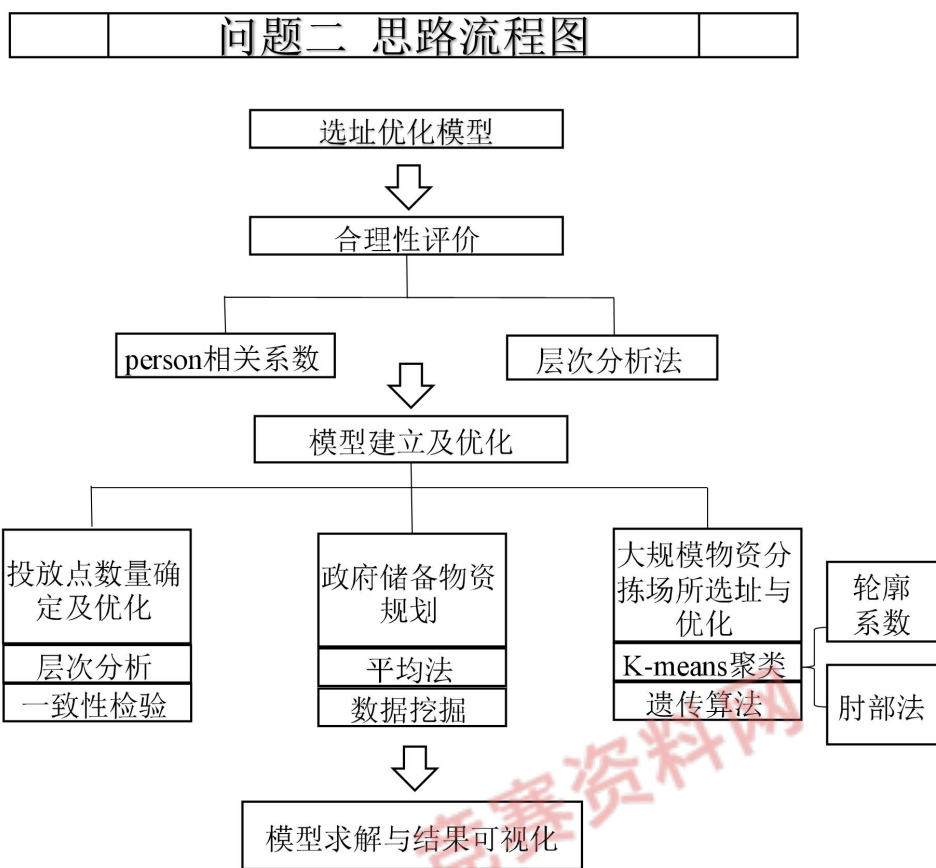


图 5-1. 问题二思路流程图

5.2. 子问题 1：投放点数量的合理性评价与优化——Person、层次分析法

5.2.1. 投放点数量的评价

针对子问题一，首先对附件 2 的隔离人口数和生活物资投放点数量做 Person 相关系数分析结果为 0.136，表 5-1 相关程度度量表来看，这表明各区隔离人口数与生活物资投放点数量存在极低的正相关关系，隔离人口数越多投放点数量不一定多。这与实际生活经验相违背，体现了附件 2 中投放点数量设置的不合理性。因此，本文假设隔离人口数对生活物资投放点数量要求越大。

表 5-1. 相关程度度量表

相关性	相关系数
极强相关	0.8~1.0
强相关	0.6~0.8
中相关	0.4~0.6
弱相关	0.2~0.4
极弱或无相关	0.0~0.2

采用层次分析法分析长春市生活物资投放点数量分布的影响因素，综合评价附件 2 中投放点数量的合理性。层次分析法（Analytic Hierarchy Process, AHP）的基本思想是将问题层次化，依照问题的性质和总目标，将其划分为不同的组成要素，并依据这些要素间的

隶属关系，进行不同层次的聚集组合得到多层次分析结构模型，从而将问题进行优劣比较排序。层次分析法的基本步骤如下所示。

表 5-1. 层次分析法的操作步骤

步骤	操作
Step 1	找准各因素之间的隶属度关系，建立递阶层次结构
Step 2	构造判断矩阵并赋值
Step 3	层次单排序(计算权向量)与检验(一致性检验)
Step 4	层次总排序(组合权向量)与检验(一致性检验)
Step 5	评价结果分析

(1) 建立层次结构模型

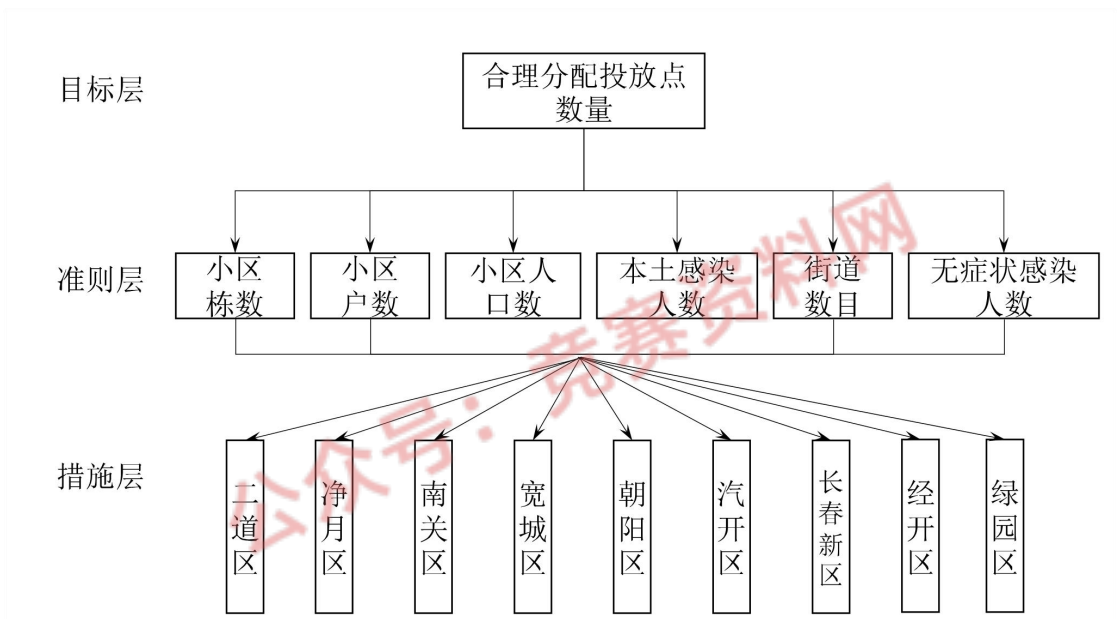


图 5-2. 投放点数量评价的层次结构模型

本文构建的蔬菜包投放点数量合理性评价的层级分析法结构模型如图 5-2 所示。具体做法是，依据附件 1 汇总 4 月 10 日到 4 月 15 日的各区本土感染人数和无症状感染者人数，提取并统计附件 3 中的各区的小区栋数、小区户数（户）和小区人口数（人），此外我们考虑到交通状况也是影响物资供应方案的重要因素，因此我们根据附件 3 中的街道编号统计出了各区的街道数量。由此得到的评价指标数据如表 5-2 所示。

表 5-2. 蔬菜包投放点数量合理性评价的层次分析指标数据

时间	区	评价指标					
		小区栋数	小区户数 (户)	小区人口 数 (人)	街道数 目 (个)	新增本土 感染人数 (例)	新增无症 状感染人 数 (例)
4 月 10	朝阳区	3740	219874	548179	69	3	32
4 月 10	南关区	2336	183877	462146	68	7	55
...	...	...	...	...	...	...	...

4 月 11	宽城区	1524	128899	306143	58	34	151
4 月 11	绿园区	2198	155261	372541	57	6	158
...	...	...	...	...	...	...	...
4 月 12	二道区	1820	166428	405784	50	8	51
4 月 12	长春新区	2116	141780	338274	29	6	55
...	...	...	...	...	...	...	...
4 月 13	净月区	1507	85748	204056	59	7	26
4 月 13	汽开区	1171	81675	193462	54	5	9
...	...	...	...	...	...	...	...
4 月 14	朝阳区	3740	219874	548179	69	7.2	10
4 月 14	南关区	2336	183877	462146	68	18.72	5
...	...	...	...	...	...	...	...
4 月 15	净月区	1507	85748	204056	59	4	17.81
4 月 15	汽开区	1171	81675	193462	54	10	6.165

（2）构建的判断矩阵

表 5-3. 层次分析法的判断矩阵

指标	小区栋数	小区户数 (户)	小区人口数(人)	街道数目	新增本土 感染者	新增无症状 感染者
小区栋数	1	1	1	1	0.333	0.333
小区户数(户)	1	1	1	1	0.333	0.333
小区人口数(人)	1	1	1	1	0.333	0.333
街道数目	1	1	1	1	0.2	0.2
新冠疫情人数	3	3	3	5	1	1
无症状感染者人数	3	3	3	5	1	1

（3）AHP 层次分析结果

根据表 5-4 的 AHP 层次分析结果，获得各项指标的权重值，基于层次分析法（方根法）的权重计算结果显示，小区栋数的权重为 9.637%，小区户数（户）的权重为 9.637%，小区人口数(人)的权重为 9.637%，街道数目的权重为 8.128%，新冠疫情人数的权重为 31.48%，无症状感染者人数的权重为 31.48%。

表 5-4. AHP 层次分析结果

AHP 层次分析结果				
项	特征向量	权重值(%)	最大特征根	CI值
小区栋数	0.693	9.637	6.044	0.009
小区户数(户)	0.693	9.637		
小区人口数(人)	0.693	9.637		
街道数目	0.585	8.128		

新冠疫情人数	2.265	31.48
无症状感染者人数	2.265	31.48

(4) 一致性检验

层次分析法的计算结果显示，最大特征根为 6.044，根据 RI 表查到对应的 RI 值为 1.25，因此 $CR = CI/RI = 0.007 < 0.1$ ，因此通过了一次性检验。具体数据如表 5-5 所示。

表 5-5. AHP 的一致性检验结果

最大特征根	CI 值	RI 值	CR 值	一致性检验结果
6.044	0.009	1.25	0.007	通过

(5) 计算各区的得分与投放点数量

假设投放点的总数不变通过各区的综合得分与得分占比可以计算出调整后的投放点数量。但是，题目给出的附件 2 中各区隔离人口数和生活物资投放点数量来看（如表 5-6 所示），总隔离人口为 322 万人，则通过平均法可以估算出每个投放点的服务能力为 0.21 万人。考虑到我国党和政府始终坚持人民至上的根本立场，在疫情防控中发挥制度优势，始终将保障人民生命安全和身体健康放在第一位。因此在设置投放点数量是往往是保持一定程度的冗余度。同时，在抗疫初期由于经验不足，在设置物资投放点时往往也具有一定的盲目性。因此，本文参考附件 3 提供的小区人口数可以汇总得到各区的小区人口数如表 5-7 所示，则九个区内的小区人口总数为 301.98 万人。即便是九个区全部小区内的居民都隔离，且每个投放点的服务能力最小为 0.21 万人的情况下，大约只需要 1438 个投放点。因此，**本文将投放点的总数修正 1300 个**。结合各区的得分情况，计算出较为合理的投放点分配数量，计算结果如表 5-8 所示。

表 5-6. 附件 2：长春市 9 个区隔离人口数量与生活物资投放点数量

区域名称	隔离人口数（万人）	生活物资投放点数量
朝阳区	57.8	94
南关区	48.9	261
宽城区	32.6	181
绿园区	38.5	470
二道区	42.6	9
长春新区(高新)	36.8	215
经开区	20.3	37
净月区	22.8	279
汽开区	21.7	10
总计	322	1556

表 5-7. 长春市九个区的小区数据汇总

所属区域	小区栋数	小区户数（户）	小区人口数（人）
二道区	1820	166428	405784
净月区	1507	85748	204056
南关区	2336	183877	462146
宽城区	1524	128899	306143

朝阳区	3740	219874	548179
汽开区	1171	81675	193462
经开区	789	76909	189240
绿园区	2198	155261	372541
长春新区(高新)	2116	141780	338274

数据来源：根据附件 3 整理得到。

表 5-8. 优化后的投放点数量

所属区域	得分	总得分	占比	修正的附件 2 投放总量 (个)	优化结果 (个)
二道区	56123.43464	422240.8	0.132918083	1300	172.7935084
净月区	28594.39379		0.067720588		88.03676493
南关区	63610.77547		0.15065048		195.8456237
宽城区	44877.04046		0.106283057		138.1679741
朝阳区	75464.63773		0.178724183		232.341438
汽开区	26983.50788		0.0639055		83.07714994
经开区	26819.0921		0.063516111		82.57094465
绿园区	52546.75656		0.124447376		161.7815887
长春新区（高新）	47221.13722		0.111834621		145.3850076

5.2.2. 投放点数量的优化

综合考虑各区的小区人口数、小区户数、街道数目、新冠肺炎本土感染人数以及无症状感染者人数，运用层次分析法结合修正后的投放点总量，计算得到各区的物资投放点的数量。各区优化前后的对比情况如图 5-3 所示。其中调整幅度最大的是二道区（1819.93%），其次是汽开区（730.77%）、再者是经开区（123.16%）。调整方案中，较原来需要增设投放点数量最多的是朝阳区（需要增加 78.79 个），较原来需要减少投放点数量最多的是绿园区（需要减少 308.22 个）。

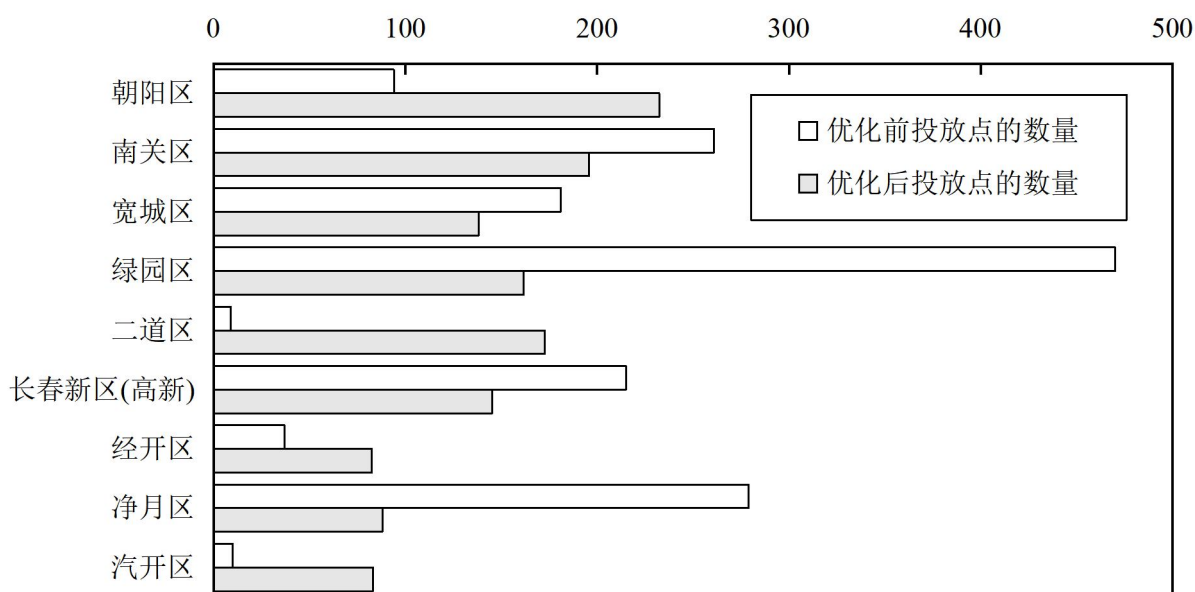


图 5-3. 投放点数量的优化前后对比图

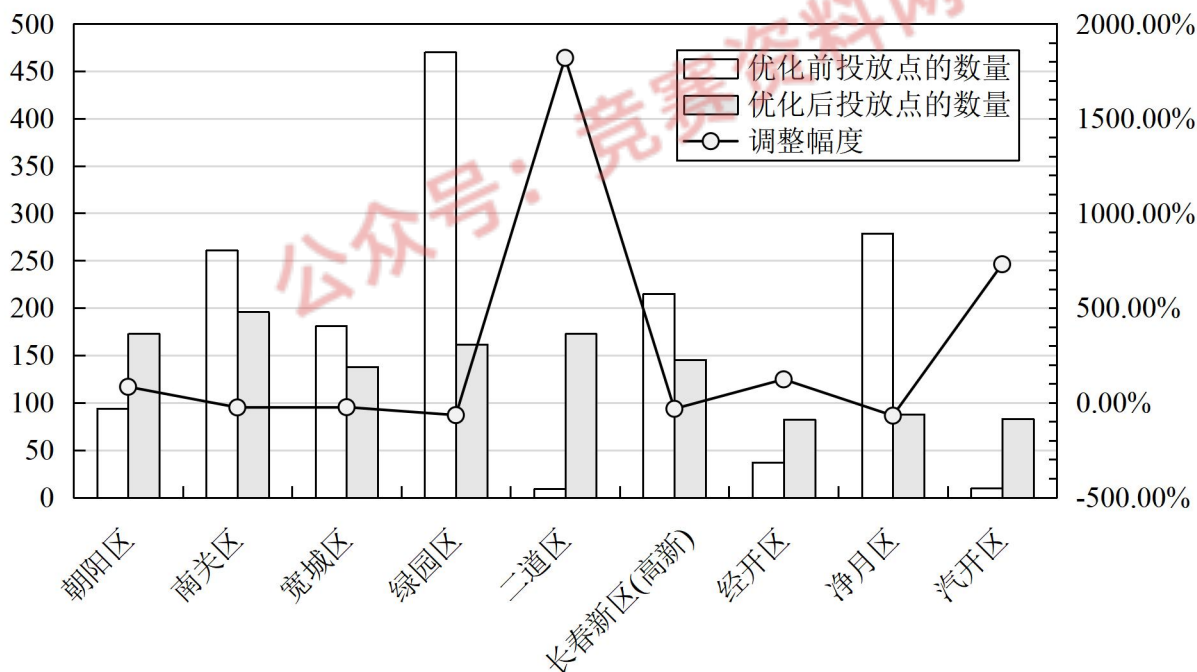


图 5-4. 投放点数量的调整幅度变化图

5.3. 子问题 2：政府储备物资规划——平均法和数据挖掘

通过对附件 4 中生活物资相关数据集文件，我们可以得知在 3 月 18 日到 5 月 23 日的物资供应情况，按照商务部印发的《生活必需品市场供应保障工作手册》（2021 版）中规定，每人每日消费粮食 325 克，食用油 30 克，蔬菜 400 克，肉类 115 克。按长春市城区 450 万人口计算，长春市城区每日消耗粮食 1462 吨、食用油 135 吨、蔬菜 1800 吨、肉类 518 吨。通过该项数据以及附件 3 中的小区人口数量我们可以大致估算出每个区的生活必需品需求情况。根据国家发改委、商务部《新冠肺炎疫情防控生活物资保障工作指南（试

行)》(发改办运行〔2021〕1053号)的通知、省疫情防控领导小组办公室《关于印发进一步做好新冠肺炎疫情防控生活物资保障工作制度意见的通知》(吉防办明电〔2022〕81号)明确的各地重要民生商品储备应达到如下标准:成品粮油达到15天(含)以上,肉类方面不低于3天,蔬菜不低于7天。通过图5-5蔬菜库存存储情况可以看出自3月25日起蔬菜的储备量低于标准储备量,其中3月25日东北亚粮油1.1万蔬菜封存可能是导致存储量低于标准存储量的原因之一。3月28日海吉星虽然关闭,但蔬菜渠道仍然畅通。并没有对重要民生商品储备情况造成太大影响。本题的政府储备物资规划的安全库存结果如表5-9所示。

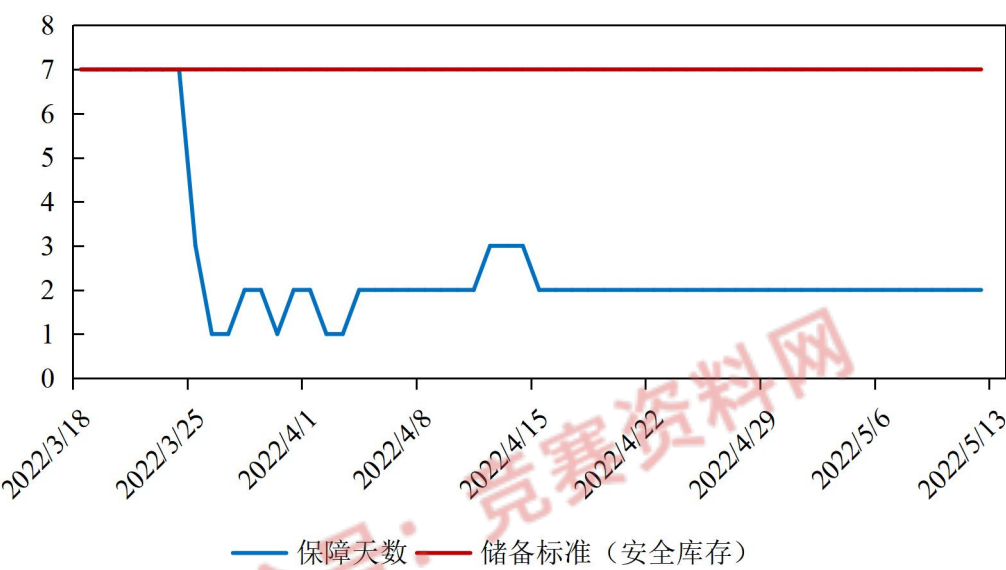


图 5-5. 蔬菜库存存储情况 (2022.03.18-2022.05.13)

表 5-9. 政府储备安全库存

区域名称	商品类别 (单位: 吨)			
	粮食 (325 克/人/日)	食用油 (30 克/人/日)	蔬菜 (400 克/人/日)	肉类 (115 克/人/日)
朝阳区	178.158175	16.44537	219.2716	63.040585
二道区	131.8798	12.17352	162.3136	46.66516
经开区	61.503	5.6772	75.696	21.7626
净月区	66.3182	6.12168	81.6224	23.46644
宽城区	99.496475	9.18429	122.4572	35.206445
绿园区	121.075825	11.17623	149.0164	42.842215
南关区	150.19745	13.86438	184.8584	53.14679
汽开区	62.87515	5.80386	77.3848	22.24813
长春新区	109.93905	10.14822	135.3096	38.90151
总计	981.443125	90.59475	1207.93	347.279875

5.4. 子问题 3：大规模物资分拣场所选址与优化——K-means 聚类 and 遗传算法的综合算法

5.4.1. 基于 K-means 的大规模物资分拣场所选址

针对大规模物资分拣场所的选址问题，本问根据附件 3 提供的小区坐标，通过聚类的方式获得类中心作为选址位置。K-means 聚类属于无监督学习，是一种基于距离的聚类算法，将距离作为相似性的评价指标，该算法认为簇是由距离相近的对象构成的，即两对象的距离越近，其相似度就越大，因此把距离相近且独立的簇作为目标。考虑到小区数量规模达 1409 个属于大规模的数据，而 K-means 可以基于欧式距离快速聚类，所求的类中心就是质心位置，与本题所求的大规模物资分拣场所选址的位置相对应。

建立基于 K-means 聚类的选址模型

设模型样本集为 $\{X\}$ 有 n 个样本以及 k 个类 $\{1, 2, \dots, K\}$ ，模型的目标是每个类中的样本到类中心的距离之和最小。建立基于聚类的选址问题数学模型如式子（5-1）到式（5-4）所示。其中，式子（5-1）表示目标函数即欧式距离最短。式（2）表示各类只能分配到一个类中心中；式（5-3）是均值向量公式。式（5-4）中 y_{ij} 为 0-1 变量。

$$\min \sum_{j=1}^k \sum_{X \in S_j} y_{ij} \|X - m_j\| \quad (5-1)$$

约束条件：

$$\sum_{j=1}^n y_{ij} = 1 (i = 1, 2, \dots, n) \quad (5-2)$$

$$m_j = \frac{1}{\sum_{i=1}^n y_{ij}} \sum_{i=1}^n y_{ij} X_i (j = 1, 2, \dots, K) \quad (5-3)$$

$$y_{ij} = \begin{cases} 1, & \text{样本 } i \text{ 聚到到聚类中心 } j \text{ 中} \\ 0, & \text{否则} \end{cases} \quad (5-4)$$

聚类结果如表 5-10 所示，其中聚类中心位置即为大规模物资分拣场所的选址位置。选址结果可视化如图 5-6 和图 5-7 所示。

表 5-10. 基于 K-means 聚类的选址结果

小区编号	小区栋数	小区户数（户）	小区人口数（人）	小区横坐标	小区纵坐标	街道编号	所属区域	cluster	聚类中心横坐标	聚类中心纵坐标
1	11	1060	2417	57.90	62.13	C0035	宽城区	13	57.56	59.55
2	37	4171	9080	51.79	61.61	C0048	宽城区	5	51.09	64.86
3	3	480	1333	54.80	54.88	C0010	宽城区	13	57.56	59.55

4	19	1093	2658	54.22	67.84	C0053	城区宽城区宽城区	5	51.09	64.86
5	15	1330	3033	56.73	61.48	C0035	城区宽城区宽城区	13	57.56	59.55
...	...	...	...	...	...	...	...	...	...	...
1405	5	231	908	25.51	34.51	I0032	汽开区汽开区	9	25.72	31.05
1406	4	267	744	35.65	40.53	I0008	汽开区汽开区	3	33.17	37.72
1407	2	185	727	26.07	36.63	I0008	汽开区汽开区	9	25.72	31.05
1408	3	115	320	33.63	36.87	I0008	汽开区汽开区	3	33.17	37.72
1409	1	48	134	45.32	42.28	I0007	汽开区汽开区	11	46.29	38.45

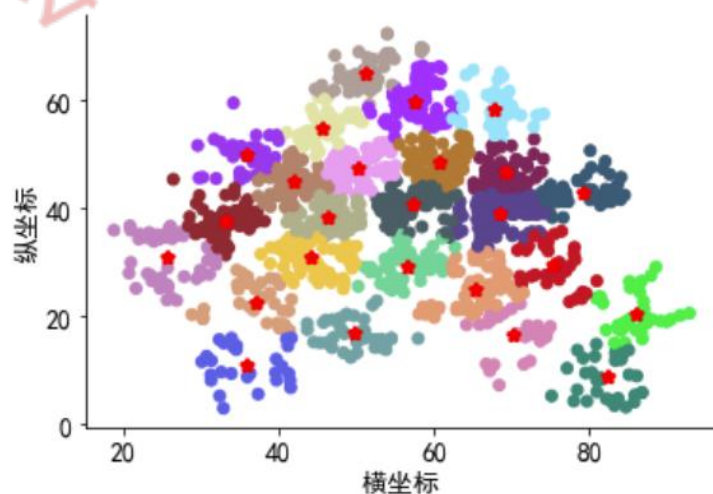


图 5-6. 大规模物资分拣场所选址与服务小区范围

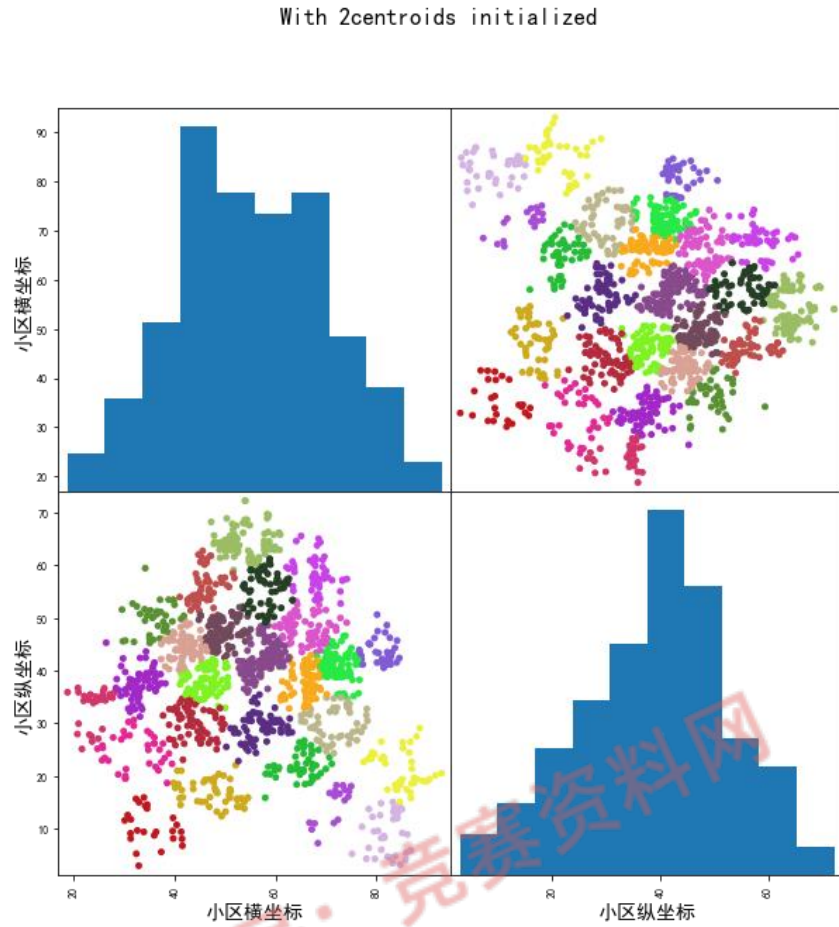


图 5-7. 小区聚类的类属性可视化

关于 K （类别数量）的选择，本文采用轮廓系数、肘部法则来最终确定最佳的 K 值。其中，轮廓系数表示聚类后各类样本间的紧密程度和各类之间的离散程度。同一类中样本间的距离越小，异类间样本距离越大，则轮廓系数的值越大，聚类效果越好，因此常被用作评价聚类结果的性能指标。轮廓系数的计算公式如式（5-5）所示，式中 a_i 代表样本内点的内聚度，其计算公式如（5-6）所示。其中 j 代表样本 i 在同一类内的其他样本点， $distance$ 代表了求 i 与 j 的距离，所以 $a(i)$ 越小说明该类越紧密。 $b(i)$ 的计算需要遍历其他类簇得到多个值从中选择最小的作为最终的结果。通过计算当 $K = 25$ 时

$$S_i = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (5-5)$$

$$a(i) = \frac{1}{n-1} \sum_{j \neq i}^n distance(i, j) \quad (5-6)$$

肘部法则的原理是成本函数，即类别畸变程度之和。每个类别的畸变程度等于每个变量点到其类别中心的位置距离平方和。当类内部越紧凑时类的畸变程度越小。在选择 K 值时，肘部法则会刻画出不同值的成本函数值。随着值的增大，每个类包含的样本数会减少，样本距离其类中心会更近且畸变程度会减小。伴随值继续增大，畸变程度的改善效果将递减，此时畸变程度的改善效果下降幅度最大的位置对应的值就是肘部。肘部法则的结果如图 5-8 所示。由此本文将 $K = 25$ 作为选址的类数较为合适。

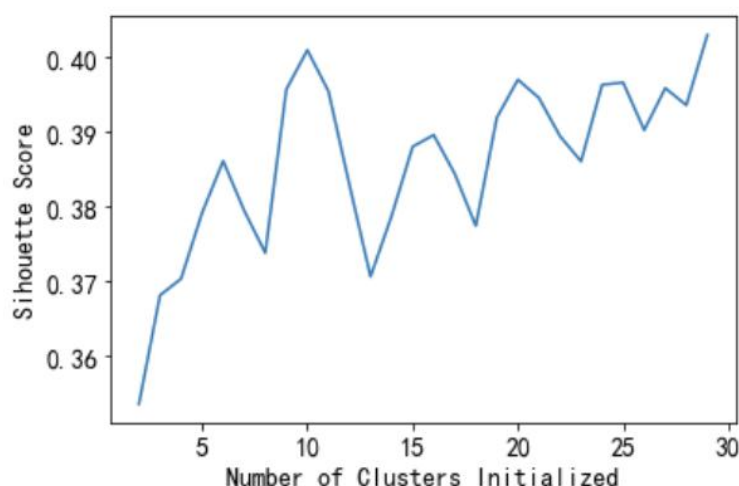


图 5-8. 肘部法则的结果可视化

5.4.2. 基于遗传算法的选址优化模型

本文在聚类确定选址位置的基础上构建基于遗传算法的选址优化模型，目的是以配送距离最短为目标，确定每个类中心（即大规模物资分拣场所选址）所服务的小区。

遗传算法的（Genetic Algorithm，简称 GA）是一种自适应随机搜索启发式算法，是模拟达尔文生物进化论的自然选择和遗传学机理的生物进化过程的计算模型。遗传算法的主要思想是采用概率寻优方法自动获取和指导优化的搜索空间，并能够自适应地调整搜索方向。遗传算法步骤如图 5-9 所示。核心步骤下所示：

表 5-11. 遗传算法的核心步骤

步骤	操作
Step 1	编码：在进行寻优前将解空间的解数据用遗传空间的基因型串结构数据来表示，通过不同的组合构成不同的点；
Step 2	生成初始群体：随机产生初始数据作为初始点开始迭代。设置迭代计数器，设置最大迭代次数；
Step 3	评价适应度值；
Step 4	选择：将选择算子作用于群体；
Step 5	交叉：将交叉算子作用于群体；
Step 6	变异：将变异算子作用于群体；
Step 7	生成下一代群体；
Step 8	终止条件判断：具有最大适应度的个体作为最优解输出，终止运算。

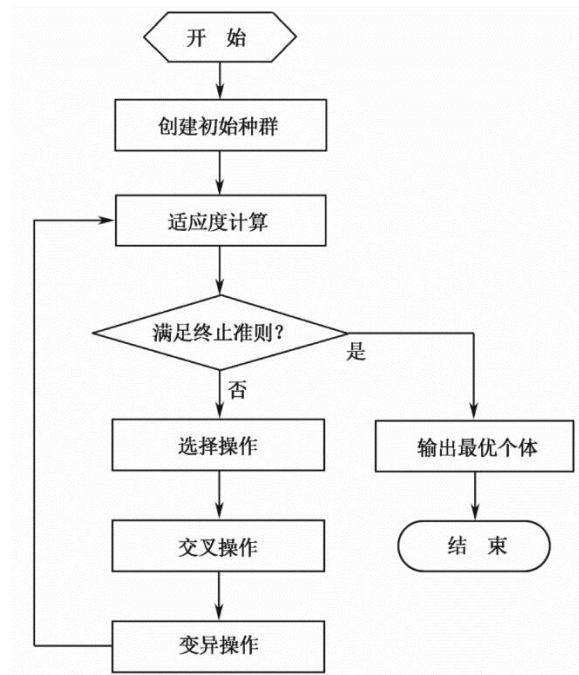


图 5-9. 遗传算法基本步骤

通过上述分析，以大规模物资分拣场到居民小区或者投放点距离最小为目标，建立的最短路径模型如下所示：

表 5-12. 表大规模物资分拣场的选址和路径优化模型

集合	
V	顶点集合， $V = N \cup \{O\}$ 。其中， O 代表配送中心。
E	路线集合， $E = \{(i, j), \forall i, j \in N, i \neq j\}$
EO, EI	起始路线编号和终止路线编号
$F(i)$	路线编号终点的前序节点集合
$A(i)$	路线编号起点的后序节点集合
K	车辆集合， $K = \{1, 2, \dots, K \}$
参数	
C_{ij}	路线距离， $(i, j) \in E$
D_{ij}	边上需求， $(i, j) \in E$
Q	车的容量
决策变量	
x_{kij}	0-1 变量：若车辆 k 经过边 (i, j) 时为 1，否则为 0. $(i, j) \in E$
u_{ki}	整数：车辆 k 离开节点 i 的装载量， $\forall k \in K, \forall i \in V$
目标函数：	

$$\min \sum_{k \in K} \sum_{(i, j) \in E} C_{ij} \cdot x_{kij} \quad (5-7)$$

约束条件：

$$\sum_{(i,j) \in EO} x_{kij} = 1, \forall k \in K \quad (5-8)$$

$$\sum_{(i,j) \in EI} x_{kij} = 1, \forall k \in K \quad (5-9)$$

$$\sum_{k \in K} \sum_{(i,j) \in EO} x_{kij} = |K| \quad (5-10)$$

$$\sum_{j \in F(i)} x_{kji} = \sum_{j \in A(i)} x_{kij}, \forall i \in N, \forall k \in K \quad (5-11)$$

$$\sum_{k \in K} D_{ij} \cdot x_{kij} \leq Q, \forall (i,j) \in E \quad (5-12)$$

$$u_{kj} \geq u_{ki} + (x_{kij} - 1) \cdot Q + D_{ij}, \forall (i,j) \in E \text{ and } j \neq 0, \forall k \in K \quad (5-13)$$

$$x_{kij} \in \{0,1\}, \forall (i,j) \in E, \forall k \in K \quad (5-14)$$

$$u_{ki} \geq 0, \forall i \in N, \forall k \in K \quad (5-15)$$

5.4.3. 计算结果

由于此问题是 NP-hard 问题，为保证求解的质量和效率采用遗传算法进行求解（求解代码请见附件），计算结果如表 5-13 所示。

表 5-13. 大规模物资分拣场所选址与优化问题结果

No.	x坐标	y坐标	管辖范围小区个数	管辖范围内人口数	总服务距离	选址半径	所属行政区
0	35.944714	10.835988	28	147343	141.16	5.04	长春新区
1	50.116419	47.424687	83	185812	256.47	3.09	绿园区
2	75.333454	29.647553	38	57373	164.05	4.32	经开区
3	33.171931	37.717431	58	98891	192.60	3.32	汽开区
4	69.229461	46.756211	57	151390	166.78	2.93	二道区
5	51.085767	64.857218	58	110980	195.35	3.37	宽城区
6	57.384641	40.61421	92	209895	293.83	3.19	南关区
7	82.247632	8.72149	34	32365	150.23	4.42	净月区
8	56.473728	29.202943	58	122016	203.91	3.52	南关区
9	25.722411	31.049356	45	94921	235.01	5.22	汽开区
10	65.375926	25.069579	61	98668	271.11	4.44	南关区
11	46.286134	38.451977	75	130886	224.98	3.00	朝阳区
12	79.12181	42.936327	50	98753	190.90	3.82	经开区
13	57.563096	59.545922	68	114498	252.90	3.72	宽城区
14	49.695732	16.72146	42	80277	154.15	3.67	长春新区

15	85.918443	20.252905	35	32226	148.77	4.25	净月区
16	45.679303	54.744489	47	86432	141.84	3.02	绿园区
17	44.069407	31.157538	73	203616	273.36	3.74	朝阳区
18	68.519579	39.019632	107	181750	367.96	3.44	经开区
19	67.83679	58.149913	50	142176	179.01	3.58	二道区
20	35.885269	49.83894	39	106757	137.10	3.52	绿园区
21	70.224214	16.55781	28	72936	121.55	4.34	净月区
22	36.99186	22.565549	34	141379	152.49	4.48	长春新区
23	42.013396	44.762825	82	113287	228.49	2.79	绿园区
24	60.760388	48.59209	67	205198	224.92	3.36	二道区

六、问题三分析与求解

6.1. 问题三分析

根据问题三要求，此题可以分为两个子问题，子问题一是依据附件 5 分析蔬菜包需求和发放规律。子问题二是根据附件 3 中的各小区位置与人口信息，评价和调整 4 月 10 日至 4 月 15 日蔬菜包供应方案。针对子问题一，本文采用文本挖掘技术，统计了附件 5 当中从 2022 年 3 月 26 日到 5 月 1 日九区总体和分区的每日蔬菜包的供给量和需求量。分别从蔬菜包的供应规律、需求规律和供需规律三方面切入，运用描述性统计、正态性检验、相关性分析的方法结合新冠肺炎感染人数变化，多层次分析蔬菜包的供需规律。针对问题二，本文采用 TOPSIS 综合评价方法，建立包含：各区本土感染人数和无症状感染者人数，各区小区栋数、小区户数和小区人口数、街道数等指标构成的评价体系，计算各指标权重，获得各区的得分与排名作为优化依据。

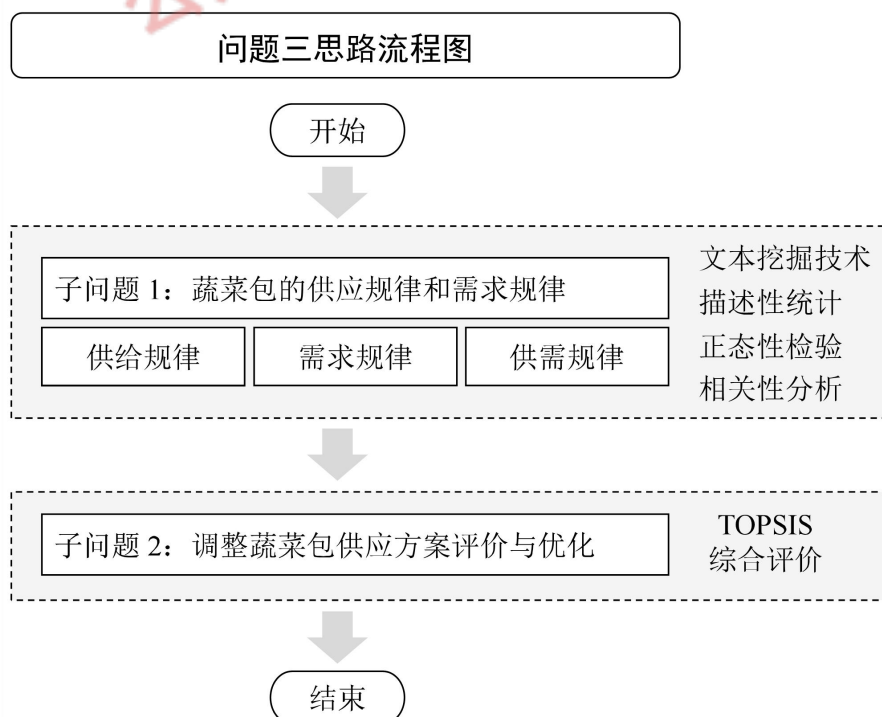


图 6-1. 问题三的思路流程图

6.2. 子问题 1：蔬菜包的供应规律和需求规律——数据挖掘和数据分析

6.2.1. 蔬菜包的供应规律

本文将附件 5 中的将九区的蔬菜接收量认定为九区的蔬菜包供应量，表示各区蔬菜包的供给能力。从供给的角度分析九个区的蔬菜包整体与分区的供应规律。

依据附件 5，发现各区蔬菜包接收记录是从 3 月 26 日开始记录到 5 月 1 日结束，故本文对于蔬菜包供应规律的分析是从 2022 年 3 月 26 日开始到 5 月 1 日结束。运用数据挖掘技术将附件 5 中相关数据进行汇总得到九个区的总体供应量和分区供应量，如图 6-3 所示。蔬菜包供应量的描述统计分析结果如表 6-1 所示，包括样本量、最大值、最小值等统计量，可用于研究数据的整体情况，其中变异系数（CV）均大于 0.15，表明供应量的变化幅度过大

表 6-1. 九个区蔬菜包供应量的总体描述统计分析结果

变量名	样本量	最大值	最小值	平均值	标准差	中位数	方差	峰度	偏度	变异系数（CV）
朝阳区	37	40355	0	10939.622	9207.865	8846	84784778.742	3.994	1.904	0.841
南关区	37	27671	0	7636.459	9058.974	3200	82065010.977	-0.882	0.837	1.186
宽城区	37	88666	0	16820.568	20663.588	9852	426983866.474	5.806	2.427	1.228
绿园区	37	118057	0	17759.973	24029.334	16356	577408891.027	7.72	2.368	1.353
二道区	37	54933	0	11734.946	15226.859	5661	231857237.219	1.479	1.463	1.299
长春新区	37	44826	0	8433.297	9487.222	4824	90007375.048	5.855	2.213	1.125
经开区	37	51347	0	10032.541	13410.316	2600	179836572.589	1.393	1.415	1.337
净月区	37	23333	0	6618.811	6805.284	4953	46311888.269	0.19	1.007	1.028
莲花山	37	16376	0	1509.676	2891.45	890	8360484.503	19.927	4.084	1.915
汽开区	37	38854	0	6241.297	9495.845	0	90171080.604	3.339	1.836	1.521
九台区	37	13588	0	991.486	2941.79	0	8654128.535	11.271	3.348	2.967
中韩示范区	37	2625	0	206.081	569.186	0	323972.41	9.119	2.979	2.762
九区蔬菜包接收数量总计	37	293518	7100	98924.757	85080.114	74113	7238625718.967	0.16	1.069	0.860

	朝阳区	南关区	宽城区	绿园区	二道区	长春新区	经开区	净月区	莲花山	汽开区	总量
3月26日	4340	3200	2000	0	2500	2109	1000	5678	0	0	20827
3月27日	0	3447	3500	0	7032	1900	33462	6553	0	4631	60525
3月28日	3100	5000	88666	4161	9000	3100	6800	5000	5000	10000	139827
3月29日	10100	23286	34000	20694	54933	13392	22658	16689	0	17575	213327
3月30日	14075	20700	11900	35175	52500	20201	8681	13500	3290	31148	211170
3月31日	11450	18220	0	30244	41844	11200	12700	19652	0	38854	184164
4月1日	14213	14346	6166	37995	15271	4000	12700	8218	0	17297	130206
4月2日	8005	17242	5707	0	0	8000	7823	0	0	19182	65959
4月3日	5754	5500	9859	45190	16000	5178	16000	1000	0	14982	119463
4月4日	8292	791	8739	32970	10000	0	0	10744	0	0	71536
4月5日	8159	600	11620	25379	20000	4824	1300	1955	0	0	73837
4月6日	16023	2320	11682	22285	24000	4810	2600	0	0	0	83720
4月7日	5601	17910	18577	16619	3000	13710	8410	0	0	0	83827
4月8日	17160	19581	45748	15366	6025	20168	7100	10451	0	12589	154188
4月9日	25771	20200	57802	18556	22673	44826	37814	8980	400	3603	240625
4月10日	40355	27671	83415	44772	27005	9187	24407	11594	960	17136	286502
4月11日	38981	23013	23126	69828	27957	33628	51347	7462	16376	1800	293518
4月12日	29120	22668	8549	118057	31212	13609	34988	23333	890	8417	290843
4月13日	12010	11857	2836	26809	30281	20000	13000	23126	0	5431	145350
4月14日	11874	5070	4511	16356	12909	5000	20000	13956	5445	16127	111248
4月15日	8846	8040	13467	22380	5661	7405	29000	11698	1000	5000	112497
4月16日	7560	7887	9852	17138	1090	12215	19414	4021	0	1800	80977
4月17日	7333	2000	0	17921	0	9764	0	4176	1440	5356	47990
4月18日	10244	2000	6323	19224	1900	8854	0	0	1313	0	49858
4月19日	9048	0	1992	0	5700	10300	0	15549	1700	0	44289
4月20日	8575	0	6992	0	1900	4700	0	0	4400	0	26567
4月21日	9971	0	5000	0	3800	0	0	0	0	0	18771
4月22日	7865	0	6500	0	0	2000	0	0	0	0	16365
4月23日	10117	0	13990	0	0	3260	0	1180	1140	0	29687
4月24日	9200	0	20686	0	0	3000	0	4953	1360	0	39199
4月25日	10725	0	22704	0	0	2645	0	5976	1550	0	43600
4月26日	8661	0	20510	0	0	2736	0	3869	1544	0	37320
4月27日	7234	0	20194	0	0	1800	0	2924	1500	0	33652
4月28日	4146	0	14048	0	0	1560	0	429	2400	0	22583
4月29日	858	0	8000	0	0	2951	0	730	650	0	13189
4月30日	0	0	9500	0	0	0	0	700	1400	0	11600
5月1日	0	0	4200	0	0	0	0	800	2100	0	7100

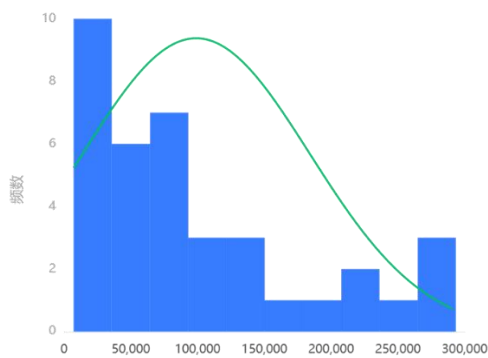
图 6-3. 九区蔬菜包供应量的情况（2022.03.26-2022.05.1）单位：包

蔬菜包供应量的正态性检验如表 6-2 所示。蔬菜包供应量的正态性检验直方图如图 6-2-(a)所示，展示了九区蔬菜包接收数量总计数据的正态性检验直方图，正态图基本上呈现出钟形（中间高，两端低），虽然不是绝对正态，但基本可接受为正态分布。图 6-2-(b)中展示了正态性检验 P-P 体现出观测的累计概率（P）与正态累计概率（P）的拟合情况，从拟合程度来看蔬菜包的供应量近似服从正态分布。6-2-(c)展示了正态性检验 Q-Q 的结果。Q-Q 图，全称“Quantile Quantile Plot”。用图形的方式比较观测值与预测值（假定正态下的分布）不同分位数的概率分布，从而检验是否吻合正态分布规律。并且将实际数据作为 X 轴，将假定正态时的数据分位数作为 Y 轴，作散点图。从图中可看出，散点与直线重合度较高近似服从正态分布。

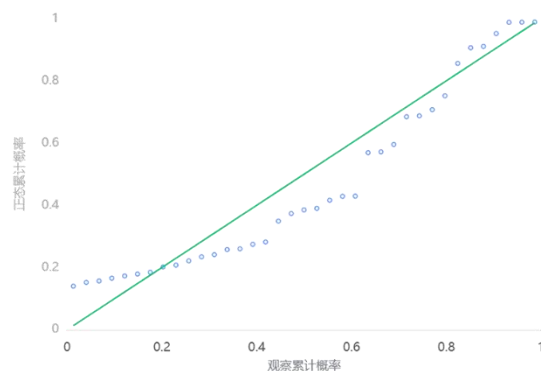
表 6-2. 蔬菜包供应量的正态性检验结果

变量名	样本量	中位数	平均值	标准差	偏度	峰度	S-W 检验	K-S 检验
九区蔬菜包接收数量总计	37	74113	98924.757	85080	1.069	0.16	0.862 (0.000***)	0.192

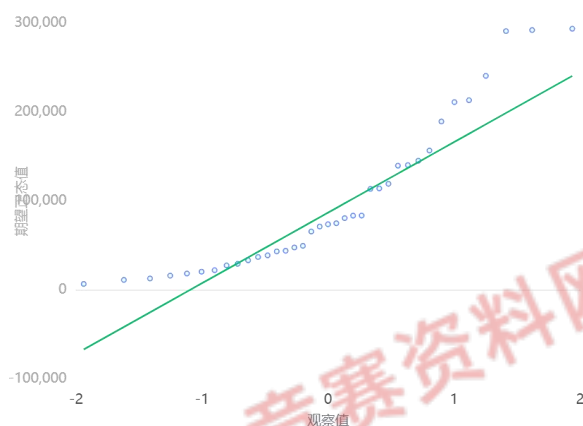
注：***、**、*分别代表 1%、5%、10%的显著性水平



(a) 正态性检验直方图



(b) 正态性检验 P-P 图



(c) 正态性检验 Q-Q 图

图 6-2. 蔬菜包供应量的正态性检验

各区本土新增与无症状感染者的汇总如图 6-4 所示。蔬菜包供应量的时序图如图 6-5 所示，蔬菜包供给量与新冠感染人数的时序变化折线图如图 6-6 所示。蔬菜包供给量与新冠感染人数的 *Spearman* 相关性检验热力图如图 6-7 所示。*Spearman* 相利用单调方程评价两个统计变量的相关性，适用于评价非正态分布数据的相关性。对于样本容量为 n 的样本， n 个原始数据被转换成等级数据，*Spearman* 相关系数 ρ 为式 (6-1) 所示。

$$\rho = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}. \quad (6-1)$$

	朝阳区	南关区	宽城区	绿园区	二道区	长春新区(高新)	经开区	净月区	汽开区	每天感染人数总计(本土+无症状)
3月4日	0	0	0	0	0	0	0	0	0	0
3月5日	0	0	0	0	0	0	0	0	0	0
3月6日	0	0	0	1	0	0	1	1	0	3
3月7日	1	0	0	0	0	0	1	2	0	4
3月8日	0	0	0	0	0	0	0	0	0	0
3月9日	0	0	1	2	0	4	0	2	0	9
3月10日	0	0	0	0	0	0	4	0	0	4
3月11日	0	7	1	2	0	35	4	0	1	50
3月12日	5	30	3	10	4	31	4	154	1	242
3月13日	10	14	4	7	2	7	12	33	2	91
3月14日	101	31	13	27	24	43	37	30	21	327
3月15日	22	12	3	4	0	14	26	2	1	84
3月16日	42	10	5	5	3	8	15	9	2	99
3月17日	29	54	16	51	28	66	67	32	21	364
3月18日	40	50	12	32	9	36	72	110	13	374
3月19日	107	109	39	33	26	69	70	13	18	484
3月20日	90	73	29	35	43	49	100	44	30	493
3月21日	63	63	36	91	25	100	67	34	31	510
3月22日	183	98	135	92	99	94	111	63	61	936
3月23日	119	64	158	140	80	90	96	50	85	882
3月24日	55	46	63	61	46	80	88	10	11	460
3月25日	54	70	153	63	72	89	57	87	67	712
3月26日	70	48	45	36	81	45	64	41	38	468
3月27日	49	51	77	70	54	126	146	74	29	676
3月28日	78	77	83	109	28	66	43	19	44	547
3月29日	67	42	457	155	41	41	64	36	23	926
3月30日	75	39	352	72	185	145	178	21	50	1117
3月31日	101	61	413	327	129	39	76	61	62	1269
4月1日	223	213	1031	250	54	71	189	39	37	2107
4月2日	375	299	833	503	542	151	262	89	97	3151
4月3日	342	737	386	377	140	222	214	115	104	2637
4月4日	206	165	488	236	136	122	165	89	64	1671
4月5日	269	151	916	313	112	122	312	74	71	2340
4月6日	192	252	621	353	110	106	306	80	38	2058
4月7日	188	296	580	233	155	80	235	89	42	1898
4月8日	48	91	333	63	61	61	51	19	12	739
4月9日	41	74	354	214	27	32	75	9	3	829
4月10日	35	62	312	241	62	21	48	22	2	805
4月11日	36	60	185	164	32	18	52	30	3	580
4月12日	84	69	375	133	59	61	98	23	18	920
4月13日	35	49	384	152	72	45	55	33	14	839
九区感染人数总计(本土+无症状)	3434	3567	3896	4656	2541	2389	3463	1636	1116	31698

图 6-4. 九区各区本土新增与无症状感染者的汇总（2022.03.04-2022.04.13）

注：根据附件 1 的感染数据中九区从 2022.04.14-2022.05.01 均为 0，所以此段数据未体现在此表中。

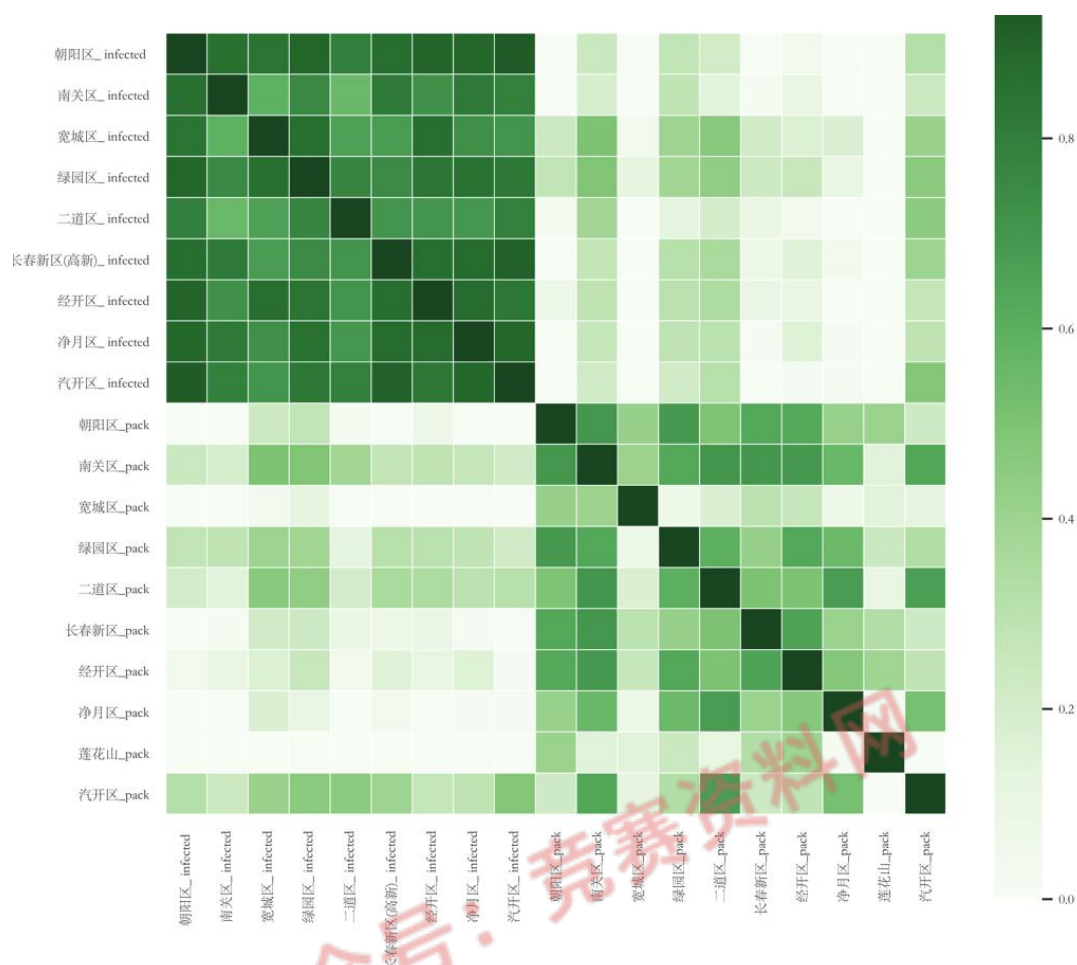


图 6-7. 蔬菜包供应量与新冠感染人数的Spearman相关性检验热力图

综合来看，九个区蔬菜包的供应规律如下：

规律 1：蔬菜包供给量随疫情变化具有阶段性特征。一是新冠肺炎疫情快速扩散阶段（3 月 26 日到 4 月 2 日），新冠病毒感染人数持续上涨并在 4 月 2 日达到顶峰，此时九区的总供给量随着上涨。从九个区总体供给量的时序变化趋势来看，呈现倒“V”的形状，说明随着感染人数的增加，政府加大了对各区物资的供给。二是新冠肺炎疫情小幅反弹阶段（4 月 3 日到 4 月 12 日），此阶段新增新冠感染人数较第一阶段有所下降但仍存在小幅度反弹的现象，但总体有所好转。此阶段蔬菜包供应量呈现上升趋势并且在 4 月 11 日达到最高水平（293518 包），主要分配给绿园区（69828 包）和经开区（51347 包），结合 4 月 11 日当天的疫情状况来看新增感染人数 580 例，而主要新增的区是宽城区（185 例）和绿园区（164 例），结合经开区当天的 46424.5 包的物资发放量来看，经开区存在 4922 包的物资浪费。三是新冠肺炎疫情下疫情的快速清零阶段（4 月 14 日到 5 月 1 日），九区自 4 月 14 日到 5 月 1 日保持了零新增，特别是长春市自 4 月 28 日起逐步解除社会管控有序恢复正常生产生活秩序，蔬菜包供应量逐步减少，直到 5 月 1 日的 7100 包供应给宽城区（4200 包）、莲山区（2100 包）和净月区（800 包）。

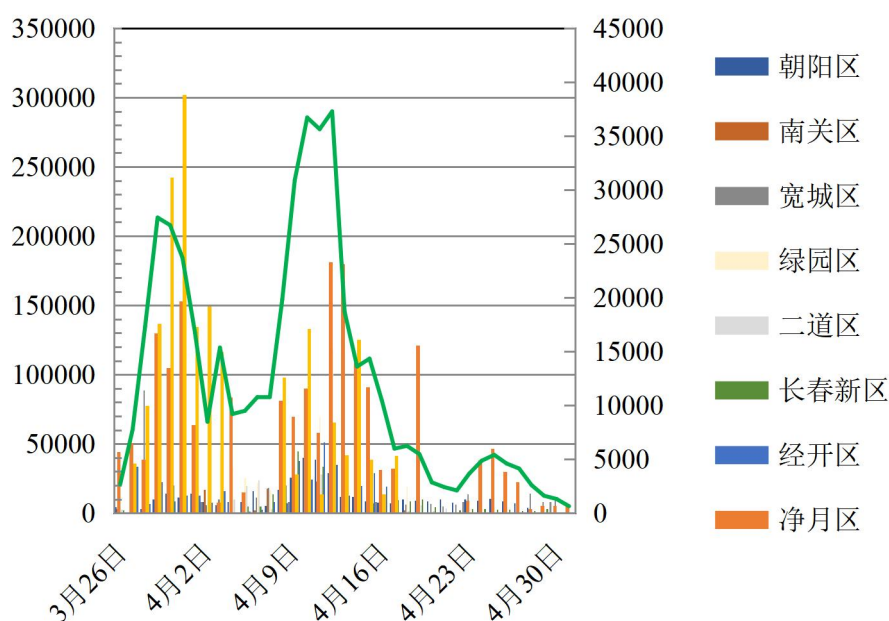


图 6-5. 各区蔬菜包供应量的时序图

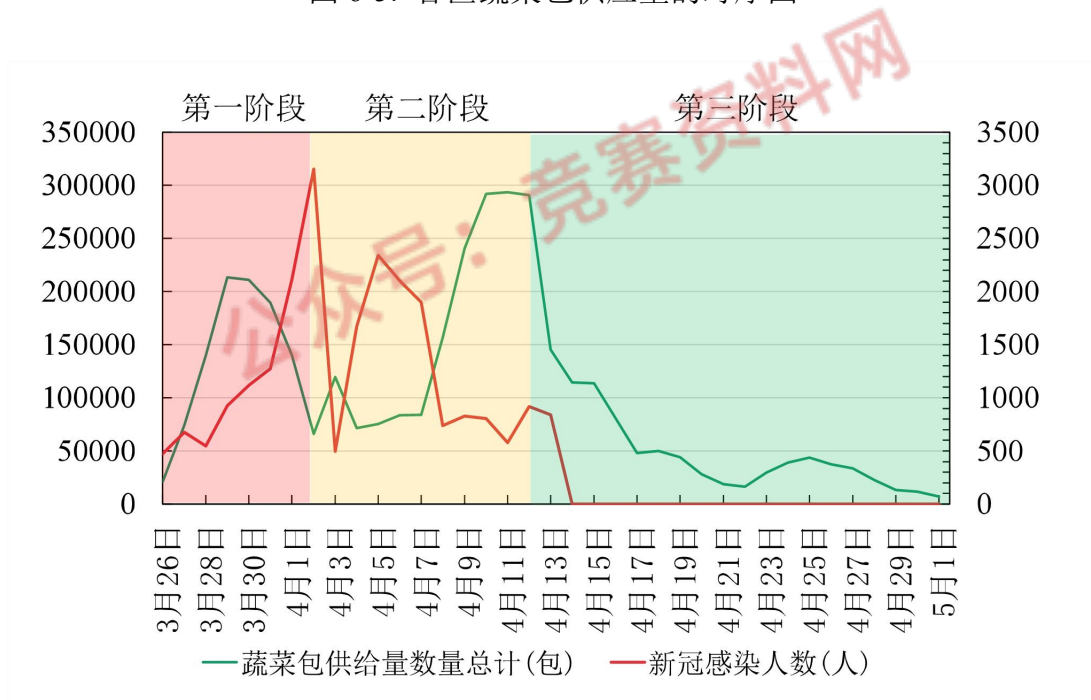


图 6-6. 供给量与新冠感染人数的时序变化折线图

规律 2：蔬菜包的供应量近似正态分布。根据正态性检验直方图、P-P 图、Q-Q 图都表明虽然数据不是绝对正态，但基本可接受为正态分布。

规律 3：蔬菜包的供应量与各区新增病例的关系是低相关，只有汽开区是中度相关（0.477113）这表明疫情新增人数是不是影响蔬菜包供应量的主要因素。

规律 4：蔬菜包供给量与疫情新增人数发展变化来看，两者发展趋势相似，但是蔬菜包供给量的变化具有滞后性。6.2.2. 蔬菜包的需求规律

本文将附件 5 中的将九区的蔬菜包每日实际发放量认定为蔬菜包的需求量，表示各区每日对蔬菜包的需求。从需求的角度分析九个区的蔬菜包整体与分区的需求规律。

6.2.2. 蔬菜包的需求规律

依据附件 5 发现蔬菜包发放量的记录是从 4 月 4 日开始记录到 5 月 1 日结束，故本文对于蔬菜包需求规律的分析是从 2022 年 4 月 4 日开始到 5 月 1 日结束。运用数据挖掘技术将附件 5 中相关数据进行汇总得到九个区的总体需求量和分区需求量，如图 6-8 所示。蔬菜包供应量的描述统计分析结果如表 6-3 所示，包括样本量、最大值、最小值等统计量，可用于研究数据的整体情况，其中变异系数（CV）均大于 0.15，表明需求量的变化幅度过大。

	朝阳区	南关区	宽城区	绿园区	二道区	长春新区	经开区	净月区	汽开区	总量
4月4日	8292	2411	11327	32970	7000	4000	0	4934	0	70934
4月5日	10482	860	16270	25379	16000	4869	1000	7329	3560	85749
4月6日	6023	2455	12532	22285	9765	9194	9226	5083	5085	81648
4月7日	10702	5142	20921	16619	15125	8015	8410	8871	2510	96315
4月8日	17260	500	42821	15366	11259	16431	7100	10451	16876	138064
4月9日	25808	19080	26557	9900	18000	14558	35414	8980	3603	161900
4月10日	38244	1629	62226	32812	26908	17874	24360	11594	8777	224424
4月11日	37189.5	19041	13251	57047	29122	21893.5	46424.5	7462	14722	246152.5
4月12日	23125	21371	12800	74611	17698	31737	16663.5	19333	5862.5	223201
4月13日	19475.5	25285	2737	39967	10637	25254.5	13994	24522	6044.5	167916.5
4月14日	11874	23223.5	5385	16356	7142	17913	20000	18948	7500	128341.5
4月15日	8285	14447	10244	22380	5748	8731	25000	14136	5000	113971
4月16日	7560	17111.5	4788	17138	5725	7543	14214	7033	1800	82912.5
4月17日	7918	5070	6500	0	1168	2000	4000	2680	0	29336
4月18日	10244	8040	8761	19224	500	8854	0	4960	700	61283
4月19日	9048	7887	3017	0	5098	10700	0	17714	676	54140
4月20日	8575	0	6992	0	300	2801	0	1804	6754	27226
4月21日	9971	2000	5000	0	4096	111	3300	3429	0	27907
4月22日	9971	0	5000	0	4096	111	3300	3429	0	25907
4月23日	9597	0	13990	0	1393	3260	0	1180	0	29420
4月24日	9200	0	20686	0	0	3000	0	4953	0	37839
4月25日	10725	0	22704	0	50	2645	0	5976	0	42100
4月26日	8661	0	20510	0	0	2736	0	3869	0	35776
4月27日	7234	0	20194	0	0	1800	0	2924	0	32152
4月28日	4146	0	14048	0	0	1560	0	429	0	20183
4月29日	858	0	8000	0	0	2951	0	730	0	12539
4月30日	0	0	9500	0	0	0	0	700	0	10200
5月1日	0	0	4200	0	0	0	0	800	0	5000

图 6-8. 九个区的总体需求量和分区需求量（2022.4.4-2022.5.1）

注：附件 5 中蔬菜包发放量的记录是从 4 月 4 日开始记录到 5 月 1 日结束，故本文对于蔬菜包需求规律的分析是从 2022 年 4 月 4 日开始到 5 月 1 日结束。

表 6-3. 蔬菜包需求的描述统计分析结果

变量名	样本量	最大值	最小值	平均值	标准差	中位数	方差	峰度	偏度	变异系数（CV）
朝阳区	28	38244	0	11802.429	9385.754	9398.5	88092383.495	2.734	1.639	0.795
南关区	28	25285	0	6269.75	8522.972	1814.5	72641049.583	-0.231	1.15	1.359
宽城区	28	62226	2737	14677.179	12815.741	11929.5	164243228.819	6.696	2.331	0.873
绿园区	28	74611	0	14359.071	19237.054	4950	370064262.884	2.644	1.608	1.339
二道区	28	29122	0	7029.643	8332.783	4597	69435264.831	1.038	1.294	1.185
长春新区	28	31737	0	8233.643	8439.917	4434.5	71232202.701	0.975	1.265	1.025
经开区	28	46424.5	0	8300.214	12201.024	2150	148864993.897	2.668	1.725	1.470

净月区	28	24522	429	7294.75	6427.184	5021.5	41308690.935	0.798	1.211	0.881
汽开区	28	16876	0	3195.357	4523.415	688	20461285.108	2.795	1.718	1.416
总量	28	246152.5	5000	81162.036	69729.606	57711.5	4862218019.295	0.23	1.097	0.859

综合来看，九个区蔬菜包的整体需求规律如下：

规律 1：蔬菜包需求量随疫情变化具有阶段性特征。结合图 6-10 各区蔬菜包需求应量的时序图和图 6-11 蔬菜包需求量与新冠感染人数的时序变化折线图。蔬菜包需求划分为四个阶段，一是新冠肺炎疫情快速反弹阶段（4 月 4 日到 4 月 8 日，阶段 1），在此阶段，居民对蔬菜包的需求量呈持续上升态势，主要需求地是绿园区和宽城区分别占此阶段九区总需求量的 23.82%和 21.97%。这一部分原因是受新冠新增感染人数的影响，新冠病毒感染人数持续上涨并在 4 月 5 日达到顶峰（新增感染 2340 人），这是自 3 月 4 日到 4 月 13 日期间的第三次较大规模反弹，主要发生在宽城区（新增感染 916 人）、绿园区（新增感染 313 人）、经开区（新增感染 312 人）。另一部分原因是绿园区小区人口数较多为 372541 人在九区中人口排名居于第四位。不过宽城区人口数为 306143 人在九区中人口排名居于第六位，人口占九区总人口的 10.13%，而总需求量占九区的 23.82%，这表明宽城区的新冠肺炎感染导致隔离的人数较其他区更多。二是新冠肺炎疫情快速缓解阶段（4 月 8 日到 4 月 11 日）在此阶段，居民对蔬菜包的需求量仍持续上升直到 4 月 11 日达到顶峰为（246152.5 包），主要分布在绿园区（57047 包，23.18%）、经开区（46424.5 包，18.86%）、朝阳区（37189.5，15.1%）。三是新冠肺炎疫情局势趋于清零阶段（4 月 11 日到 4 月 17 日），随着新增人数的减少直到 4 月 14 日实现零增长，蔬菜包的需求量呈下降趋势，在 4 月 17 日降低至 4 月 4 日以来需求量的最低点 29336 包。四是新冠肺炎疫情持续零增长阶段（4 月 17 日到 5 月 1 日），九区的蔬菜包每日需求量小幅度上升，但上升幅度极小，相比于前三个阶段，一直保持较低的需求水平。

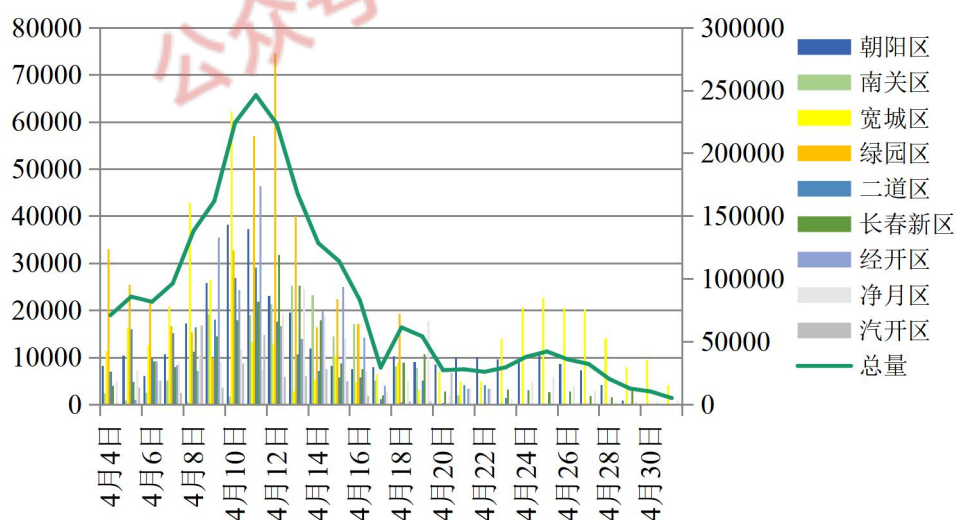


图 6-10. 各区蔬菜包需求应量的时序图（2022.4.4-2022.5.1）

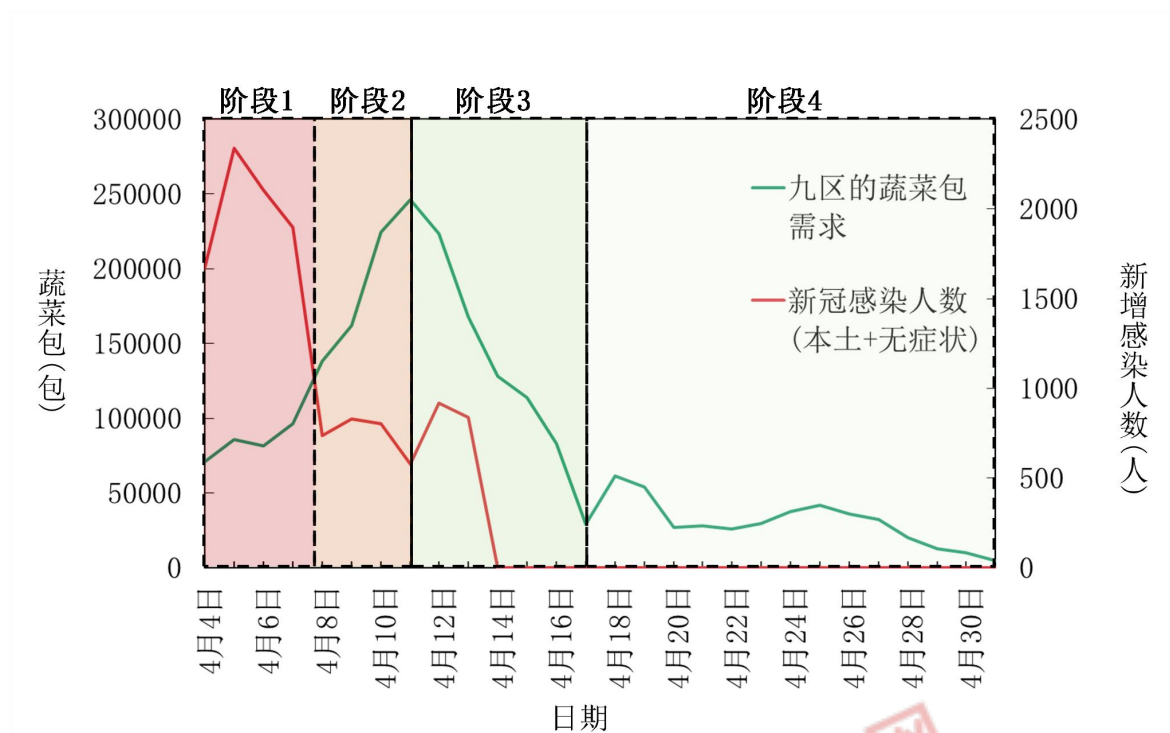


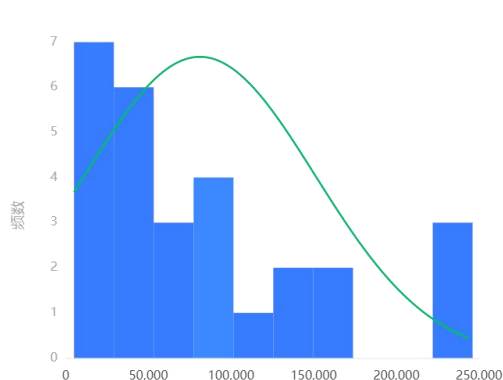
图 6-11. 蔬菜包需求量与新冠感染人数的时序变化折线图（2022.4.4-2022.5.1）

规律 2：蔬菜包的需求量近似正态分布。根据正态性检验直方图、P-P 图、Q-Q 图都表明虽然数据不是绝对正态，但基本可接受为正态分布。蔬菜包供应量的正态性检验如表 6-4 所示。蔬菜包需求量的正态性检验直方图如图 6-9-(a)所示，展示了九区蔬菜包发放总计数数据的正态性检验直方图，正态图基本上呈现出钟形（中间高，两端低），则说明数据虽然不是绝对正态，但基本可接受为正态分布。图 6-9-(b)中展示了正态性检验 P-P 体现出观测的累计概率（P）与正态累计概率（P）的拟合情况，从拟合程度来看蔬菜包的供应量近似服从正态分布。6-9-(c)展示了正态性检验 Q-Q 的结果。从图中可看出，散点与直线重合度较高蔬菜包的需求近似服从正态分布。

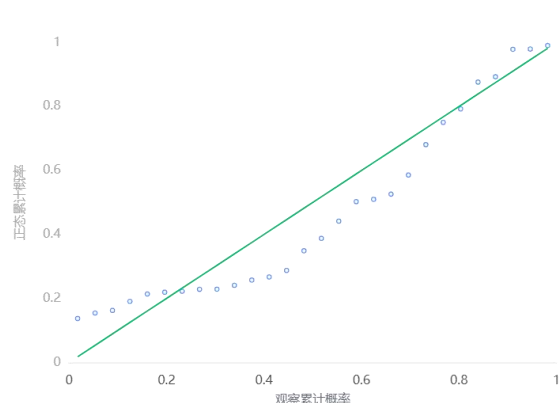
表 6-4. 蔬菜包需求量的正态性检验结果

变量名	样本量	中位数	平均值	标准差	偏度	峰度	S-W 检验	K-S 检验
28	57711.5	81162.036	69729.606	1.097	0.23	0.861(0.002***)	0.177(0.309)	28

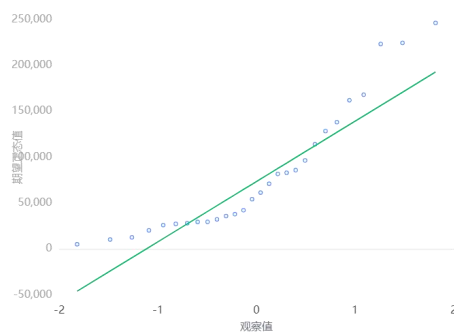
注：***、**、*分别代表 1%、5%、10%的显著性水平



(a) 正态性检验直方图



(b) 正态性检验 P-P 图



(c)正态性检验 Q-Q 图
图 6-9. 蔬菜包供应量的正态性检验

规律 3：蔬菜包的需求量与各区新增感染病例的关系是大部分是弱相关。如图 6-12 蔬菜包需求量与新冠感染人数的 *Spearman* 相关性检验热力图中绿园区和二道区的蔬菜包的需求量与各区新增感染病例是中度相关，*Spearman* 相关系数分别为 0.568426 和 0.556370。其他均为弱相关。

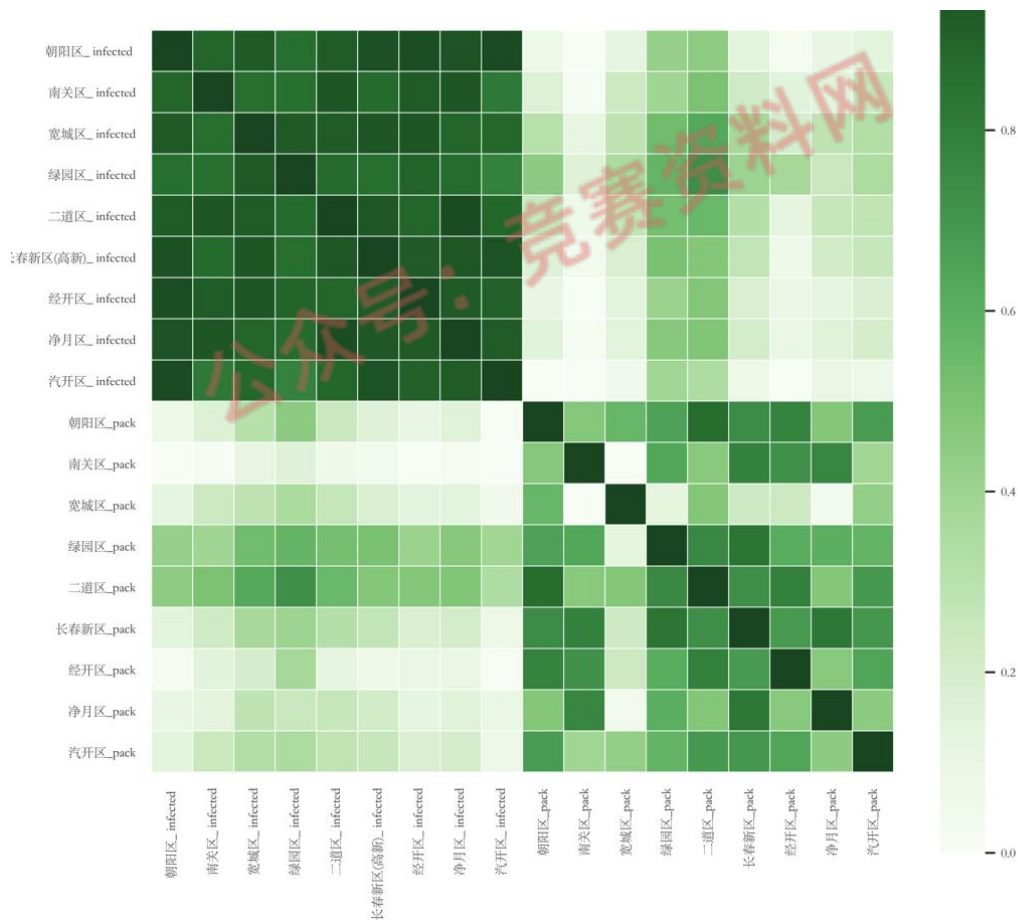


图 6-12. 蔬菜包需求量与新冠感染人数的 *Spearman* 相关性检验热力图

规律 4：蔬菜包的各区需求量与各小区信息、街道数和感染人数的具有相关性。由图 6-13 的 *person* 相关系数热力图来看，蔬菜包的各区需求量总量与新增感染人数强相关（0.779），与街道数目极弱相关（0.181），与小区栋数弱相关（0.349）、与小区户数弱

相关（0.334）。同时，也可反映出小区人口数与小区户数、小区栋数之间是极强相关，同时小区的相关信息都与街道数目中度相关。

新增感染人数（本土+...	0.779	0.146	0.164	0.000	0.213	1.000
街道数目	0.181	0.444	0.424	0.453	1.000	0.213
小区栋数	0.349	0.911	0.912	1.000	0.453	0.000
小区户数（户）	0.358	0.999	1.000	0.912	0.424	0.164
小区人口数（人）	0.334	1.000	0.999	0.911	0.444	0.146
蔬菜包需求总量(4...	1.000	0.334	0.358	0.349	0.181	0.779
	蔬菜包需求总量(4...	小区人口数（人）	小区户数（户）	小区栋数	街道数目	新增感染人数（本土+...

图 6-13. 蔬菜包的各区需求量与各小区信息、街道数和感染人数的person相关性分析

6.2.3. 蔬菜包的供需规律

对比 4 月 4 日到 5 月 1 日之间，九个区蔬菜包需求量、供给量的情况可以探究蔬菜包的供需规律。由于蔬菜包既有蔬菜又穿插肉、蛋、奶和水果，都属于易腐产品。若当日的供给量大于需求量，则会造成蔬菜包的浪费。若当日的供给量小于需求量，则会产生缺货问题不能满足隔离群众的基本生活需要。从图 6-14 蔬菜包的供需数量对比分析来看再 4 月 4 日到 4 月 8 日之间以及 4 月 17 日到 4 月 24 日之间存在蔬菜包的轻微浪费，这两个时间段处于疫情扩散和反弹阶段。而在 4 月 8 日到 13 日之间存在蔬菜包的较为严重的短缺问题。从图 6-15 各区蔬菜包的浪费和缺货情况来看，蔬菜包浪费量最大的区是绿园区（浪费 94764 包），其次是宽城区（浪费 75122）。发生蔬菜包浪费问题频次最高的区是二道区发生了 11 次。蔬菜包缺货量最大的区是南关区（缺货 68255 包），其次是长春新区（缺货 55248.5 包），净月区（缺货 46953 包）。发生缺货问题频次最高的区是净月区发生了 12 次。发生浪费或缺货频次最高的区是二道区共发生了 21 次。

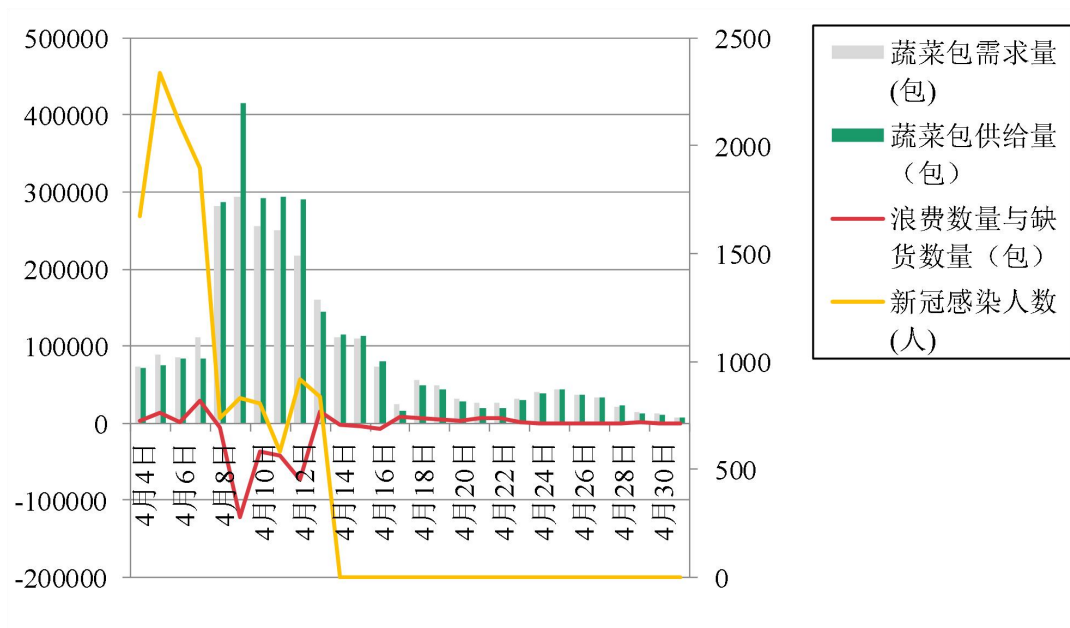


图 6-14. 蔬菜包的供需数量对比分析

	朝阳区	南关区	宽城区	绿园区	二道区	长春新区	经开区	净月区	汽开区
4月4日	0	-1620	-2588	0	3000	-4000	0	5810	0
4月5日	-2323	-260	-4650	0	4000	-45	300	-5374	-3560
4月6日	10000	-135	-850	0	14235	-4384	-6626	-5083	-5085
4月7日	-5101	12768	-2344	0	-12125	5695	0	-8871	-2510
4月8日	-100	19081	2927	0	-5234	3737	0	0	-16876
4月9日	-37	1120	31245	8656	4673	30268	2400	0	-3203
4月10日	2111	26042	21189	11960	97	-8687	47	0	-7817
4月11日	1792	3972	9875	12781	-1165	11735	4922.5	0	1654
4月12日	5995	1297	-4251	43446	13514	-18128	18324.5	4000	-4973
4月13日	-7466	-13428	99	-13158	19644	-5255	-994	-1396	-6045
4月14日	0	-18153.5	-874	0	5767	-12913	0	-4992	-2055
4月15日	561	-6407	3223	0	-87	-1326	4000	-2438	-4000
4月16日	0	-9224.5	5064	0	-4635	4672	5200	-3012	-1800
4月17日	-585	-3070	-6500	17921	-1168	7764	-4000	1496	1440
4月18日	0	-6040	-2438	0	1400	0	0	-4960	613
4月19日	0	-7887	-1025	0	602	-400	0	-2165	1024
4月20日	0	0	0	0	1600	1899	0	-1804	-2354
4月21日	0	-2000	0	0	-296	-111	-3300	-3429	0
4月22日	-2106	0	1500	0	-4096	1889	-3300	-3429	0
4月23日	520	0	0	0	-1393	0	0	0	1140
4月24日	0	0	0	0	0	0	0	0	1360
4月25日	0	0	0	0	-50	0	0	0	1550
4月26日	0	0	0	0	0	0	0	0	1544
4月27日	0	0	0	0	0	0	0	0	1500
4月28日	0	0	0	0	0	0	0	0	2400
4月29日	0	0	0	0	0	0	0	0	650
4月30日	0	0	0	0	0	0	0	0	1400
5月1日	0	0	0	0	0	0	0	0	2100

图 6-15. 各区蔬菜包的浪费和缺货情况

6.3. 子问题 2：调整蔬菜包供应方案——TOPSIS 综合评价与优化

6.3.1. TOPSIS 介绍

TOPSIS (Technique for Order Preference by Similarity to an Ideal Solution) 法是一种综合评价方法, 由 C.L.Hwang 和 K.Yoon 于 1981 年首次提出。TOPSIS 法根据有限个评价对象与理想化目标的接近程度进行排序的方法, 是在现有的对象中进行相对优劣的评价。TOPSIS 法是一种逼近于理想解的排序法, 该方法要求各效用函数具有单调递增 (或递减) 性就行。TOPSIS 法是多目标决策分析中一种常用的有效方法, 又称为优劣解距离法。建构 TOPSIS 评价体系的核心步骤如下所示:

步骤	操作
Step 1	1) 定义各个维度的理想化目标; 2) 计算各个维度距离理想化的距离 3) 保证各维度效用函数单调性
Step 2	TOPSIS 将数据指标分为四种类型 1) 极大型指标: 数值越大越好, 如收入、利润、成绩及肺活量等。 2) 极小型指标: 数值越小越好, 如成本、损失及污染程度等。 3) 中间型指标: 越接近某个固定值越好, 如水的 pH 值。 4) 区间型指标: 落在某个区间上最好, 如人的体温。
Step 3	数据正向化: 为了统一指标维度, 需要将数据指标正向化, 即让其指标数值越大越好; 1) 极大型指标不需要做正向化处理; 2) 常用极小型指标转换公式。如式 (6-2) 所示。 3) 中间型指标转换, 计算 M 值 (即距离最好的固定值距离) 和转换公式。如式 (6-3) 和式 (6-4) 所示。 4) 区间型指标转换。假设最好的区间为 $[a, b]$ 同样计算 M 值 (即距离最好的区间值距离), 如公式 (6-5) 所示。转换公式如式 (6-6) 所示。
Step 4	数据模标准化处理: 为了消除各数据指标之间的量纲影响, 对其进行模标准化, 公式如 (6-7) 所示。
Step 5	构建评分: 1) 找出最优数据指标点, 即各个指标维度的最大值, 计算当前样本距离该最优数据点的距离, 记为 D^+ ; 2) 找出最劣数据指标点, 即各个指标维度的最小值, 计算当前样本距离该最劣数据点的距离, 记为 D^- ; 3) 构建评分 C 并对样本进行排序, 得到最终结果。计算方法如式 (6-8) 所示。

$$x_i = \begin{cases} max - x_i & (\text{通用公式}) \\ \frac{1}{x} & (\text{数据指标均为正的情况下}) \end{cases} \quad (6-2)$$

$$M = \max\{|X - x_{\text{best}}|\} \quad (6-3)$$

$$x_i = 1 - \frac{(x_i - x_{\text{best}})}{M} \quad (6-4)$$

$$M = \max\{a - \min\{X\}, \max\{X\} - b\} \quad (6-5)$$

$$x_i = \begin{cases} 1 - \frac{a - x_i}{M} & x < a \\ 1 & a \leq x \leq b \\ 1 - \frac{x_i - b}{M} & x > b \end{cases} \quad (6-6)$$

$$z_i = \frac{x_i}{\sqrt{\sum_{i=1}^n x_i^2}} \quad (6-7)$$

$$C = \frac{(D -)}{D_+ + D_-} \quad (6-8)$$

Python 实现请见附录，4 月 10 日至 4 月 15 日 TOSIS 指标体系权重与各区得分和排名请见附件。

6.3.2. 评价指标体系

本文构建的蔬菜包保供方案的 TOPSIS 评价指标体系如表 6-4 所示。具体做法是，依据附件 1 汇总 4 月 10 日到 4 月 15 日的各区本土感染人数和无症状感染者人数，提取并统计附件 3 中的各区的小区栋数、小区户数（户）和小区人口数（人），此外我们考虑到交通状况也是影响物资供应方案的重要因素，因此我们根据附件 3 中的街道编号统计出了各区的街道数量。

表 6-4. 蔬菜包保供方案的 TOPSIS 评价指标体系

时间	区	评价指标					
		小区栋数	小区户数（户）	小区人口数（人）	街道数目（个）	本土感染人数（例）	无症状感染者人数（例）
4 月 10	朝阳区	3740	219874	548179	69	3	32
4 月 10	南关区	2336	183877	462146	68	7	55
...	...	...	...	...	...	...	...
4 月 11	宽城区	1524	128899	306143	58	34	151
4 月 11	绿园区	2198	155261	372541	57	6	158
...	...	...	...	...	...	...	...
4 月 12	二道区	1820	166428	405784	50	8	51
4 月 12	长春新区	2116	141780	338274	29	6	55
...	...	...	...	...	...	...	...
4 月 13	净月区	1507	85748	204056	59	7	26
4 月 13	汽开区	1171	81675	193462	54	5	9
...	...	...	...	...	...	...	...
4 月 14	朝阳区	3740	219874	548179	69	7.2	10
4 月 14	南关区	2336	183877	462146	68	18.72	5
...	...	...	...	...	...	...	...
4 月 15	净月区	1507	85748	204056	59	4	17.81
4 月 15	汽开区	1171	81675	193462	54	10	6.165

6.3.3. 评价结果

各区的评分结果展示如表 6-5 所示。使用 TOPSIS 评价 4 月 10 日到 4 月 15 日每天每个区的得分情况作为制定蔬菜包供应方案的依据。由此将获得每天各评价指标的权重如表 6-6 所示（以 4 月 10 日的指标权重计算结果为例），其中小区栋数的权重为 12.499%、小区户数（户）的权重为 16.921%、小区人口数（人）的权重为 18.96%、感染人数的权重为 21.894%、无症状感染者的权重为 22.384%、街道数目的权重为 7.342%，其中指标权重最大值为无症状感染者（22.384%），最小值为街道数目（7.342%）。在此基础上，可以各区的综合得分和排名，如表 6-7 所示（以 4 月 10 日为例）。其中， D^+ 和 D^- 值，分别代表评价对象与最优或最劣解（即 A^+ 或 A^- ）的欧式距离，其实际意义在于评价对象与最优或最劣解的距离，值越大说明距离越远，研究对象 D^+ 值越大，说明与最优解距离越远； D^- 值越大，说明与最劣解距离越远。综合得分越大说明研究对象越好。中间值计算结果如表 6-8 所示。

表 6-5. 蔬菜包保供方案的 TOPSIS 评价各区综合得分情况

时间	区	综合得分	求和
4 月 10	朝阳区	0.46537736	3.11868048
4 月 10	南关区	0.40893815	3.11868048
4 月 10	宽城区	0.66279791	3.11868048
4 月 10	绿园区	0.53561046	3.11868048
4 月 10	二道区	0.34455529	3.11868048
4 月 10	长春新区	0.26565694	3.11868048
4 月 10	经开区	0.17950934	3.11868048
4 月 10	净月区	0.14693533	3.11868048
4 月 10	汽开区	0.1092997	3.11868048
4 月 11	朝阳区	0.52775655	3.19815469
4 月 11	南关区	0.44533184	3.19815469
4 月 11	宽城区	0.63806236	3.19815469
4 月 11	绿园区	0.497046	3.19815469
4 月 11	二道区	0.34045462	3.19815469
4 月 11	长春新区	0.28826481	3.19815469
4 月 11	经开区	0.1529691	3.19815469
4 月 11	净月区	0.19437041	3.19815469
4 月 11	汽开区	0.113899	3.19815469
4 月 12	朝阳区	0.51438545	3.07800004
4 月 12	南关区	0.42111607	3.07800004
4 月 12	宽城区	0.65760758	3.07800004
4 月 12	绿园区	0.45946199	3.07800004
4 月 12	二道区	0.32338452	3.07800004
4 月 12	长春新区	0.26319514	3.07800004
4 月 12	经开区	0.29535501	3.07800004
4 月 12	净月区	0.14349428	3.07800004
4 月 12	汽开区	0.10915525	3.07800004
4 月 13	朝阳区	0.41640057	2.78678981
4 月 13	南关区	0.36289706	2.78678981
4 月 13	宽城区	0.69920281	2.78678981
4 月 13	绿园区	0.4259294	2.78678981

4 月 13	二道区	0.30964448	2.78678981
4 月 13	长春新区	0.22633662	2.78678981
4 月 13	经开区	0.11533849	2.78678981
4 月 13	净月区	0.13482233	2.78678981
4 月 13	汽开区	0.09621805	2.78678981
4 月 14	朝阳区	0.41693092	2.78817051
4 月 14	南关区	0.36071757	2.78817051
4 月 14	宽城区	0.69696051	2.78817051
4 月 14	绿园区	0.42326617	2.78817051
4 月 14	二道区	0.30783534	2.78817051
4 月 14	长春新区	0.22579334	2.78817051
4 月 14	经开区	0.10846292	2.78817051
4 月 14	净月区	0.13490041	2.78817051
4 月 14	汽开区	0.11330333	2.78817051
4 月 15	朝阳区	0.4195516	2.80742699
4 月 15	南关区	0.36077513	2.80742699
4 月 15	宽城区	0.69460301	2.80742699
4 月 15	绿园区	0.43217672	2.80742699
4 月 15	二道区	0.30893402	2.80742699
4 月 15	长春新区	0.22926816	2.80742699
4 月 15	经开区	0.11377876	2.80742699
4 月 15	净月区	0.13600009	2.80742699
4 月 15	汽开区	0.1123395	2.80742699

表 6-6. 蔬菜包保供方案的 TOPSIS 评价指标权重（以 4 月 10 日为例）

评价指标	TOPSIS		
	信息熵值 e	信息效用值 d	权重(%)
小区栋数	0.876	0.124	12.499
小区户数（户）	0.832	0.168	16.921
小区人口数（人）	0.811	0.189	18.96
感染人数	0.782	0.218	21.894
无症状感染者	0.777	0.223	22.384
街道数目	0.927	0.073	7.342

表 6-7. 蔬菜包保供方案中各区的综合得分与排名（以 4 月 10 日为例）

索引值	正理想解距离（D+）	负理想解距离（D-）	综合得分指数	排序
朝阳区	0.43174588	0.48249842	0.52775655	2
南关区	0.44869475	0.36024793	0.44533184	4
宽城区	0.31381615	0.55322867	0.63806236	1
绿园区	0.41786229	0.41295383	0.497046	3
二道区	0.51620738	0.26646413	0.34045462	5
长春新区	0.53799241	0.21789604	0.28826481	6
经开区	0.63752883	0.11513418	0.1529691	8
净月区	0.60478974	0.14591473	0.19437041	7

表 6-8. 中间值计算结果（以 4 月 10 日为例）

项	正理想解	负理想解
小区栋数	0.71336706	2e-8
小区户数（户）	0.62073166	0
小区人口数（人）	0.6345996	0
感染人数	0.91425388	0.00000269
无症状感染者	0.67670733	4.4e-7
街道数目	0.48126413	0.0000012

6.3.4. 结果分析

根据各区的每天的综合得分，按照每天各区的得分比重计算出每天合理的蔬菜包供应量。为评价蔬菜包供应方案的调整效果，本文利用数据挖掘技术总结和统计出 4 月 10 日至 4 月 15 日，各区每天的蔬菜包接收数目和发放数目如表 6-9 所示。其中，发放数目是指各区实际发放给管辖小区的蔬菜包数量，因此本文将其定义为小区居民每日蔬菜包的需求量。接收数目是指各区接收到的市州支援、市级集采和属地自保的蔬菜包，表示各区每日蔬菜包的供应能力，即每日可提供给其管辖小区的蔬菜包的数量。

由于蔬菜包属于易腐商品，特别是在包装或仓储条件不满足保鲜要求时极易在短时间内发生霉烂和变质。通过查阅资料我们发现，长春市的蔬菜包一般采用纸箱包装如图 6-16 所示，这意味着蔬菜包在不具保鲜作用的纸箱中隔夜存放很大概率会发生变质。因此，本文假设假设蔬菜包隔夜存放第二天会变质不能再继续发放给隔离群众。每区当日的蔬菜包接收量等价于每日每区的供应能力，每区当日的蔬菜包发放量等价于每日各区隔离群众的需求量。附件 5 中所存在的每日发放量大于接收量的情况视为缺货，反之视为浪费。将蔬菜包浪费或缺货数目进行对比作为供应方案调整效果的评价指标。各区蔬菜包供应方案的调整计算结果如表 6-10 所示，各区调整效果对比如表 6-11 所示。蔬菜包供应方案调整后九个区的蔬菜包浪费量或缺货问题得到明显改善。

表 6-9. 各区每天的蔬菜包接收数目和发放数目（2022.4.10-2022.4.15）

时间	区	发放数目(需求数目)	接收数目(供应数目)
4 月 10	朝阳区	38244	40355
4 月 10	南关区	25285	27671
4 月 10	宽城区	62226	83415
4 月 10	绿园区	32812	44772
4 月 10	二道区	26908	27005
4 月 10	长春新区	17874	9187
4 月 10	经开区	24360	24407
4 月 10	净月区	11594	11594
4 月 10	汽开区	8777	17136
4 月 11	朝阳区	37189.5	38981
4 月 11	南关区	23223.5	23013
4 月 11	宽城区	13251	23126
4 月 11	绿园区	57047	69828
4 月 11	二道区	29122	27957

4月11	长春新区	21893.5	33628
4月11	经开区	46424.5	51347
4月11	净月区	7462	7462
4月11	汽开区	14722	16376
4月12	朝阳区	23125	29120
4月12	南关区	14447	22668
4月12	宽城区	12800	8549
4月12	绿园区	74611	118057
4月12	二道区	17698	31212
4月12	长春新区	31737	13609
4月12	经开区	16663.5	34988
4月12	净月区	19333	23333
4月12	汽开区	5862.5	8417
4月13	朝阳区	19475.5	12010
4月13	南关区	17111.5	11857
4月13	宽城区	2737	2836
4月13	绿园区	39967	26809
4月13	二道区	10637	30281
4月13	长春新区	25254.5	20000
4月13	经开区	13994	13000
4月13	净月区	24522	23126
4月13	汽开区	6044.5	5431
4月14	朝阳区	11874	11874
4月14	南关区	5070	5070
4月14	宽城区	5385	4511
4月14	绿园区	16356	16356
4月14	二道区	7142	12909
4月14	长春新区	17913	5000
4月14	经开区	20000	20000
4月14	净月区	18948	13956
4月14	汽开区	7500	16127
4月15	朝阳区	8285	8846
4月15	南关区	8040	8040
4月15	宽城区	10244	13467
4月15	绿园区	22380	22380
4月15	二道区	5748	5661
4月15	长春新区	8731	7405
4月15	经开区	25000	29000
4月15	净月区	14136	11698
4月15	汽开区	5000	5000

数据来源：依据附件 5 整理而得。



图 6-16. 长春市的蔬菜包的包装规格

图片来源: [多种“蔬菜包”供选择, 长春长青街道送菜忙 \(baidu.com\)](http://www.baidu.com)

表 6-10. 各区蔬菜包供应的优化方案与调整幅度

时间	区	得分占比	调整后的供应方案(包) 接收蔬菜包*得分占比	调整幅度
4 月 10	朝阳区	0.1492225	43573	7.97%
4 月 10	南关区	0.1311254	38289	38.37%
4 月 10	宽城区	0.2125251	62058	-25.60%
4 月 10	绿园区	0.1717427	50149	12.01%
4 月 10	二道区	0.1104811	32261	19.46%
4 月 10	长春新区	0.0851825	24873	170.75%
4 月 10	经开区	0.0575594	16807	-31.14%
4 月 10	净月区	0.0471146	13758	18.66%
4 月 10	汽开区	0.0350468	10234	-40.28%
4 月 11	朝阳区	0.1650191	48436	24.26%
4 月 11	南关区	0.1392465	40871	77.60%
4 月 11	宽城区	0.1995095	58560	153.22%
4 月 11	绿园区	0.1554165	45618	-34.67%
4 月 11	二道区	0.1064535	31246	11.76%
4 月 11	长春新区	0.0901347	26456	-21.33%
4 月 11	经开区	0.0478304	14039	-72.66%
4 月 11	净月区	0.0607758	17839	139.06%
4 月 11	汽开区	0.035614	10453	-36.17%
4 月 12	朝阳区	0.1671168	48605	66.91%
4 月 12	南关区	0.1368148	39792	75.54%
4 月 12	宽城区	0.2136477	62138	626.84%
4 月 12	绿园区	0.1492729	43415	-63.23%
4 月 12	二道区	0.1050632	30557	-2.10%

4月12	长春新区	0.0855085	24870	82.74%
4月12	经开区	0.0959568	27908	-20.23%
4月12	净月区	0.0466193	13559	-41.89%
4月12	汽开区	0.035463	10314	22.54%
4月13	朝阳区	0.1494194	21718	80.83%
4月13	南关区	0.1302205	18928	59.63%
4月13	宽城区	0.250899	36468	1185.90%
4月13	绿园区	0.1528387	22215	-17.14%
4月13	二道区	0.1111115	16150	-46.67%
4月13	长春新区	0.0812177	11805	-40.98%
4月13	经开区	0.0413876	6016	-53.73%
4月13	净月区	0.0483791	7032	-69.59%
4月13	汽开区	0.0345265	5018	-7.60%
4月14	朝阳区	0.1495357	17097	43.98%
4月14	南关区	0.1293743	14791	191.75%
4月14	宽城区	0.2499705	28579	533.55%
4月14	绿园区	0.1518078	17356	6.12%
4月14	二道区	0.1104076	12623	-2.22%
4月14	长春新区	0.0809826	9259	85.18%
4月14	经开区	0.0389011	4448	-77.76%
4月14	净月区	0.0483831	5532	-60.36%
4月14	汽开区	0.0406372	4646	-71.19%
4月15	朝阳区	0.1494435	16999	92.16%
4月15	南关区	0.1285074	14617	81.81%
4月15	宽城区	0.2474162	28143	108.98%
4月15	绿园区	0.1539405	17510	-21.76%
4月15	二道区	0.1100417	12517	121.11%
4月15	长春新区	0.0816649	9289	25.44%
4月15	经开区	0.0405278	4610	-84.10%
4月15	净月区	0.048443	5510	-52.90%
4月15	汽开区	0.0400151	4552	-8.97%

蔬菜包保供方案调整效果对比如表 6-11 所示。本文基于 TOSIS 综合评价结果调整各区蔬菜保供应方案能有效减少蔬菜包每日浪费或缺货的问题，从改善效果来看能减轻 13.84% 的浪费或缺货数量，降低 14.8% 的浪费问题发生频次，降低 25% 的缺货问题发生频次。综上，本文提出的蔬菜包供应调整方案有效且可行。

表 6-11. 蔬菜包供应方案的调整效果评价结果

效果评价指标	浪费或缺货数量		浪费发生的频次		缺货发生的频次	
日期	调整前	调整后	调整前	调整后	调整前	调整后
2022/4/10	36540	36540	7	7	1	2
2022/4/11	42635	40735	6	4	2	1
2022/4/12	73717	54031.178	7	4	0	0
2022/4/13	-14621	-12621	2	1	7	6

2022/4/14	1949	1877	2	4	3	2
2022/4/15	3388	3128	3	3	3	1
总计	172850	148932.178	27	23	16	12
总体优化效果	减轻 13.84%		降低 14.81%		降低 25%	

七、问题四分析与求解

7.1. 问题分析

要求在第二、三问的基础上，结合附件 3 给出的中长春市街道和小区情况，制作特殊时期保障居民生活物资供应的详细预案（有序网络图）。该网络是一个多级物流网络由上游物资来源地、中游物资集散地和下游长春市所有小区，以及两种规格的卡车（大卡车限载 10 吨，小卡车限载 4 吨）组成。此问题根据有序网络的层级，可以分解为三个子问题：子问题一是对上游九个物资来源地进行合理选址，子问题二是对中游物资集散地进行合理的分配和选址，子问题三是在考虑车辆限重的情况下，将“工作量”（工作量 = 运输里程 × 小区居民人数）的最小化合理规划末端配送路径以及配送量的重要目标。子问题一的难点在于附件三并未给出交通路口节点的区域划。子问题二和子问题三的难点在于数据规模较大且所建立的数学模型属于混合 0-1 整数非线性问题难以快速找到全局最优解。

针对子问题一，根据附件 3 小区的坐标边界，采用射线法法“由点连面”，用于判定交通路口节点是否在小区群边界内，克服了附件 3 交通路口节点数据缺失区域划分的问题，同时能够过滤掉冗余的交通路口节点信息。采用枚举法在保留下来的交通路口节点中寻找到所有小区的“工作量”指标之和最小的位置。每个区重复上述方法便可得到各区的中心点坐标，即为九个上游物资来源地。

针对子问题二，本文采用轴辐射网络对中游的集散地进行选址和分配。针对子问题三，构建考虑病毒传播和车辆容量限制的路径优化模型。鉴于此问题属于 NP-hard 问题，本文采用 ALNS 和 SA 构成的混合启发式算法进行求解，最终得到由上游物资来源地-中游货物集散地-下游小区的多级有序物流网络、最短配送路径以及合理的配送车辆数量与种类。

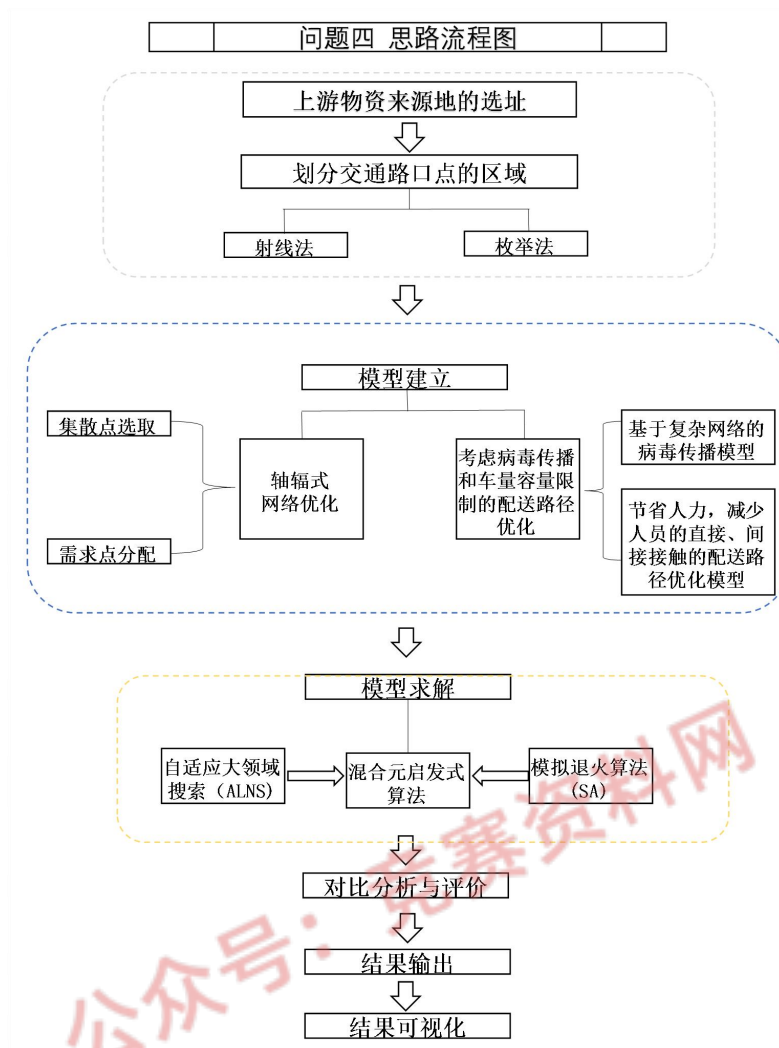


图 7-1. 问题四的思路流程图

7.2. 上游物资来源地的选址——射线法、枚举法

7.2.1. 划分交通路口点的区域

上游物资来源地的选址需要依据附件 3 的交通路口节点数据。但是由于该数据并未给出每个交通路口节点的区域划分，因此不能直接获取各区交通路口节点状况，也就不能保证每个区都有一个上游物资来源地的目标。针对此问题，本文首先采用射线法判断交通路口点是否位于由全区小区构成的不规则形状内，若在则该区交通路口点划分到该区，否则从该区中剔除。通过此步骤能够克服附件 3 交通路口节点数据缺失区域划分的问题，同时能够过滤掉冗余的交通路口节点信息。射线法的一般思路如表 7-1 所示。（实现代码请见附录-问题四算法程序）

表 7-1. 射线法的一般思路

步骤	操作
Step 1	从目标点出发朝 x 轴正方向引一条射线，计算此射线与小区构成的多边形所有边的交点数目；
Step 2	若交点个数为奇数，则目标点在小区构成的多边形内部； 若交点个数为偶数，则目标点在小区构成的多边形外部。

7.2.2. 确定上游物资来源地在各区的选址

采用枚举法在保留下来的交通路口节点中寻找到所有小区的“工作量”指标之和最小的位置，计算方法如式（7-1）所示。每个区重复上述方法便可得到各区的中心点坐标，即为九个上游物资来源地。九个上游来源地坐标结果如表 7-2 所示。

$$\sqrt{\sum_{i=1}^n (x_i - y_i)^2 * \text{各小区人口数}} \quad (7-1)$$

表 7-2. 上游物资来源地位置与行政区域划分

No.	区	编号	x	y
1	长春新区	3191	38.79708	20.7877
2	汽开区	3849	34.29753	36.3538
3	南关区	6797	57.00713	33.26233
4	绿园区	761	42.16913	49.3732
5	宽城区	7701	55.64605	59.49537
6	净月区	7425	72.76918	17.93157
7	经开区	2258	74.4338	37.50083
8	二道区	2558	69.9437	46.98793
9	朝阳区	988	49.3975	38.09937

7.3. 中游集散点的辐射网络模型

轴辐式网络优化可以分为两层：一方面包括集散点的选取与运输网络的构建；另一方面包括需求点的分配与配送网络的构建，相比较于基础的选址模型，轴辐式网络具有更加复杂的网络结构，模型的求解也相对较为困难。本文中基于启发式算法求解轴辐式网络问题，首先可以将算法看做两个层级，第一层级用于集散点的选取，采用了随机生成的方式生成所需要的枢纽。第二层用于需求点的分配，结合附件 3 的路线实际距离计算节点到各个集散点的距离，将节点分配到最近的集散点。在构建模型时以最小工作量为目标函数（工作量：工作量按运输里程与小区居民人数乘积计算），对于集散点的数量和需求节点与集散点之间的连接进行了相关约束。

$$\min \left(\sum_{ik} P^C C_{ik} O_i x_{ik} + \sum_{ikh} P^T C_{kh} y_{ikh} + \sum_{ik} P^D C_{ki} D_i x_{ik} \right) \cdot N_p \quad (7-2)$$

s.t.

$$\sum_k x_{ik} = 1, \forall i \quad (7-3)$$

$$x_{ik} \leq x_{kk}, \forall i, k \quad (7-4)$$

$$\sum_k x_{kk} = H \quad (7-5)$$

$$\sum_h y_{ikh} - \sum_h y_{ihk} = O_i x_{ik} - \sum_j W_{ij} x_{jk}, \forall i, k \quad (7-6)$$

$$\sum_{h \neq k} y_{ikh} \leq O_i x_{ik}, \forall i, k \quad (7-7)$$

$$x_{ik} \in \{0,1\}, \forall i, k \quad (7-8)$$

$$y_{ikh} \geq 0, \forall i, k, h \quad (7-9)$$

(7-3)为约束条件，表示一个需求节点只分配一个集散点；(7-4)为约束条件，表示一个需求节点只能和一个集散点连接；(7-5)对集散点的数量进行限制；约束(7-7)确定每个需求节点在集散点路径上的流量；(7-8)表示 x_{ik} 为 0-1 整数变量；(7-9)表示 y_{ikh} 为大于 0 的连续型变量

表 7-3. 中游集散点的辐射网络模型的集合、数据与变量

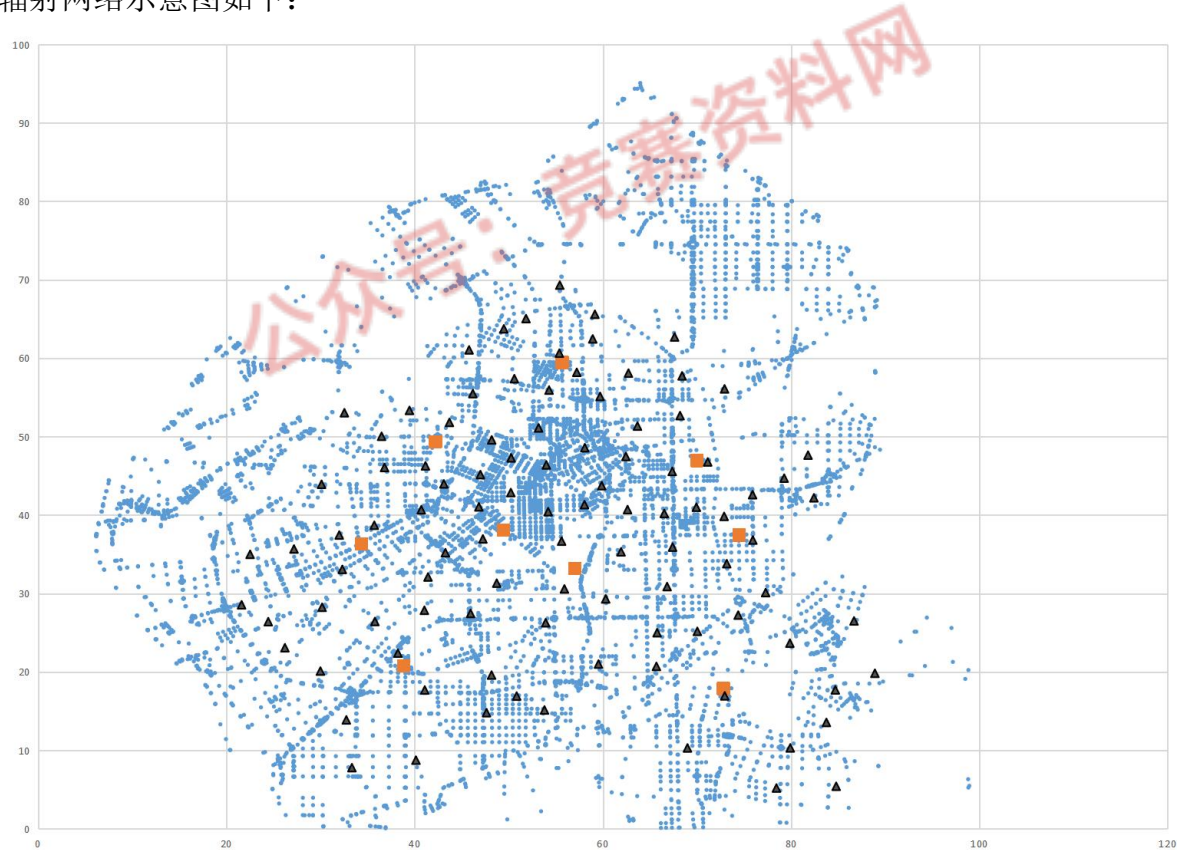
集合	
N_p	小区居民人数
H	需要选取的集散点个数
Q	集散点容量的集合
数据	
P	需要选取的集散点数量
K	车辆集合 $K \in \{1,2, \dots, K \}$
P^C, P^D, P^T	表示辐节点到集散点，集散点到集散点，集散点到辐节点的单位距离费用
W_{ij}	需求节点之间的流量 $i, j \in N$
$O_i = \sum_j W_{ij}$	流出节点 i 的流量 $i \in N$
$D_i = \sum_j W_{ji}$	流入节点 i 的流量 $j \in N$
变量	
x_{ik}	表示需求节点 i 分配到集散点 k ， $x_{kk} = 1$ 表示 k 为集散点， $i, k \in N$
y_{ikh}	表示需求节点 i 经过集散点 k 到达集散点 h 的流量 $i, k, h \in N$
$x_{ij} \in \{0,1\}$	表示需求节点与集散点的服务关系，若需求节点 i 分配到集散点 j ， $x_{ij} = 1$ ，否则 $x_{ij} = 0$
$y_i \in \{0,1\}$	表示集散点 i 是否开放

求得的中游物资集散地的位置如表 7-4 所示。

表 7-4. 基于轴辐射网络的中游物资集散地选址结果

No.	x 坐标	y 坐标	行政区域划分
1	88.89116	19.82388	净月区
2	41.15211	46.26814	绿园区
3	69.92888	40.99724	二道区
4	38.22631	22.3927	长春新区(高新)
5	58.92209	62.45739	宽城区
.....			
96	31.99186	37.44983	汽开区
97	68.19788	52.67407	二道区
98	50.22061	42.86536	朝阳区
99	43.10932	43.9721	汽开区
100	58.07429	48.57984	宽城区

轴辐射网络示意图如下：



7.4. 考虑病毒传播和车量容量限制的路径优化模型

7.4.1 基于复杂网络的病毒传播模型

重要的生活物资运送服务将导致病毒在人与人之间传播。复杂网络模拟人群的分组和相互作用，因此能够很好地描述疫情传播过程。涉及流行病爆发阈值的设置以及受感染者

随时间的演变特征。复杂网络包含两个元素：节点和边。对于生活物资交付流程，节点表示小区、集散点。边缘表示小区、集散点之间的距离。

复杂网络用于描述由于生物物质输送而导致的新冠肺炎在小区之间传播的动态过程。一般来说，一个小区*i*包含了一些感染和疑似新冠肺炎病例的人群。考虑到这些 COVID 病例的不同传播能力，我们将其设置为不同的病毒传播风险。因此，邻域可以用初始 COVID 概率 r_i 描述为等式（7-10），其中 P_c^i 、 P_s^i 表示确诊和无症状新冠肺炎病例的数量。 ε_0 和 ε_1 是这两种情况的相应传输概率设置为 1 和 0.15。通过规范化小区的概率，我们将小区中的 COVID 感染风险 β_i 统一为等式（7-11），其中 r_{min} 和 r_{max} 是城市所有小区的最小和最大风险值。新冠肺炎病例越多， β_i 值越大。

$$r_i = \varepsilon_0 \times P_c^i + \varepsilon_1 \times P_s^i \quad (7-10)$$

$$\beta_i = \frac{r_i - r_{min}}{r_{max} - r_{min}} \quad (7-11)$$

当送货员在城市送货时，由于人与人或人与物之间的必要接触，新冠肺炎可能被传播。一般来说，从集散点到小区的生活物资分布可以用有向图 $G(V, E)$ 来描述， $V = \{v_1, v_2, \dots, v_n\}$ 是所有节点和 $E = \{e_{12}, e_{23}, \dots, e_{ij}\} (i, j \in V)$ 的是所有边的集合。在本文中，节点表示小区、集散点和物资来源点。边缘表示从一个小区到另一个小区的旅行路线段。

病毒在传播过程中分两种情况。一种情况是，当送货员分发生活物资时，新冠肺炎可能会从小区居民传播给送货员，具体取决于接触者和病毒预防水平。这里，交付的病毒传播风险由 θ_i 表示交付中的接触方式。 θ_i 的范围在 0 和 1 之间。从形式上讲，一次发放物资行为的传递概率可以计算为（7-12），其中 d_i 是小区*i*的保护水平， Δt 是送货员在小区*i*停留的时间，由装卸时间决定。装卸时间越长，生活物资、送货员和居民之间的接触越多，病毒传播概率 p_i 越高。在本文中，我们将 θ_i 的值设为 1。

$$p_i = (1 - \exp(-d_i \times \Delta t)) \times \theta_i \quad (7-12)$$

另一种情况是病毒可能从送货员传播给居民。一般来说，送货员服务的小区越多，病毒传播风险就越高。因此，病毒传播的风险应该沿着送货员的路线累积。在一次传播后，邻居的感染风险 β_i^+ 等于感染前的风险 β_i^- ，以及该交付的附加风险 $\gamma_{hi} \times p_i$ ，如式（7-13）所示。对于送货员来说，一次送货行动后的病毒传播风险 γ_{ij} 等于之前的风险加上访问节点的风险，如式（7-14）所示，其中 γ_{hi} 和 γ_{ij} 分别是边缘 e_{hi} 和 e_{ij} 的风险， h 是之前访问的节点。配送网络的风险最终由指标 δ 表示，如式（7-15）所示，等于所有小区的累积风险和配送后送货员的累积风险。所有交付完成的时间用 T 表示。

$$\beta_i^+ = \beta_i^- + \gamma_{hi} \times p_i \quad (7-13)$$

$$\gamma_{ij} = \gamma_{hi} + \beta_i^- \times p_i \quad (7-14)$$

$$\delta = \sum_{i \in V} \beta_i^+(T) + \sum_{i, j \in V, j \neq i} \gamma_{ij} \quad (7-15)$$

7.4.2 考虑节省人力，减少人员的直接、间接接触的配送路径优化模型

我们构建了配送路径优化模型，目标是节省人力，并减少因必要的生活物资输送而导致的病毒传播风险。在这里，我们假设仓库将有足够的物流车辆。小区的生活用品订单被分配给最近的集散点。一辆车从物资来源地出发，在集散点领取生活用品，将其送到相应的物资投放点，然后返回出发的物资来源地。汽车在城市里四处奔波，运送生活用品包，直到物资投放点的所有订单都送达为止。模型定义如下：

D	物资来源点集合
$H = H_1 + H_2$	车辆集合
M	集散点集合
C	小区集合
d	物资来源点
H_1	负载能力为 4 的车辆
H_2	负载能力为 10 的车辆
m	集散点

假设所有车辆离开各自的物资来源点并最终返回物资来源点车辆负载不得超过其负载能力 H_1, H_2 , 送货员的工作时间不得超过规定的工作时间 MT 。

配送路径优化模型可以表示为有向图 $G(V, E)$ 。图 V 的节点 $= \{D, M, C\}$ 是物资来源点、集散点和小区的交集。边缘 E 表示从一个节点移动到另一个。每条边 e_{ij} 表示从节点 i 到节点 j 的一个路段, 并带有行驶距离 d_{ij} 。

新冠病毒传播因素涉及以下两个部分: 新冠肺炎传播风险和旅行距离。在模型中总传播风险包括所有小区的风险和所有送货员的风险。一般来说, 从一家集散点直接运送到一个小区将产生最小的病毒传播风险, 因为不存在从小区到小区的间接病毒传播。然而, 这将需要更多数量的货车和送货员, 加大了送货员接触新冠肺炎。因此, 为时目标协调使用加权参数 α , 考虑新冠病毒传播的配送路径优化模型转换为单目标问题, 并构建了混合整数规划模型。模型如下所示:

决策变量	
$x_{ij}^{dk} \in (0,1)$	二进制变量, 如果从物资来源点 d 出发的车辆 k 从节点 i 行驶到节点 j , 则等于 1
T_i^{dk}	k 车在节点 i 离开物资来源点 d 的到达时间
Q_i^{dk}	物资来源点 d 离开的 k 车离开节点 i 时的载荷
$\beta_i^+(T), \beta_i^-(T)$	T 后和 T 前节点 i 的病毒传播风险
γ_{ij}^{dk}	k 车从 d 站从节点 i 行驶到节点 j 的病毒传播风险。

目标函数:

$$\begin{aligned} \min F = & \alpha \times \left(\sum_{i \in V} \beta^+(T_i) \right. \\ & \left. + \sum_{d \in D} \sum_{k \in H_d} \sum_{i \in V} \sum_{j \in D} \gamma_{ij}^{dk} \right) + (1 - \alpha) \times \sum_{d \in D} \sum_{k \in H_d} \sum_{i \in V} \sum_{j \in V, i \neq j} d_{ij} \times x_{ij}^{dk} \end{aligned} \quad (7-16)$$

约束条件:

$$\beta_i^+(T_i^{dk}) = \beta_i^-(T_i^{dk}) + x_{hi}^{dk} \times \gamma_{ij}^{dk} \times p_i, \forall d \in D, k \in H_d \quad (7-17)$$

$$\gamma_{ij}^{dk} = \gamma_{hi}^{dk} \times x_{hi}^{dk} + \beta_i^+(T_i^{dk}) \times p_i \times x_{hi}^{dk}, \forall i, j \in V, d \in D, k \in H_d \quad (7-18)$$

$$\sum_{d \in D} \sum_{k \in H_d} \sum_{j \in V} x_{ji}^{dk} = \sum_{d \in D} \sum_{k \in H_d} \sum_{j \in V} x_{ij}^{dk}, \forall i \in V \quad (7-19)$$

$$\sum_{i \in D} \sum_{j \in M} x_{ij}^{dk} < |H_d|, \forall d \in D, k \in H_d \quad (7-20)$$

$$\sum_{j \in V} x_{dj}^{dk} = \sum_{j \in V} x_{id}^{dk} = 1, \forall d \in D, k \in H_d \quad (7-21)$$

$$Q_i^{dk} \leq MQ \leftarrow \sum_{j \in V} x_{ij}^{dk} = 1, \forall i \in V, d \in D, k \in H_d \quad (7-22)$$

$$T_j^{dk} = T_i^{dk} + t_{ij} + s_i, \leftarrow x_{ij}^{dk} = 1, \forall i, j \in V, d \in D, k \in H_d \quad (7-23)$$

$$T_i^{dk} \leq MT, \forall i \in V, d \in D, k \in H_d \quad (7-24)$$

$$T_i = \text{MAX}(T_i^{dk}), \forall i \in V \quad (7-25)$$

式（8）的第一部分表示总病毒传播风险，包括所有小区和所有送货人的累积风险。式（8）的第二部分表示所有配送路线的总长度。 α 和 $1-\alpha$ 是两个目标的相应权重。等式（9）和（10）表明，小区风险和送货员风险在一次生活材料运送行动后更新。等式（11）表明，小区*i*的到达车辆数量等于左侧车辆数量。等式（12）要求从一个物资来源点出发的车辆数量小于物资来源点内的车辆数量 $|H_d|$ 。等式（13）规定，一辆车必须返回出发物资来源点。（14）确保一辆车交付的生活物资总量小于车辆容量 MQ 。等式（15）规定，车辆到达节点*j*的时间等于前一节点*i*的到达时间加上节点*i*的服务时间和节点*i*到*j*的行程时间 MT 。等式（17）表示活性物质交付过程在节点*i*, T_i^{dk} 结束，等于最后一辆交付车辆的到达时间。

7.4.3 混合元启发式算法

当涉及数千个节点时，很难在合理的时间内得到全局最优解。在这里，我们开发了一种结合 ALNS（大规模邻域算法）和 SA（模拟退火算法）的混合元启发式算法来解决这个问题。

在大规模邻域搜索（LNS）中，邻域是由 **destroy** 和 **repair** 方法隐式定义的。**destroy** 方法会破坏当前解的一部分，**repair** 方法会对被破坏的解进行重建。**destroy** 方法通常包含随机性的元素，以便在每次调用 **destroy** 方法时破坏解的不同部分。那么，解 x 的邻域 $N(x)$ 就可以定义为：首先通过利用 **destroy** 方法破坏解 x ，然后利用 **repair** 方法重建解 $\bar{x}$ ，从而得到的一系列解的集合，图 1 展示了一种邻域生成方法。

ALNS 在 LSN 的基础上，允许在同一个搜索中使用多个 **destroy** 和 **repair** 方法来获得当前解的邻域。ALNS 会为每个 **destroy** 和 **repair** 方法分配一个权重，通过该权重从而控制每个 **destroy** 和 **repair** 方法在搜索期间使用的频率。在搜索的过程中，ALNS 会对各个 **destroy** 和 **repair** 方法的权重进行动态调整，以便获得更好的邻域和解。

邻域生成算子（operator）**destroy/repair** 成对出现，算法以“先删除后插入”的策略进行寻优；在寻优的同时，动态调整算子的被选概率。

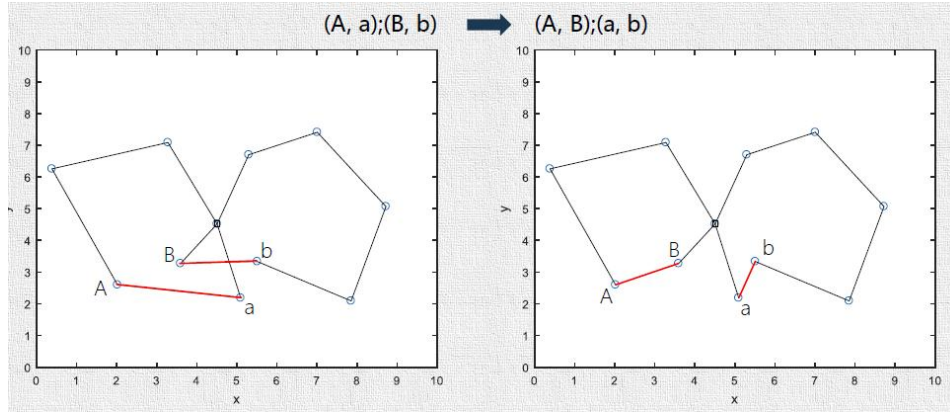


图 7-2. 2-opt method (local search algorithm)

在轴辐式网络问题求解中,可行解 x 是枢纽序列,即 $x ::= \langle i_1, i_2, \dots, i_{HUBS} \rangle$,则辐节点集合为 $\hat{x} = N - x$ 。下面设计 7 种邻域生成算子。

表 7.5. 邻域生算子

Operator	Define
$\tilde{x} \leftarrow \text{ExchangeOne}(x)$	Select one node between x and $\hat{x}$ randomly, exchange them and obtain a new solution $\tilde{x}$
$\tilde{x} \leftarrow \text{ExchangeTwo}(x)$	Select two nodes between x and $\hat{x}$ randomly, exchange them and obtain a new solution $\tilde{x}$
$\tilde{x} \leftarrow \text{ExchangeThree}(x)$	Select three nodes between x and $\hat{x}$ randomly, exchange them and obtain a new solution $\tilde{x}$
$\tilde{x} \leftarrow \text{SwapTwo}(x)$	Select two nodes randomly, swap them and obtain a new solution $\tilde{x}$
$\tilde{x} \leftarrow \text{New}(x)$	Select HUBS nodes from N randomly, and obtain a new solution $\tilde{x}$
$\tilde{x} \leftarrow \text{RandlOne}(x)$	Remove a node from x randomly, and insert a new solution $\tilde{x}$
$\tilde{x} \leftarrow \text{Reverse}(x)$	Select two nodes from x randomly, reverse them and obtain a new solution $\tilde{x}$

算子集合 $\lambda \in O = \{o_1, o_2, \dots, o_m\}$, 算子的初始分数 $I(\lambda) = M$, 算子改进当前解对应的分数增量为 $\Delta_\lambda = \{\Delta_1, \Delta_2, \dots, \Delta_m\}$ 。定义算子权重 $\omega_\lambda = \frac{I(\lambda)}{S(\lambda)}$, $\lambda \in O$, $I(\lambda)$ 是算子 λ 在最近一次迭代后的分数, $S(\lambda)$ 是已经被选中的所有算子当前的总分, $S(\lambda) = \sum_\lambda I(\lambda)$ 。采用轮盘赌选择策略,动态控制算子被选中的概率,直到满足停止准则,算法终止。算子被选中的概率 $P(\lambda) = \frac{I(\lambda)}{S(\lambda)} \cdot \frac{1}{\sum_{\lambda=1}^m \omega_\lambda}$, 其中 $S(\lambda) \neq 0$ 。ALNS 的伪代码如表 7-6 所示。

表 7-6. ALNS 的伪代码

Algorithm	Adaptive Larger Neighborhood Search
Input:	初始可行解 Π
Output:	当前最优解 Γ
Steps	
Step 1	初始化: 算子的初始分数 $I(\lambda) = M$, $\Gamma \leftarrow \Pi$, $\omega_\lambda = \frac{1}{m}$; 分数增量 α, β , $\alpha < \beta$

- Step 2 当满足停止准则, 算法终止; 否则, 转到 Step3.
- Step 3 基于当前权重 ω_λ , 通过轮盘赌选择策略, 依概率选择一个算子 λ , 并计算得解 Π^*
- Step 4 If $Makespan \Pi^* < Makespan \Pi$, then update $\Pi \leftarrow \Pi^*$; Set $I(\lambda) \leftarrow I(\lambda) + \alpha$
- Step 5 If $Makespan \Pi^* < Makespan \Gamma$, then update $\Gamma \leftarrow \Pi^*$; Set $I(\lambda) \leftarrow I(\lambda) + \beta$
- Step 6 当连续 3 次迭代, $Makespan \Gamma$ 未改进, Clear $I(\lambda), S(\lambda)$, Set $\omega_\lambda = \frac{1}{m}$. 并返回 Step2.
-

混合元启发式算法包括两个步骤: (1) 构造一个好的初始解; (2) 通过 ALNS 和 SA 对当前解决方案进行迭代改进。1) 初始解决方案: 初始解决方案生成方法迭代地将活性物质订单插入路线。工作流程如算法 1 所示。首先, 为 H (1 号线) 的所有车辆生成从物资来源点开始的初始空路线。然后, 随机选择一个订单 c_i 。将选择一条交货路线 r_d 来插入此订单。路线 r_d 中的最佳插入位置 (i, j) 是插入成本最小的位置 (第 3-9 行)。插入操作将重复, 直到插入所有订单。

Algorithm 1 Construct the Initial Solution

Input: H: the vehicle set; C: the neighborhood demand order set

Output: the initial solution S

```

1:  $R \equiv \{r_1, r_2, \dots, r_d\} \leftarrow \text{initialize route for } h_d \in H$ 
2: for all  $c_i \in C$  do
3:    $f_{min} \leftarrow \infty$ 
4:   for all  $r_d \in R$  do
5:     if  $f_{min} > \text{insert\_cost}(i, j)$  and  $q_d + q_j < Q_d$  then
6:        $\text{best\_insert} = (r_d, i, j)$ 
7:        $f_{min} = \text{insert\_cost}(i, j)$ 
8:     end if
9:   end for
10:   $\text{insert\_order}(r_d, i, j), q_d += q_j$ 
11: end for

```

Algorithm 2 ALNS

Input: M Supermarkets set; C Neighborhoods set;

Max_Iter Maximum iterations; S initial solution.

Output: Best solution S^*

```

1: Initialize parameters, set  $S^* = S$ 
2: for  $k = 1$  to  $Max\_Iter$  do.
3:   Choose a destroying operator  $O_i$ 
4:   Remove Q nodes from S by  $O_i$ 
5:   Reinsert Q nodes by the repair operator, produce a new solution  $S'$ ,
6:   Accept current solution  $S'$  by simulated annealing
7:   if accept  $S'$  then
8:      $S = S'$ 
9:   if  $f(S') < f(S^*)$  then

```

```

10:    $S^* = S'$ 
11:   end if
12:   Update the weight of destroying operators  $\pi_i$ 
13: end if
14: end for

```

7.5. 结果分析

本文选取所有小区中通过聚类得到的 100 个点作为集散中心，由 7.2 节求解的 9 个物资来源地进行配送服务。算法由 python 编译在 pycharm 平台实现，使用的电脑配置为 Windows 10 64 Bit Intel(R)Core(TM) i7-11510U CPU @ 1.8 GHz with Memory by 16GB。使用混合元启发式算法对考虑病毒传播和车量容量限制的路径优化模型进行求解，将最优结果展示如下：

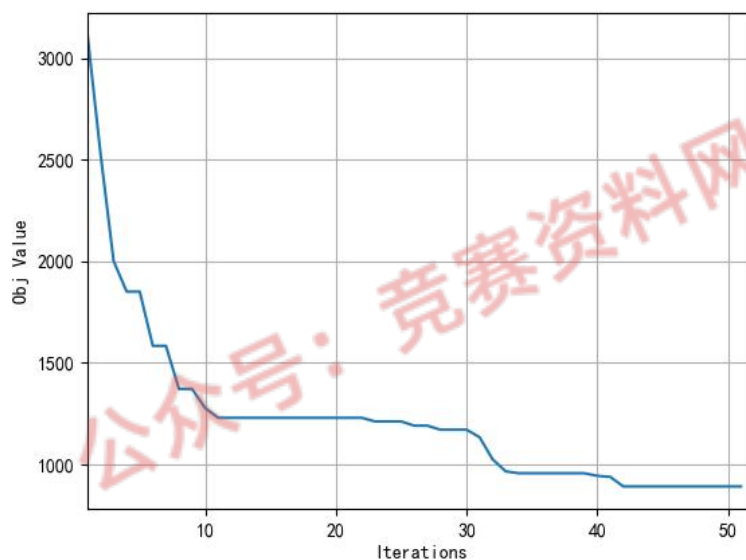


图 7-3. 考虑病毒传播和车量容量限制的路径优化模型目标函数迭代图

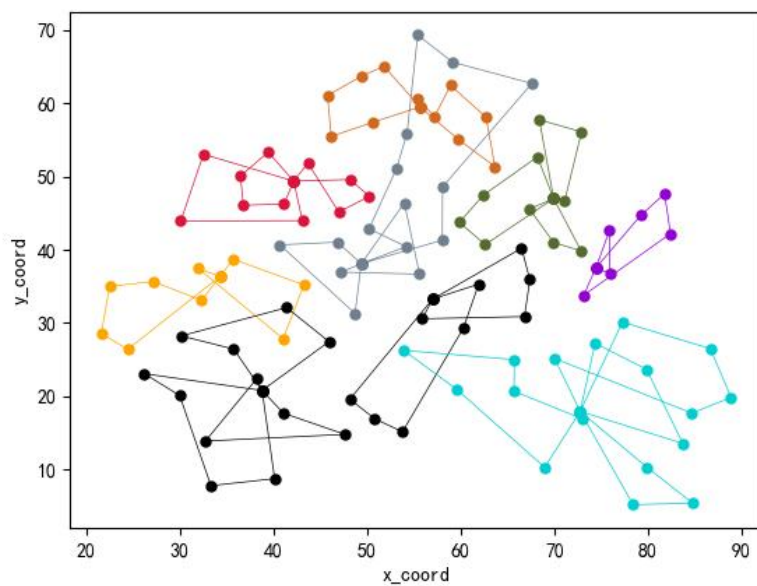


图 7-4. 考虑病毒传播和车量容量限制的有序网络路径优化结果

表 7-5. 考虑病毒传播和车量容量限制的有序网络路径优化结果

目标函数	感染风险	运输距离 (km)	运行时间 (s)	车辆数	
				H_1	H_2
891.05280610	362.5	552.87431	3780	8	24

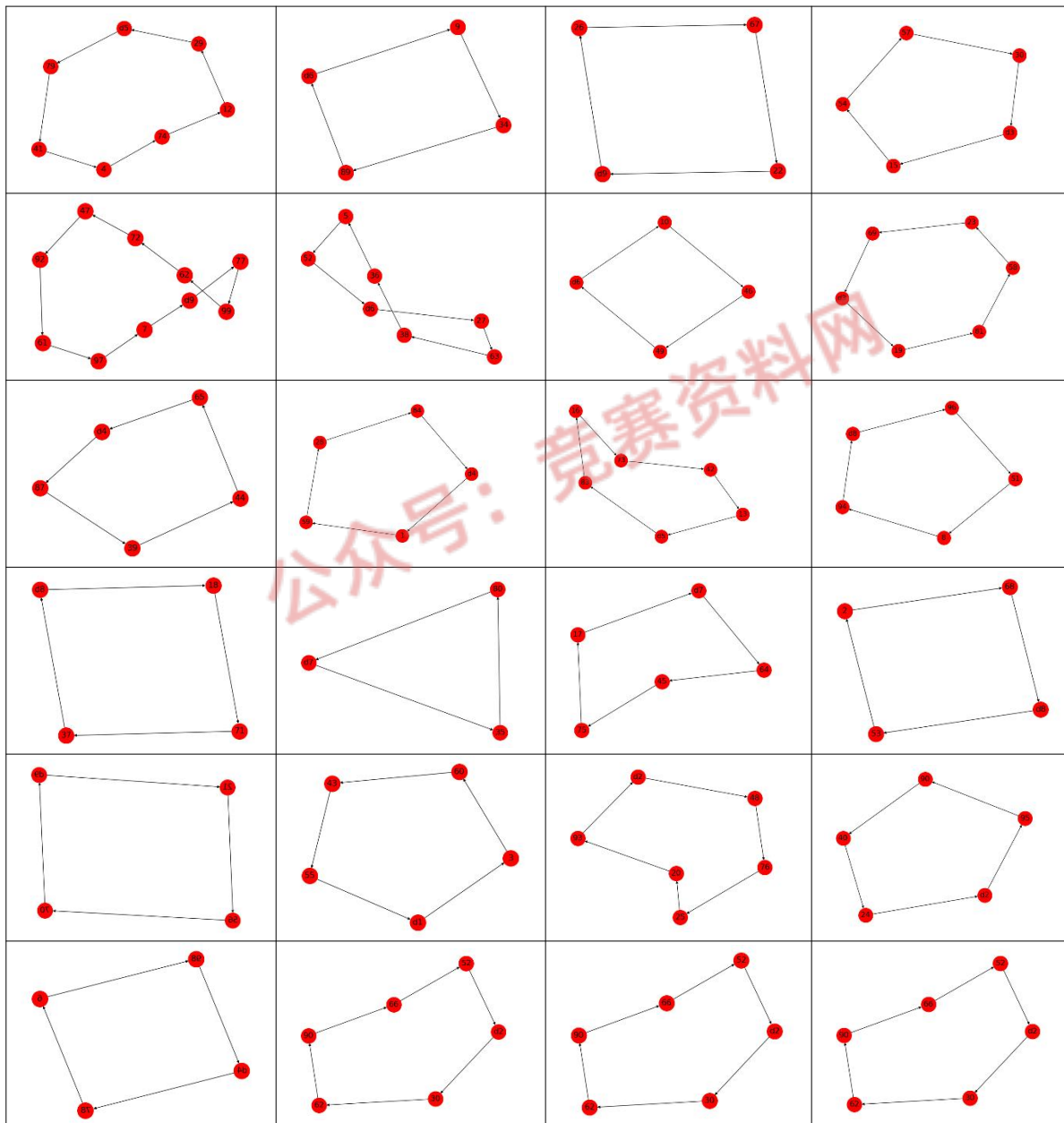
表 7-6. 考虑病毒传播和车量容量限制的路径优化结果

v1	d7-64-38-5-54-2-d7
v2	d1-3-20-76-48-90-d1
v3	d6-27-31-0-66-46-d6
v4	d7-10-86-68-d7
v5	d8-37-74-29-51-53-d8
v6	d4-1-98-44-16-d4
v7	d8-96-35-d8
v8	d1-50-36-19-81-43-55-d1
v9	d2-25-93-88-d2
v10	d5-47-72-62-18-41-d5
v11	d5-4-13-42-73-79-d5
v12	d2-22-70-6-d2
v13	d8-21-61-99-94-30-d8
v14	d8-17-45-80-77-12-d8
v15	d6-52-9-34-89-d6
v16	d2-95-67-24-d2
v17	d9-40-11-56-97-d9
v18	d4-59-78-28-84-65-d4
v19	d8-75-71-92-83-8-d8

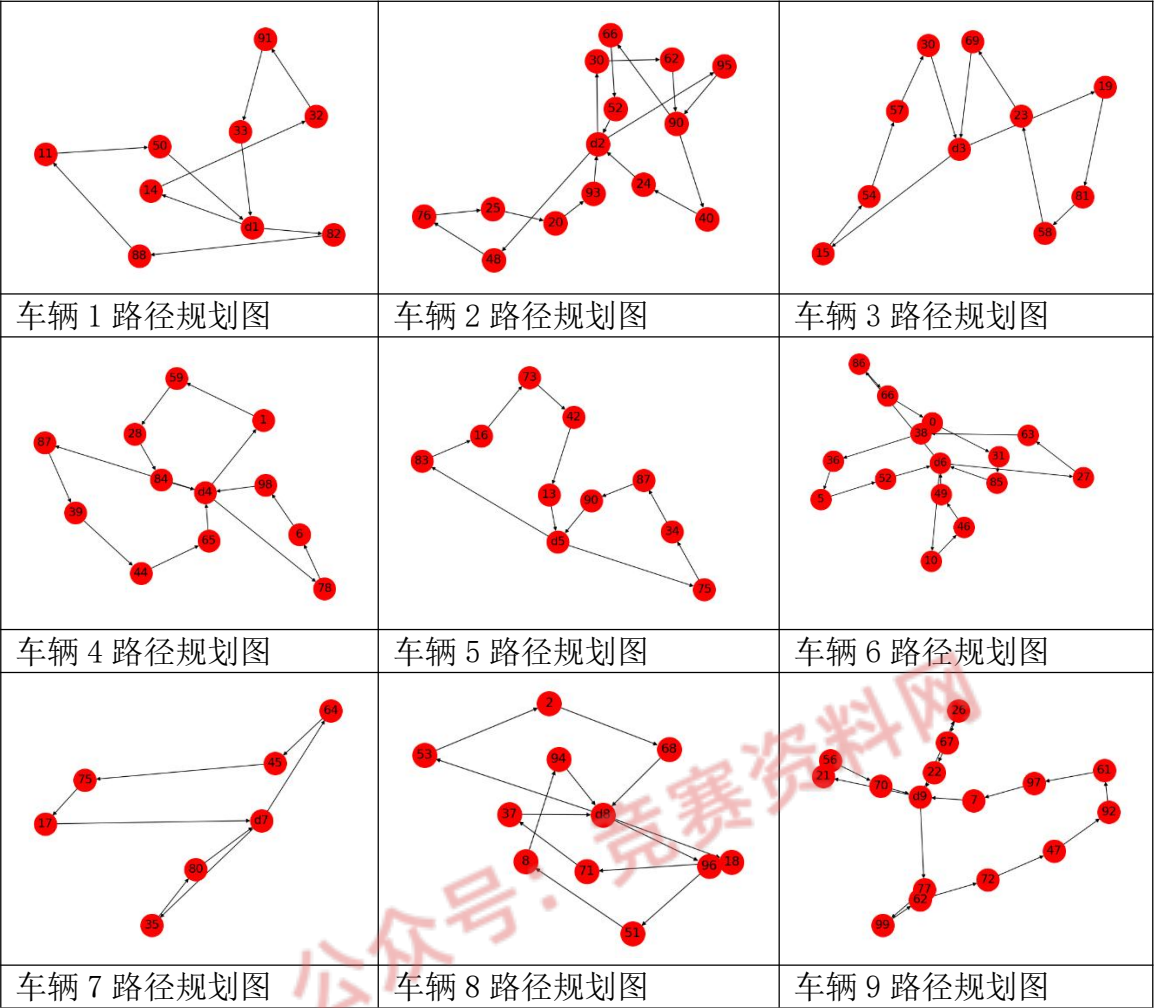
v20
v21
v22
v23
v24
v25

d1-82-33-91-60-d1
d6-49-85-63-d6
d3-69-57-23-15-d3
d9-7-39-87-26-d9
d1-32-14-58-d1
D8-69-57-23-13-d2

7.6. 有序网络结果可视化



车辆路径规划如下图所示：



八、模型的改进与推广

8.1. 模型的优点

本文构建模型提出了一种有效的优化方法，并考虑了配送过程中可能发生的新冠病毒传播，构建了一个给予复杂网络的病毒传播模型，模型加入多种应急物资、转运中心容量限制、转运中心最大服务半径等诸多符合现实情况的条件和约束。在构建模型的过程中充分分析供给需求以及可能产生的相关成本，对于选择的指标通过综合评价给予相应权重，对于实际问题的研究及解决具有十分重要的现实意义。

8.2. 模型的缺点

在本文模型构建过程中，由于无法获取全部真实数据以及当前数据与实际数据可能存在一定误差，建模过程中对相关步骤研究不够严谨，假设过于理想化等原因，本文的研究内容及算法都有待进一步完善。

8.3. 模型的改进与推广

由于时间有限，本篇文章的模型的构建还有很多不足之处，但得到的结构还是比较合理的。未来研究可以在此基础上加入对突发重大疫情应急物资需求预测与配送路径规划的研究，当前长春新区创新打造“配送单位—小区—物业公司”的物转模式，建立以小区书记为中枢，小区网格长、小区指挥长、物业、志愿者、消杀队伍共同参与的配送队伍体系，对管控区居民能够做到货品配送到单元门口，电话分批通知下楼取货；对封控区居民能够做到消杀队员着二级防护装备，将相关货品配送到居民家门口，打通服务居民“最后一米”。只有打通应急物资保障各环节壁垒，突发重大疫情下需求点的应急物资才能够及时、合理地得到保障，发挥应急物资的最大价值，切实降低群众生命及财产威胁。

公众号：竞赛资料网

参考文献

- [1]澎湃新闻. 【防疫科普】静默管理的含义 [EB/OL]. (2022-08-28).
https://www.thepaper.cn/newsDetail_forward_19648281
- [2]卫生健康委网站. 关于印发新型冠状病毒肺炎防控方案(第九版)的通知 [EB/OL].
(2022-06-28). http://www.gov.cn/xinwen/2022-06/28/content_5698168.htm
- [3]侯冷晨, 沈兵, 张成刚, 郭小毛, 姜艳, 盛伟琪, 钱明平, 胡龙军, 方秉华. 突发新型冠状病毒肺炎疫情后医院封闭式管理的探索与思考 [J]. 华西医学, 2022, 37(08): 1145-9.
- [4]陈镜羽, 张立. 疫情背景下应急生活保障物资末端物流配送模式研究 [J]. 物流科技, 2020, 43(10): 47-50.
- [5]姜凯凯, 高滢尘, 孙洁. 依托便利店构建生活物资应急配送终端体系——以日本便利店的灾后救援经验为例 [J]. 国际城市规划, 2021, 36(05): 121-8.
- [6]朱爱萍, 周应恒. 我国蔬菜市场需求分析 [J]. 华中农业大学学报(社会科学版), 2001, (03): 26-31.
- [7]王宇歌. Omicron BA.2 几乎完全不同于新冠病毒原始毒株 [EB/OL]. (2022-04-12).
<https://new.qq.com/rain/a/20220402A08KLT00>
- [8]疫情新闻. 长春新冠肺炎疫情图 [EB/OL]. (2022).
http://m.sy72.com/covid19/covid20_202248.html
- [9]吉林省人民政府新闻办公室. 长春就买菜难问题致歉 东北亚粮油、海吉星两大市场暂时关闭 [EB/OL]. (2022).
https://view.inews.qq.com/k/20220329A0BEGC00?web_channel=wap&openApp=false
- [10]谢聪慧, 吴世新, 张晨, 孙文涛, 何海芳, 裴韬, 罗格平. 基于谱系聚类的全球各国新冠疫情时间序列特征分析 [J]. 地球信息科学学报, 2021, 23(02): 236-45.

附录 Python 程序

问题一算法程序

程序编号	1	程序说明	新冠肺炎感染人数预测	编程软件	Python
<pre> import pandas as pd import numpy as np import matplotlib.pyplot as plt import seaborn as sns from scipy import stats import warnings import palettable sns.set_style("darkgrid") sns.set_context("paper") plt.rcParams["font.sans-serif"] = "SimHei" plt.rcParams["axes.unicode_minus"] = False np.set_printoptions(suppress=True, precision=10, threshold=2000, linewidth=150) pd.set_option('display.float_format', lambda x : '%.2f' % x) pd.set_option('display.max_columns',10) pd.set_option('display.max_rows',20) warnings.filterwarnings("ignore") %matplotlib inline covid_sum = pd.read_excel("附件 1: 长春市 COVID-19 疫情期间病毒感染人数数据.xlsx",sheet_name="总计") covid_sum # SEIR2 模型，考虑潜伏期具有传染性 from scipy.integrate import odeint # 导入 scipy.integrate 模块 def dySEIR2(y, t, lamda, lam2, delta, mu): # SEIR 模型，导数函数 s, e, i = y ds_dt = - lamda*s*i - lam2*s*e de_dt = lamda*s*i + lam2*s*e - delta*e di_dt = delta*e - mu*i return np.array([ds_dt,de_dt,di_dt]) # 设置模型参数 number = 8534000 # 总人数 lamda = 1.0 # 日接触率，患病者每天有效接触的易感者的平均人数 lam2 = 0.25 # 日接触率 2，潜伏者每天有效接触的易感者的平均人数 delta = 0.05 # 日发病率，每天发病成为患病者的潜伏者占潜伏者总数的比例 mu = 0.05 # 日治愈率，每天治愈的患病者人数占患病者总数的比例 sigma = lamda / mu # 传染期接触数 fsig = 1-1/sigma tEnd = 57 # 预测日期长度 t = np.arange(0.0, tEnd, 1) # (start,stop,step) i0 = 633/number # 患病者比例的初值 </pre>					

```

e0 = 495/number # 潜伏者比例的初值
s0 = 1-i0 # 易感者比例的初值
Y0 = (s0, e0, i0) # 微分方程组的初值
# odeint 数值解, 求解微分方程初值问题
ySEIR2 = odeint(dySEIR2, Y0, t, args=(lamda,lam2,delta,mu)) # SEIR 模型
# print(ySEIR2)
# 输出绘图
# print("lamda={} \tmu={} \tsigma={} \t(1-1/sig)={}".format(lamda,mu,sigma,fsig))
plt.title("SEIR model")
plt.xlabel('天数')
plt.axis([0, tEnd, -0.1, 1.1])
plt.plot(t, ySEIR2[:,0], '-g', label='易感者比例') # 易感者比例
plt.plot(t, ySEIR2[:,1], '-b', label='潜伏者比例') # 潜伏者比例
plt.plot(t, ySEIR2[:,2], '-m', label='患病者比例') # 患病者比例
plt.legend(loc='upper right')
plt.savefig("SEIR model.svg", format="svg")
plt.show()

```

问题二算法程序

程序编号	5	程序说明	对集散地选址 K-means	编程软件	Python
<pre> data = pd.read_excel('D:/anaconda_package/jupyternotebook/华为杯/2022 华为杯/data/问题二/ 小区数据.xlsx') data X = data[["小区横坐标","小区纵坐标"]] X km = KMeans(n_clusters=25).fit(X) data['cluster'] = km.labels_ data.sort_values('cluster') data # data.to_excel("聚类.xlsx") #类中心 cluster_centers = km.cluster_centers_ cluster_centers cluster_coordinate=pd.DataFrame(cluster_centers) cluster_coordinate cluster_coordinate.iloc[0,0] x_coordinat=[] y_coordinat=[] for i in range(len(data)): for j in range(25): if data['cluster'].values[i] == j: x_coordinat.append(cluster_coordinate.iloc[j,0]) y_coordinat.append(cluster_coordinate.iloc[j,1]) data['聚类中心横坐标']=x_coordinat </pre>					

```

data['聚类中心纵坐标']=y_coordinat
data
def distance(x,y):
    d=np.sqrt((x[0]-y[0])**2+(x[1]-y[1])**2)
    return d
x1 = data[['小区横坐标','小区纵坐标']]
x2 = data[['聚类中心横坐标','聚类中心纵坐标']]
x1_=np.array(x1)
x2_=np.array(x2)
distance(x1_[1],x2_[1])
#计算各点到中心的距离
dis = []
for i in range(len(data)):
    dis.append(distance(x1_[i],x2_[i]))
data['距离'] = dis
village_num=data.groupby('cluster')['小区编号'].count()
peo_num = data.groupby('cluster')['小区人口数（人）'].sum()
dis_sum = data.groupby('cluster')['距离'].sum()
dis_mean = data.groupby('cluster')['距离'].mean()
area_x = data.groupby('所属区域')['小区横坐标'].mean()
area_y = data.groupby('所属区域')['小区纵坐标'].mean()
area_x
area_y
area=[]
for i in range(9):
    area.append((area_x[i],area_y[i]))
total=[]
for i in range(len(cluster_centers)):
    tmp = []
    for j in range(len(area)):
        d = distance(cluster_centers[i],area[j])
        tmp.append(d)
    total.append(tmp)
belong_area=[]
for i in range(len(total)):
    ba=total[i].index(min(total[i]))
    belong_area.append(ba)
cluster_coordinate.columns=['x 坐标','y 坐标']
cluster_coordinate['管辖范围小区个数']=village_num.tolist()
cluster_coordinate['管辖范围内人口数']=peo_num.tolist()
cluster_coordinate['总服务距离']=dis_sum.tolist()
cluster_coordinate['平均距离']=dis_mean.tolist()
cluster_coordinate['所属区域']=belong_area
cluster_coordinate

```

```

area_name=area_x.index.tolist()
d_area = {}
for i in range(9):
    d_area[i]= area_name[i]
d_area
area_belong=[]
for i in range(len(cluster_coordinate)):
    for j in range(len(area_name)):
        if cluster_coordinate['所属区域'].iloc[i] == j:
            area_belong.append(area_name[j])
cluster_coordinate['所属行政区']=area_belong
cluster_coordinate
plt.scatter(data["小区横坐标"], data["小区纵坐标"],c=colors[data["cluster"]])
plt.scatter(centers.小区横坐标, centers.小区纵坐标, linewidths=3, marker='*', s=30, c='r')
ax = plt.axes()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.xlabel("横坐标")
plt.ylabel("纵坐标")
plt.savefig('中心.jpg')
plt.scatter(data["小区横坐标"], data["小区纵坐标"],c=colors[data["cluster"]],s=10,alpha=1)
ax = plt.axes()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.savefig('小区坐标.jpg')
scatter_matrix(data[["小区横坐标","小区纵坐标"]],s=100, alpha=1,
c=colors[data["cluster"]], figsize=(10,10))
plt.suptitle("With 2centroids initialized")

```

程序编号	6	程序说明	轮廓系数和肘部法则确定聚类的个数	编程软件	Python
<pre> #轮廓系数 score = metrics.silhouette_score(X,data.cluster) print(score) scores = [] for k in range(2,30): labels = KMeans(n_clusters=k).fit(X).labels_ score = metrics.silhouette_score(X, labels) scores.append(score) scores plt.plot(list(range(2,30)), scores) plt.xlabel("Number of Clusters Initialized") plt.ylabel("Sihouette Score") </pre>					

程序编号	7	程序说明	P-中值选址优化	编程软件	Python
<pre> class pMedianCapacitated: def __init__(self): self.dir = "pmedcap1.txt" self.Problem = [] def read(self): dir = self.dir f = open(dir, "r") lines = f.readlines() f.close() problemNum = lines.pop(0) X, Y, D = [], [], [] pi = None id = 0 for s in lines: si = s.split() if len(si) == 2: if pi is not None: self.addProblem(id, bi, X, Y, D, P, Q) pi, id, bi = {}, int(si[0]), int(si[1]) X, Y, D = [], [], [] continue if len(si) == 3: P = int(si[1]) Q = int(si[2]) continue X.append(int(si[1])) Y.append(int(si[2])) D.append(int(si[3])) self.addProblem(id, bi, X, Y, D, P, Q) def addProblem(self, id, bi, X, Y, D, P, Q): C = zip(X, Y) C = [[a, b] for (a, b) in C] C = pdist(C, metric='euclidean') C = (squareform(C)).astype(np.int) pi = {"id": id, "bestSolution": bi, "D": D, "P": P, "Q": Q, "X": X, "Y": Y, "C": C} self.Problem.append(pi) def getProblem(self, item=1): if len(self.Problem) == 0: </pre>					

```

        self.read()
        pi = self.Problem[item-1]
        r = (pi["id"], pi["bestSolution"], pi["D"], pi["P"],
            pi["Q"], pi["X"], pi["Y"], pi["C"])
        return r
def solve_by_scip(D, PN, Q, C):
    num = len(D)
    N = range(num)
    model = Model("p-median-capacitated")
    x, y = {}, {}
    for i in N:
        y[i] = model.addVar(vtype="B", name="y(%s)"%i)
        for j in N:
            x[i, j] = model.addVar(vtype="B", name="x(%s, %s)"%(i, j))
    model.setObjective(quicksum(C[i, j]*x[i, j] for i in N for j in N))
    for j in N:
        model.addCons(quicksum(x[i, j] for i in N) == 1)
    for i in N:
        model.addCons(quicksum(D[j]*x[i, j] for j in N) <= Q*y[i])
    model.addCons(quicksum(y[i] for i in N) == PN)
    model.redirectOutput()
    model.optimize()
    A = [0] * num
    for i in N:
        if model.getVal(y[i]) > 0.5:
            for j in N:
                if model.getVal(x[i, j]) >= 0.5:
                    A[j] = i
    return model.getObjVal(), A
def draw(X, Y, A):
    N = list(range(len(X)))
    H = list(set(A))
    for i in set(N) - set(H):
        plt.plot([X[i], X[A[i]]], [Y[i], Y[A[i]]], color='k', linewidth=0.5)
        plt.plot(X[i], Y[i], 'ko')
        plt.text(X[i], Y[i], i, ha='center', va='bottom',
            color='b', fontsize=9)
    for i in H:
        plt.plot(X[i], Y[i], 'ro', markersize=10)
        plt.text(X[i], Y[i], i, ha='center', va='bottom',
            color='g', fontsize=10)
    plt.show()

def Report(z, A):

```

```

A = np.array(A)
P = set(A)
print(""" * 50)
print("目标函数值 = %g" % z)
for i in P:
    print("仓库 {}".format(i), end="")
    pi = np.where(A == i)
    print(list(pi[0]))
print(""" * 50)
def test():
    pMC = pMedianCapacitated()
    problem = 1
    id, bsolution, D, PN, Q, X, Y, C = pMC.getProblem(problem)
    z, A = solve_by_scip(D, PN, Q, C)
    Report(z, A)
    draw(X, Y, A)
test()

```

问题三算法程序

程序编号	8	程序说明	相关性分析与热力图	编程软件	Python
<pre> data = pd.read_excel(r"C:\Users\94199\Desktop\问题三-各区物资需求量与新增病例的相关性分析(4.4-5.1).xlsx") #跳过第一行，一二三行组成多叠列索引，第1列作为行索引 skiprows=3,header=None,index_col=0 cor = data.corr(method='pearson') print(cor) # 输出相关系数 rc = {'font.sans-serif': 'SimHei', 'axes.unicode_minus': False} sns.set(font_scale=0.7,rc=rc) # 设置字体大小 plt.rcParams['font.sans-serif'] = ['STSong'] # 指定默认字体 plt.rcParams['axes.unicode_minus'] = False # 解决保存图像是负号 '-' 显示为方块的问题 plt.rcParams['font.size'] = 10.5 #全局有效 sns.set_palette(sns.color_palette("Greys")) plt.figure(figsize=(10,10)) #图的宽和长，单位为 inch sns.heatmap(cor, annot=False, # 显示相关系数的数据 center=0.5, # 居中 linewidth=0.5, # 设置每个单元格的距离 vmin=0, vmax=1, # 设置数值最小值和最大值 xticklabels=True, yticklabels=True, # 显示 x 轴和 y 轴 square=True, # 每个方格都是正方形 cbar=True, # 绘制颜色条 cmap='Greens', # 设置热力图颜色) </pre>					

```
plt.savefig("热力图.png",dpi=1000)#保存图片，分辨率为 600
```

程序编号	9	程序说明	TOPSIS 综合评价	编程软件	Python
<pre> class TOPSIS: def __init__(self): self.testfile="/users/suixin/desktop/测试样本.xlsx" #这里四种数据样本 self.df=pd.read_excel(self.testfile) def minimum_data_type(self,x,max_value): return max_value-x def middle_data_type(self,x,x_best,X): M=np.abs(X-x_best).max() return 1-(x-x_best)/M def region_data_type(self,x,a,b,X): max_value=X.max() min_value=X.min() M=np.max([a-min_value,max_value-b]) if x<a: return 1-(a-x)/M elif x>=a and x<=b: return 1 elif x>b: return 1-(x-b)/M def norm_standardization(self,x,X): return x/np.sqrt(np.square(X).sum()) def compute_distance(self,x1,x2): return np.sqrt(np.square(np.array(x1)-np.array(x2)).sum()) def main(self): middle_value=7 region_value=[36,37] #将极小型数据正向化 self.df['minimum_data1']=self.df['minimum_data'].apply(lambda x:self.minimum_data_type(x,self.df['minimum_data'].max())) #将中间型数据正向化 self.df['middle_data1']=self.df['middle_data'].apply(lambda x:self.middle_data_type(x,middle_value,self.df['middle_data'])) #将区间型数据正向化 </pre>					

```

        self.df['region_data1'] = self.df['region_data'].apply(lambda x: self.region_data_type(x,
region_value[0],region_value[1], self.df['region_data']))
        # 对正化向的数据标准化
        col_list=['maximum_data','minimum_data1','middle_data1','region_data1']
        for col in col_list:
            self.df[col.split("_")[0]+"_norm"]=self.df[col].apply(lambda
x:self.norm_standardization(x,self.df[col]))
            #找出最优及最劣数据点
            norm_cols=['maximum_norm','minimum_norm','middle_norm','region_norm']
            best_value=[self.df[i].max() for i in norm_cols]
            worst_value=[self.df[i].min() for i in norm_cols]
            self.df['D_best']=self.df[norm_cols].apply(lambda
x:self.compute_distance(x,best_value),axis=1)
            self.df['D_worst'] = self.df[norm_cols].apply(lambda x: self.compute_distance(x,
worst_value), axis=1)
            self.df['C']=self.df['D_worst']/(self.df['D_worst']+self.df['D_best'])
            #打分排序结果如下所示

print(self.df[['id','maximum_data','minimum_data','middle_data','region_data','C']].sort_values('C
',ascending=False))

if __name__=="__main__":
    TOPSIS().main()

```

问题四算法程序

程序编号	10	程序说明	射线法判断点是否在多边形内	编程软件	Java
<pre> function isPointInPolygon(pt, pts) { // 交点个数 var counter = 0; // 水平射线和多边形边的交点 x 坐标 var xinters; // 线段起点和终点 var p1, p2; // for 循环 for (let i = 0; i < pts.length; i++) { p1=pts[i] p2 = pts[(i+1) % pts.length];// 最后一个点等于起点 pts[0] if (pt[1] > Math.min(p1[1], p2[1]) &&pt[1] <= Math.max(p1[1], p2[1])) { xinters = (pt[1] - p1[1]) * (p2[0] - p1[0]) / (p2[1] - p1[1]) + p1[0]; if (p1[1] == p2[1] pt[0] <= xinters) { </pre>					

```

        counter++;
    }
}
}
if (counter % 2 == 0) {
    return false;
} else {
    return true;
}
}

```

程序编号	11	程序说明	大规模邻域算法与遗传算法	编程软件	Python
<pre> class randNeighborhoodOrding: def __init__(self): self.mDir = 'D:\\研究生\\比赛\\数学建模\\解题\\notebook\\VRP\\demand3.csv' self.mData = 'D:\\研究生\\比赛\\数学建模\\解题\\notebook\\VRP\\depot.csv' def readCsvFile(demand_file, depot_file, model): with open(demand_file, 'r') as f: demand_reader = csv.DictReader(f) for row in demand_reader: node = Node() node.id = int(row['id']) node.x_coord = float(row['x_coord']) node.y_coord = float(row['y_coord']) node.demand = float(row['demand']) model.demand_dict[node.id] = node model.demand_id_list.append(node.id) with open(depot_file, 'r') as f: depot_reader = csv.DictReader(f) for row in depot_reader: node = Node() node.id = row['id'] node.x_coord = float(row['x_coord']) node.y_coord = float(row['y_coord']) node.depot_capacity = float(row['capacity']) model.depot_dict[node.id] = node with open(self.mDir + filename, 'r') as f: data_list = [] for i in range(7): data_list.append(f.readline().strip().split(':')) temp = data_list[1][2].strip().split(",") K = int(temp[0]) optimalSolution = data_list[1][3].split("") optSol = int(optimalSolution[0]) N = int(data_list[3][1]) </pre>					

```

Q = int(data_list[5][1])
node_x = []
node_y = []
D = []
C = np.zeros([N, N])
for i in range(N):
    n, x, y = f.readline().strip().split(" ")
    node_x.append(float(x))
    node_y.append(float(y))
f.readline()
for i in range(N):
    n, d = f.readline().strip().split(" ")
    D.append(int(d))
for i in range(N):
    for j in range(N):
        C[i][j] = np.sqrt((node_x[i]-node_x[j])**2 +
                           (node_y[i]-node_y[j])**2)

C = np.round(C).astype("int32")
VS = list(range(N))
NS = VS.copy()
NS.pop(0)
KS = list(range(K))
self.mData['Cost'] = C
self.mData['Vehicle'] = KS
self.mData['Capacity'] = D
return N, K, D, C, Q, node_x, node_y, NS, VS, KS, optSol

def solveBySCIP(self, N, K, D, C, Q, NS, VS):
    m = Model('cvrp')

    # Decision variables
    x, u = {}, {}
    for i in VS:
        for j in VS:
            x[i, j] = m.addVar(vtype='B', name='x(%s,%s)'
                               % (i, j))

    for i in NS:
        u[i] = m.addVar(vtype='C', name='u(%s)' % i)

    # define Constraints
    for i in NS:
        m.addCons(quicksum(x[i, j] for j in VS if i != j) == 1)

    for j in NS:
        m.addCons(quicksum(x[i, j] for i in VS if i != j) == quicksum(x[j, i] for i in
VS if i != j))

    m.addCons(quicksum(x[i, 0] for i in VS) == K)
    m.addCons(quicksum(x[0, j] for j in VS) == K)
    for i in NS:

```

```

        for j in NS:
            if i != j:
                m.addCons(u[j] >= u[i] + (x[i, j] - 1) * Q + D[j])
    for i in NS:
        m.addCons(D[i] <= u[i])
        m.addCons(u[i] <= Q)

# define Objective function
m.setObjective(quicksum(C[i][j] * x[i, j] for (i, j) in x), "minimize")

# solving...
m.hideOutput()          # silent mode
m.redirectOutput()      # showing the solution process
m.optimize()            # starting solving
solution = m.getObjVal()

if solution:
    active_node = [(i, j) for (i, j) in x if m.getVal(x[i, j]) > 0.5]
    # print(active_node)
    # print("obj:%s" % solution)
    tour = {}
    list_i = []
    list_j = []
    for k in range(len(active_node)):
        list_i.append(active_node[k][0])
        list_j.append(active_node[k][1])

    print("two index Obj: \t", solution)
    k_ = 0
    while (k_ <= K - 1):
        Visited = []
        list_k = [0]
        for i in range(len(list_i)):
            T_index1 = list_i.index(list_k[-1])
            list_k.append(list_j[T_index1])
            Visited.append(T_index1)
            list_i.pop(T_index1)
            list_j.pop(T_index1)
            if list_k[-1] == 0:
                print("K=", k_, ":", list_k)
                break

        tour[k_] = list_k
        k_ = k_ + 1
else:
    print("\nsomething went wrong!")
    print("Model error!")
print(tour)
return tour

```

```

def greedyAligorithm(self, D, C, Q, NS, VS, KS):
    route = {i: [] for i in KS}
    A = []
    B = set(VS)-set(A)

    for k in KS:
        route[k].append(VS[0])
        A.append(VS[0])
        Capacity_k = 0
        while Capacity_k <= Q:
            B = set(NS) - set(A)
            DisSet = []
            for b in B:
                DisSet.append(C[route[k][-1]][b])

            if len(DisSet) != 0:
                idx = DisSet.index(min(DisSet))
            else:
                break

            B_L = list(B)
            Capacity_k += D[B_L[idx]]
            if Capacity_k <= Q:
                route[k].append(B_L[idx])
                A.append(B_L[idx])
                print("%s: (%s) - %s" % (k, Capacity_k - D[B_L[idx]], route[k]))
            else:
                break

    if len(B) != 0:
        print('the remaining nodes due to the capacity constraint')
        print('B:', B)

    B = list(B)
    while len(B) != 0:
        CAPA = []
        for p in range(len(route)):
            CAPA_i = 0
            for q in route[p]:
                CAPA_i += D[q]
            CAPA.append(CAPA_i)
        print(CAPA)
        CAPA_min_idx = CAPA.index(min(CAPA))
        route[CAPA_min_idx].append(B[-1])
        B.pop()

    #append the depot 0, indicating each vehicle must return depot
    for k in KS:
        route[k].append(VS[0])

```

```

return route

def interRouteShiftOne(self,route,K):
    # print("Operator: interRoute_Shift_One...")
    route_shift_One = copy.deepcopy(route)
    randi = random.sample(range(0, K), 2)
    randi_a = random.randint(1, len(route_shift_One[randi[0]]) - 2)
    node_i = route_shift_One[randi[0]].pop(randi_a)
    randi_b = random.randint(1, len(route_shift_One[randi[1]]) - 2)
    route_shift_One[randi[1]].insert(randi_b, node_i)
    return route_shift_One

def interRouteShiftTwo(self,route,K):
    # print("Operator: interRoute_Shift_Two...")
    route_shift_Two = copy.deepcopy(route)
    randi = random.sample(range(0, K), 2)
    randi_a = random.randint(1, len(route_shift_Two[randi[0]]) - 3)
    node_i = route_shift_Two[randi[0]].pop(randi_a)
    node_j = route_shift_Two[randi[0]].pop(randi_a)
    randi_b = random.randint(1, len(route_shift_Two[randi[1]]) - 2)
    route_shift_Two[randi[1]].insert(randi_b, node_i)
    route_shift_Two[randi[1]].insert(randi_b + 1, node_j)
    return route_shift_Two

def interRouteSwapOne(self,route,K):
    # print("Operator: interRoute_Swap_One...")
    route_swap_one = copy.deepcopy(route)
    randi = random.sample(range(0, K), 2)
    randi_a = random.randint(1, len(route_swap_one[randi[0]]) - 2)
    randi_b = random.randint(1, len(route_swap_one[randi[1]]) - 2)
    node_i = route_swap_one[randi[0]].pop(randi_a)
    node_j = route_swap_one[randi[1]].pop(randi_b)
    route_swap_one[randi[1]].insert(randi_b, node_i)
    route_swap_one[randi[0]].insert(randi_a, node_j)

    return route_swap_one

def interRouteSwapTwo(self,route,K):
    # print("Operator: interRoute_Swap_Two...")
    route_swap_two = copy.deepcopy(route)
    randi = random.sample(range(0, K), 2)
    randi_a = random.randint(1, len(route_swap_two[randi[0]]) - 3)
    randi_b = random.randint(1, len(route_swap_two[randi[1]]) - 3)
    node_i = route_swap_two[randi[0]].pop(randi_a)
    node_ii = route_swap_two[randi[0]].pop(randi_a)
    node_j = route_swap_two[randi[1]].pop(randi_b)
    node_jj = route_swap_two[randi[1]].pop(randi_b)
    route_swap_two[randi[1]].insert(randi_b, node_i)
    route_swap_two[randi[1]].insert(randi_b + 1, node_ii)

```

```

route_swap_two[randi[0]].insert(randi_a, node_j)
route_swap_two[randi[0]].insert(randi_a + 1, node_jj)

return route_swap_two

def interRouteSwapThree(self,route,K):
    # print("Operator: interRoute_Swap_Three...")
    route_swap_three = copy.deepcopy(route)
    randi = random.sample(range(0, K), 2)
    randi_a = random.randint(1, len(route_swap_three[randi[0]]) - 3)
    randi_b = random.randint(1, len(route_swap_three[randi[1]]) - 2)
    node_i = route_swap_three[randi[0]].pop(randi_a)
    node_ii = route_swap_three[randi[0]].pop(randi_a)
    node_j = route_swap_three[randi[1]].pop(randi_b)
    route_swap_three[randi[1]].insert(randi_b, node_i)
    route_swap_three[randi[1]].insert(randi_b + 1, node_ii)
    route_swap_three[randi[0]].insert(randi_a, node_j)

    return route_swap_three

def interRouteCross(self,route,K):
    # print("Operator: interRoute_Cross...")
    route_cross = copy.deepcopy(route)
    randi = random.sample(range(0, K), 2)
    randi_a = random.randint(1, len(route_cross[randi[0]]) - 3)
    randi_b = random.randint(1, len(route_cross[randi[1]]) - 3)
    node_epi_i = route_cross[randi[0]][:randi_a]
    node_epi_ii = route_cross[randi[0]][randi_a:]
    node_epi_j = route_cross[randi[1]][:randi_b]
    node_epi_jj = route_cross[randi[1]][randi_b:]
    route_cross[randi[0]] = node_epi_j + node_epi_ii
    route_cross[randi[1]] = node_epi_i + node_epi_jj

    return route_cross

def intraRouteOrOpt(self,route,K,num):
    # print("Operator: intraRoute_Or_Opt...")
    route_or_opt = copy.deepcopy(route)
    randi = random.randint(0, K - 1)
    rand_a = random.randint(1, len(route_or_opt[randi]) - num - 1)
    pop_list = []
    for p in range(num):
        node_i = route_or_opt[randi].pop(rand_a)
        pop_list.append(node_i)
    rand_b = random.randint(1, len(route_or_opt[randi]) - 2)
    node_a = route_or_opt[randi][:rand_b]
    node_b = route_or_opt[randi][rand_b:]
    route_or_opt[randi] = node_a + pop_list + node_b

    return route_or_opt

```

```

def intraRoute2Opt(self,route, K):
    # print("Operator: intraRoute_2_Opt...")
    route_2_opt = copy.deepcopy(route)
    randi = random.randint(0, K - 1)
    rand_a = random.randint(1, int(np.floor(0.5 * len(route_2_opt[randi]))))
    rand_b = random.randint(int(np.ceil(0.5 * len(route_2_opt[randi])) + 1,
len(route_2_opt[randi]) - 2)
    node_a = route_2_opt[randi][:rand_a]
    node_b = route_2_opt[randi][rand_a:rand_b]
    node_b = node_b[::-1]
    node_c = route_2_opt[randi][rand_b:]
    route_2_opt[randi] = node_a + node_b + node_c

    return route_2_opt

def intraRouteExchange(self,route,K):
    # print("Operator: intraRoute_Exchange...")
    route_exchange = copy.deepcopy(route)
    randi = random.randint(0, K - 1)
    rand_ab = random.sample(range(1, len(route_exchange[randi])-1), 2)

    if rand_ab[0] < rand_ab[1]:
        node_j = route_exchange[randi].pop(rand_ab[1])
        node_i = route_exchange[randi].pop(rand_ab[0])
        route_exchange[randi].insert(rand_ab[0], node_j)
        route_exchange[randi].insert(rand_ab[1], node_i)
    else:
        node_j = route_exchange[randi].pop(rand_ab[0])
        node_i = route_exchange[randi].pop(rand_ab[1])
        route_exchange[randi].insert(rand_ab[1], node_j)
        route_exchange[randi].insert(rand_ab[0], node_i)

    route_exchange[randi].insert(rand_ab[0], node_j)
    route_exchange[randi].insert(rand_ab[1], node_i)

    return route_exchange

def intraRouteReverse(self,route,K):
    # print("Operator: intraRoute_Reverse...")
    route_reverse = copy.deepcopy(route)
    randi = random.randint(0, K - 1)
    route_reverse[randi].reverse()

    return route_reverse

def intraRouteAdExchange(self, route, K):
    # print("Operator: intraRoute_Ad_Exchange...")
    route_ad_exchange = copy.deepcopy(route)
    randi = random.randint(0, K - 1)

```

```

        rand_a = random.randint(1, len(route_ad_exchange[randi]) - 3)
        node_i = route_ad_exchange[randi].pop(rand_a)
        node_j = route_ad_exchange[randi].pop(rand_a)
        route_ad_exchange[randi].insert(rand_a, node_j)
        route_ad_exchange[randi].insert(rand_a + 1, node_i)

    return route_ad_exchange

def routeEjectionChain(self, route, K):
    # print("Perturbation Operator: double - bridge...")
    route_ejection_chain = copy.deepcopy(route)
    rand_chain, route_chain = [], []
    for r in range(K):
        rand_chain.append(random.randint(1, len(route_ejection_chain[r]) - 2))
        route_chain.append(route_ejection_chain[r].pop(rand_chain[-1]))
    for k in range(K - 1):
        route_ejection_chain[k + 1].insert(rand_chain[k + 1], route_chain[k])
    route_ejection_chain[0].insert(rand_chain[0], route_chain[-1])

    del rand_chain, route_chain

    return route_ejection_chain

def routeDoubleBridge(self, route, K):
    # print("Perturbation Operator: double - bridge...")
    route_bridge = copy.deepcopy(route)
    randi = random.randint(0, K - 1)
    if len(route_bridge[randi]) >= 6:
        rand_a = random.randint(1, int(np.floor(0.5 * len(route_bridge[randi]))) - 1)
        rand_b = random.randint(int(np.floor(0.5 * len(route_bridge[randi]])),
len(route_bridge[randi]) - 3)
        route_j = route_bridge[randi].pop(rand_b)
        route_jj = route_bridge[randi].pop(rand_b)
        route_i = route_bridge[randi].pop(rand_a)
        route_ii = route_bridge[randi].pop(rand_a)
        route_bridge[randi].insert(rand_a, route_j)
        route_bridge[randi].insert(rand_a + 1, route_jj)
        route_bridge[randi].insert(rand_b, route_i)
        route_bridge[randi].insert(rand_b + 1, route_ii)
    else:
        print('the length of selected route is not long enough')

    return route_bridge

def solveByNeighborSearch(self, N, K, D, C, Q, NS, VS, KS, optSol):
    tour = {}
    Route = self.greedyAligorithm(D, C, Q, NS, VS, KS)
    print("===== initial routes =====")
    print(Route)
    print("the Known Optimal Solution is: %s" % optSol)

```

```

        tour = Route.copy()
        InterRouteDir = {1: 'inter_shift_one', 2: 'inter_swap_one', 3: 'inter_cross',
                          4: 'inter_shift_two', 5: 'inter_swap_two', 6:
'inter_swap_three'}
        IntraRouteDir = {1: 'intra_2_opt', 2: 'intra_or_opt', 3: 'intra_exchange',
                          4: 'intra_reverse', 5: 'intra_ad_exchange'}
        perturRouteDir = {1: 'route_ejection_chain', 2: 'route_bridge', 3:
'inter_swap_one'}

        Operator_score = {'inter_shift_one': 0, 'inter_shift_two': 0, 'inter_swap_one': 0,
                           'inter_swap_two': 0, 'inter_swap_three': 0, 'inter_cross': 0,
                           'intra_or_opt': 0, 'intra_2_opt': 0, 'intra_exchange': 0,
                           'intra_reverse': 0, 'intra_ad_exchange': 0}
        perturbation_score = {'route_ejection_chain': 0, 'route_bridge': 0,
'inter_swap_one': 0}

        Cost = self.routeCost(Route)
        print("the current cost is %s" % Cost)
        sol = Cost
        route = copy.deepcopy(Route)

        iteration_limit = 50000
        iter_i = 0
        iter_no_improve = 0
        while iter_i != iteration_limit:
            route_temp = {}

            ##### Choose a neighborhood among the InterRouteDir
            #####-----###
            rand_inter = random.randint(1, len(InterRouteDir))
            interDir_idx = InterRouteDir[rand_inter]

            if interDir_idx == 'inter_shift_one':
                route_temp = self.interRouteShiftOne(route, K)
            elif interDir_idx == 'inter_shift_two':
                route_temp = self.interRouteShiftTwo(route, K)
            elif interDir_idx == 'inter_swap_one':
                route_temp = self.interRouteSwapOne(route, K)
            elif interDir_idx == 'inter_swap_two':
                route_temp = self.interRouteSwapTwo(route, K)
            elif interDir_idx == 'inter_swap_three':
                route_temp = self.interRouteSwapThree(route, K)
            elif interDir_idx == 'inter_cross':
                route_temp = self.interRouteCross(route, K)

            if self.CapacityCheck(route_temp, Q) == True:
                # print('capacity check passing...')
                Cost = self.routeCost(route_temp)
                # print("the operator cost is %s" % Cost)
                if Cost < sol:

```

"1"

variable parameter

increases "1"

```
sol = Cost
# If the solution is optimized by the operator, the fraction increases

Operator_score[InterRouteDir[rand_inter]] += 1
print('inter operator:', InterRouteDir[rand_inter])
print("the current cost is %s" % sol)
route = route_temp
iter_no_improve = 0      # no improve iter set "0"
print(route)

rand_intra = random.randint(2, len(IntraRouteDir))
intraDir_idx = IntraRouteDir[rand_intra]

if intraDir_idx == 'intra_or_opt':
    num = 2
    # num = 3
    route_temp = self.intraRouteOrOpt(route, K, num) # num is a
variable parameter

elif intraDir_idx == 'intra_2_opt':
    route_temp = self.intraRoute2Opt(route, K)
elif intraDir_idx == 'intra_exchange':
    route_temp = self.intraRouteExchange(route, K)
elif intraDir_idx == 'intra_reverse':
    route_temp = self.intraRouteReverse(route, K)
elif intraDir_idx == 'intra_ad_exchange':
    route_temp = self.intraRouteAdExchange(route, K)

if self.CapacityCheck(route_temp, Q) == True:
    # print('capacity check passing...')
    Cost = self.routeCost(route_temp)
    # print("the operator cost is %s" % Cost)
    if Cost < sol:
        sol = Cost
        # If the solution is optimized by the operator, the fraction
increases "1"

        Operator_score[IntraRouteDir[rand_intra]] += 1
        print('intra operator:', IntraRouteDir[rand_intra])
        print("the current cost is %s" % sol)
        route = route_temp
        iter_no_improve = 0  # no improve iter set "0"
        print(route)
    else:
        iter_no_improve += 1

if iter_no_improve >= 500:
    rand_pertur = random.randint(1, len(perturRouteDir))
    perturDir_idx = perturRouteDir[rand_pertur]

    if perturDir_idx == 'route_ejection_chain':
```

```

        route_temp = self.routeEjectionChain(route, K)
    elif perturDir_idx == 'route_bridge':
        route_temp = self.routeDoubleBridge(route, K)
    elif perturDir_idx == 'inter_swap_one':
        route_temp = self.interRouteSwapOne(route, K)
        route_temp = self.interRouteSwapOne(route_temp, K)
    iter_no_improve = 0

    if self.CapacityCheck(route_temp, Q) == True:
        # print('capacity check passing...')
        Cost = self.routeCost(route_temp)
        # print("the operator cost is %s" % Cost)
        if Cost <= 1.2 * sol:
            # sol = Cost
            # If the solution is optimized by the operator, the fraction
            perturbation_score[perturRouteDir[rand_pertur]] += 1
            print('perturbation', perturbation_score, 'operator:',
                  perturRouteDir[rand_pertur])

            print("the current cost is %s" % sol)
            route = route_temp
            iter_no_improve = 0 # no improve iter set "0"
            print(route)

        iter_i += 1

    print('terminal Cost:', sol)
    print('the operator score:', Operator_score)
    print('the perturbation score', perturbation_score)
    tour = route

    return tour

def routeCost(self, route):
    C = self.mData['Cost']
    KS = self.mData['Vehicle']
    Cost = 0
    for k in KS:
        cost = 0
        for i in range(len(route[k]) - 1):
            cost += C[route[k][i]][route[k][i+1]]
        Cost += cost

    return Cost

def CapacityCheck(self, route, Q):
    KS = self.mData['Vehicle']
    D = self.mData['Capacity']

    State = True
    for k in KS:

```

```

        Cap_k = 0
        for e in route[k]:
            Cap_k += D[e]
        if Cap_k <= Q + 30:
            # print('CAP: ', Cap_k)
            continue

        else:
            State = False
            break

    return State

def plot(self, tour, node_x, node_y):
    # print(tour)
    plt.xlabel('x/m')
    plt.ylabel('y/m')
    for k in range(len(tour)):
        list_k = tour[k]
        c = (np.random.rand(), np.random.rand(), np.random.rand())
        for i in range(0, len(list_k) - 1):
            plt.plot([node_x[list_k[i]], node_x[list_k[i + 1]]],
                    [node_y[list_k[i]], node_y[list_k[i + 1]]], color=c,
linewidth=2)
    plt.scatter(node_x, node_y)
    plt.show()

def experiment(self):
    N, K, D, C, Q, node_x, node_y, NS, VS, KS, optSol =
self.data_generation_rand(inst)
    # tour = self.solveBySCIP(N, K, D, C, Q, NS, VS)
    tour = self.solveByNeighborSearch(N, K, D, C, Q, NS, VS, KS, optSol)
    self.plot(tour, node_x, node_y)

if __name__ == '__main__':
    a = randNeighborhoodOrding()
    a.experiment()

```